Вінницький національний технічний університет

Факультет інформаційних технологій та комп'ютерної інженерії

Кафедра програмного забезпечення

Пояснювальна записка

до магістерської кваліфікаційної роботи
магістр
(ступінь вищої освіти)

на тему: «Розробка програмного комплексу для навчання дітей «Time Teacher»

Виконав: студент II курсу, групи 1ПІ-17м
спеціальності
121 - Інженерія програмного забезпечення
(шифр і назва напряму підготовки спеціальності)

Слободяника Д. С.
(прізвище та ініціали)
Керівник ст.викладач, к. т. н. Бабюк Н.П.
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Рецензент: к.т.н., доцент Колесницький О.К.
(прізвище та ініціали)

ВНТУ – 2019

Вінницький національний технічний університет

Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення
Ступінь вищої освіти магістр
Спеціальність 121 - Інженерія програмного забезпечення

ЗАТВЕРДЖУЮ
Завідувач кафедрою ПЗ
_____А. М. Пєтух
"____"_____ ____ р.

З А В Д А Н Н Я
НА МАГІСТЕРСЬКУ КВАЛІФІКАЦІЙНУ РОБОТУ СТУДЕНТУ
Слободянику Денису Сергійовичу

1. Тема роботи: «Розробка програмного комплексу для навчання дітей «Time Teacher»
керівник роботи: к.т.н., ст. викладач кафедри ПЗ Бабюк Н. П. затверджені наказом вищого навчального закладу від "___"_____20__року №__

2. Строк подання студентом роботи _____

3. Вихідні дані до роботи :

Мова програмування back-end – C#
Технологія розробки back-end – ASP.NET Core 2.1
Мова програмування front-end – TypeScript
Технологія розробки front-end – Angular 7
Браузери Chrome 21+, Safari 6.1+, Firefox 22+, Opera 12+, IE 11+, Edge 17+

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити): вступ; аналіз та постановка задачі; розробка архітектури та алгоритмів програмного комплексу; розробка програмного комплексу; тестування програми; економічна частина; висновки; перелік посилань; додатки.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень): Демонстрація роботи аналогів, діаграми варіантів використання, послідовності, класів, структурна карта Константайна, структурна схема компонентів додатку, спроектована база даних, інтерфейс головної сторінки та основних модулів ігрового функціоналу, підбір кольорової гамми, загальний алгоритм роботи та тестування програмного комплексу.

6. Консультанти розділів магістерської кваліфікаційної роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видано	завдання прийнято
1-4	к.т.н. Бабюк Н. П., ст. викладач кафедри ПЗ		
5	к.е.н. Бальзан М. В., доц. кафедри ЕПВМ		

7. Дата видачі завдання_____

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів магістерської кваліфікаційної роботи	Строк виконання етапів роботи	Примітка
1	Аналіз проблеми, обґрунтування актуальності розробки програмного комплексу та постановка задачі		Вик.
2	Розробка архітектури та алгоритмів роботи програмнго комплексу, проектування інтерфейсу		Вик.
3	Вибір мови та технології розробки		Вик.
4	Розробка програмного продукту		Вик.
5	Тестування роботи додатку		Вик.
6	Економічне обгрунтування розробки програмного комплексу		Вик.
7	Оформлення матеріалів до захисту МКР		Вик.

Студент _____ **Слободяник Д.С.**
(підпис) (прізвище та ініціали)

Керівник магістерської кваліфікаційної роботи _____ **Бабюк Н.П.**
(підпис) (прізвище та ініціали)

АНОТАЦІЯ

У магістерській кваліфікаційній роботі спроектовано та розроблено програмний комплекс для навчання дітей «Time Teacher». Програмний комплекс призначений для закріплення знань користувачами пізнання часу, поглиблене вивчення часу на різних мовах світу. Використано групу методів навчання, зокрема програмоване навчання, що розвивають логічні та математичні здібності, а також контролюють та покращують ефективність пізнавання часу користувачами, зокрема дітьми.

Програмний комплекс розроблено за допомогою ASP.NET Core 2.1 (мова програмування – C#) – використано для обробки даних, виконання обчислень, взаємодії з базою даних (MySQL) з використанням Entity Framework Core, а також Angular 7 (мова програмування TypeScript) – використано для візуалізації.

ABSTRACT

In the master's qualification work the «Time Teacher» program complex for teaching children was designed and developed. The program complex is intended for fixing users knowledge’s of time, in-depth studying of time in different languages of the world. A group of teaching methods, programmable learning in particular, that develop logic and mathematical abilities is used, as well as control and improve the efficiency of time recognition by users, including children.

The software package was developed using ASP.NET Core 2.1 (programming language - C #) - used for data processing, computing, database collaboration (MySQL) using Entity Framework Core, and Angular 7 (TypeScript programming language) - used for visualization.

ЗМІСТ

ВСТУП 8
1. АНАЛІЗ ТА ПОСТАНОВКА ЗАДАЧІ 11
1.1 Аналіз стану проблеми 11
1.2 Порівняльний аналіз аналогів 24
1.3 Постановка задачі 27
1.4 Висновки 30
2 РОЗРОБКА АРХІТЕКТУРИ ТА АЛГОРИТМІВ ПРОГРАМНОГО КОМПЛЕКСУ 31
2.1 Розробка загальної архітектури додатку 31
2.2 Проектування бази даних програмного комплексу 36
2.3 Розробка інтерфейсу програмного комплексу 39
2.4 Вибір кольорової гами 47
2.5 Розробка блок-схеми алгоритму роботи програмного комплексу 50
2.6 Висновки 52
3 РОЗРОБКА ПРОГРАМНОГО КОМПЛЕКСУ 53
3.1 Аналіз вибору мови програмування та технологію розробки 53
3.2 Програмна реалізація 64
3.3 Висновки 72
4 ТЕСТУВАННЯ ПРОГРАМИ 74
4.1 Методи тестування 74
4.2 Інструкція тестування програми 77
4.3 Висновки 83
5 ЕКОНОМІЧНА ЧАСТИНА 84
5.1 Оцінювання комерційного потенціалу розробки 84
5.2 Прогнозування витрат на виконання науково-дослідної та конструкторсько–технологічної роботи. 85
5.3 Прогнозування комерційних ефектів від реалізації результатів розробки. 89
5.4 Розрахунок ефективності вкладених інвестицій та період їх окупності 90
5.5 Висновки 94

ВИСНОВКИ 95
ПЕРЕЛІК ПОСИЛАНЬ 97
ДОДАТКИ 100
Додаток А. Технічне завдання 101
Додаток Б. Лістинг програми 106
Додаток В. Зображення інтерфейсу ігрового процесу програмного комплексу 136
Додаток Г. Ілюстративний матеріал до захисту магістерської кваліфікаційної роботи 140

ВСТУП

Вважається, що для дитячого розвитку важливою є гра. Як наслідок впровадження сучасних технологій, дитячі ігри стають комп’ютеризованими. Важливою складовою виховання дітей стає навчання комп’ютерній грамотності. Є корисним, якщо дитина змалечку починає вивчати основи роботи за комп’ютером. Ранній розвиток дозволяє швидко знаходити потрібну дитині інформацію, працювати зі складними документами, відправляти листи електронною поштою. Проте, навчаючись, дитині звісно ж заманеться грати в комп’ютерні ігри.

Комп'ютерні ігри особливу роль відіграють у розвитку дитини, оскільки можуть допомагати у розвитку багатьох навичок. Мова йде про розвивальні ігри, з допомогою яких дитина навчається. Ігри бувають різних видів, що розраховані на різний вік та характер дітей. Гра тренує ряд навичок [1]: бути зібраним в потрібні моменти; запам’ятовувати великий обсяг інформації; розвивати логічне та образне мислення; розвивати просторове мислення та часові уявлення.

Проблема часу для людини є буденною, оскільки все її життя тісно пов’язане з часом, з умінням розподіляти, вимірювати і цінувати час. Час є регулятором людської діяльності. Особливості часу важко сприймаються дітьми. Уявлення часу розвиваються повільно, в процесі нагромадження досвіду, тривалих спостережень, вивчення різних величин. Початкові часові уявлення діти отримують в дошкільний період. Зміна пори року, зміна дня і ночі, повторення різних моментів у дитячому житті – все формує її часові уявлення. Проте часова послідовність подій та поняття про тривалість подій засвоюється дітьми з великими труднощами.

Також важливо у наш час орієнтуватися у часі згідно міжнародних стандартів, зокрема, відповідно правил англійської мови. Багато хто відкладає вивчення часу відповідно правил англійської мови на потім, а насправді вона

необхідне, адже в розмовній мові постійно запитують час один у одного. Англійська мова є найбільш поширена, проте також важливим є знання часу і на інших мовах світу. Крім того, вивчення часу і на інших мовах позитивно впливає на розвиток дитини. Отже, розробка програмного комплексу, що складається з ігрового підходу до вирішення проблеми пізняння користувачів часу є актуальною.

Метою магістерської кваліфікаційної роботи є покращення існуючих методів у вирішенні проблеми вивчення та засвоєння дітьми пізнавання часу шляхом удосконалення дитячої комп'ютерної гри «TeachTime» та розробки на її основі програмного комплексу «Time Teacher».

Наукова новизна полягає у вдосконаленні методів для швидкого пізнання часу дітьми, застосуванні методу вивчення часу на різних мовах світу, зокрема, англійської. Також використано групу методів навчання, зокрема програмоване навчання, що спрямовані на: стимулювання й мотивацію до навчально-пізнавальної діяльності; контроль та корекцію ефективності пізнавання часу дітьми.

Об'єктом дослідження є процес розробки програмного комплексу з ігровим функціоналом та системи обробки результатів.

Предметом дослідження є методи і засоби розробки програмного комплексу «Time Teacher».

Практичне значення розробленого програмного засобу полягає у допомозі дітям засвоїти отримані знання із пізнання часу.

У першому розділі проаналізовано предметну область. Розглянуто методи вирішення проблеми навчання дітей пізнання часу в навчальних закладах та три найбільш типових гри. В результаті, виявлено доцільність розробки програмного комплексу «Time Teacher». Розроблено структуру програмного комплексу, сценарій ігрового процесу, основні задачі по розробці «Time Teacher».

У другому розділі розроблено загальну архітектуру додатку за допомогою відповідних діаграм варіантів використання, послідовності, класів, структурної

карти Константайна, структурної схеми компонентів додатку. Спроектовано базу даних програмного комплексу, графічні схеми інтерфейсу головної сторінки та основних модулів ігрового процесу, виконано підбір кольорової гамми інтерфейсу, а також розроблено загальний алгоритм роботи комплексу.

У третьому розділі проведено аналіз мов програмування та технології розробки front-end, back-end програмного комплексу Time Teacher. Обрано C# (ASP.NET Core) для розробки back-end та TypeScript (Angular 7) для розробки front-end. Розглянуто реалізацію ключового функціоналу додатку.

У четвертому розділі проведено тестування модулів ігрового процесу програмного комплексу. В результаті виконання тестування помилок не було виявлено.

У п’ятому розділі проведено економічне обгрунтування розробки програмного комплексу «Time Teacher». Обрахувавши термін окупності наукової розробки (0,84 року) досліджено, що фінансування наукової розробки є доцільним.

Результати досліджень у кваліфікаційній роботі доповідались та обговорювались на таких конференціях:

1. XLVI Науково–технічна конференція підрозділів Вінницького національного технічного університету (2017) [2].

2. Міжнародна науково–практична Інтернет–конференція «Молодь в технічних науках: дослідження, проблеми, перспективи» (2017) [3].

Отримано свідоцтво про авторське право на твір «Дитяча комп'ютерна гра «TeachTime» №73041.

1. АНАЛІЗ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Аналіз стану проблеми

Життя кожної людини пов’язано з часом, умінням його вимірювати, розподіляти та цінувати. Час не можна ні повернути, ні спинити, тому важко сприймати його проміжки за тривалістю, порівнювати події. Сприйняття часу недосконале: інколи здається, що час збігає то повільніше, то швидше, залежно від того, чим людина займається. Тому час – одна із важких для вивчення величин.

Послідовність подій у часі, особливості уявлення тривалості подій засвоюються дітьми з великими труднощами. Зазвичай, є помилки щодо послідовності подій (наприклад, плутаються поняття «завтра» і «вчора») [4].

Розглянемо діючі програми покращення вмінь пізнання часу дітьми різних вікових категорій.

У шість років необхідно ознайомити дітей з одиницями часу: секундою, хвилиною, годиною. Покращується часове пізнання під час виконання практичних завдань, коли діти повинні виконувати їх у відведений час, планувати і розраховувати дії, орієнтуючись за годинником.

При навчанні дітей старшого дошкільного віку потрібно орієнтуватися на наповнення освітньої роботи з дітьми, акцентує увагу на ігровій діяльності – провідній діяльності дошкільного дитинства, ключовому засобі розвитку дітей. Основний програмовий зміст упорядковано за розділами, що розглянемо.

Розділ «Пізнавальний розвиток» це завдання, необхідні розвитку пізнавальних інтересів, здібностей, операцій мислення та психічних процесів, стимуляції самостійної активності, що є основою успіху в подальшому навчанні. Завдання містять логіко-математичний та загально-пізнавальний аспекти. Вони згруповані за напрямками: «У світі природи», «У світі предметів», «У світі людей», «У світі чисел і цифр», «У світі форм і величин», «У світі простору і часу».

За напрямком «У світі простору і часу», під час навчання діти у підготовчій групі повинні мати наступні елементарні часові уявлення:

1. Розрізняти просторове розташування предметів та визначати напрямок руху.

2. Визначати відстань, відрізняти поняття: близько, далеко, подалі, поруч.

3. Володіти вмінням визначати розташування предметів відносно будь-якого предмета чи відносно себе за просторовим розміщенням на площині.

4. Відрізняти та вірно вживати поняття часу: згодом, пізніше, зараз, раніше, вчора, сьогодні, завтра тощо.

5. Формувати та покращувати знання одиниць часу: рік (пори), місяць (назви 12 місяців, їх послідовність), тиждень (послідовність та назви днів тижня), доба, година, хвилина.

6. Ознайомлюватись з різними видами календарів та годинників та вчити користуватися ними.

У програмі математики загальноосвітніх навчальних закладів (1-4 класи) визначено наступні державні вимоги рівня загальноосвітньої підготовки учнів: учень знає одиниці часу та їх співвідношення; вміє користуватись календарем; визначає час за годинником з точністю до хвилини; скорочено записує одиниці відповідних величин; виконує арифметичні дії з одиницями вимірювання часу; замінює крупні одиниці вимірювання часу дрібними і навпаки .

Учні молодшого шкільного віку починають пізнавати і сприймати, в основному на рівні почуттів, певні властивості часу: необоротність, плинність, періодичність.

Реальний зміст властивостей часу засвоюється під час шкільного навчання, але в перші три роки життя в дитини формується загальне пізнання часу, чуттєві сприйняття його тривалості.

Сприйняття часу спирається, насамперед, на м’язові та слухові відчуття. Спілкуючись з дорослими, дитина під впливом спеціального навчання

оволодіває словником, який характеризує різні часові відрізки, частини доби, пори року, а також словником, який відображає плинність часу.

Хоч часове сприйняття в ранньому віці складне, але діти вже розрізняють і правильно вживають дієслова в минулому, теперішньому та майбутньому часі (ми ходили, йдемо, підемо). Повинні бути сформовані уявлення про час (сьогодні, завтра, потім, зараз, ранок, день, вечір). Формуванню уявлень сприяє чіткий режим дня, занять тощо.

Спочатку молодші школярі розуміють час за подіями, що відбулися з кожним із них безпосередньо за день і викликали сильні емоції. Поступово вони починають пов'язувати час з подіями, які стались у навколишній дійсності. Ознайомлення дітей з частинами доби слід починати з контрастних відрізків: ранок – вечір; день – ніч.

Дітей шести років навчають оцінювати послідовність дій: була, є, буде, зараз, після, раніш, пізніше тощо. В результаті навчання в дітей формуються навички розуміти і правильно позначати тривалість часу, визначати послідовність логічно пов'язаних подій на зрозумілих сюжетах.

Після цілеспрямованої роботи у дітей формуються поняття про деякі властивості часу:

1. Об'єктивність – неможливість прискорити або уповільнити його.
2. Необоротність – не можна повернути день вчорашній, бо час завжди йде вперед.

Оволодінню учнями умінь розрізняти, вказувати частини доби і враховувати їхню послідовність характерні наступні особливості:

1. Нерівномірність у розумінні назв частин доби.
2. Співвідношення показників частин доби з власним досвідом життя і діяльності.
3. Виділення частин доби, що частіше називаються дорослими, мають конкретні ознаки і зв'язані з характерними видами діяльності.
4. Визначення послідовності частин доби, як правило, починаючи з ранку.

Учні сьомого року життя ознайомлюються з одиницями часу: годиною, хвилиною, секундою. Покращується часове орієнтування. Виконуючи практичні вправи з моделями годинника, учні навчаються визначати час за годинником.

Знайомство дітей з часовими одиницями вимірювання необхідно здійснювати в строгій системі і послідовності, де знання одних інтервалів часу, можливість їхнього визначення і виміру були б основою для ознайомлення з наступними і розкривали дітям істотні характеристики часу: його безперервність, плинність, необоротність.

Зміст та способи формування часових уявлень були предметом дискусій, що пов'язувались з розробкою різних психологічних засад методики формування математичних уявлень молодших школярів (М. Немировська, К. Лебединцев, Л. Виготський, Г. Леушина, Н. Менчинська, М. Макляк) [4].

Мета формування уявлень про час полягає в навчанні оптимальних способів виконання дій над іменованими числами, формуванні відповідних умінь та навичок (Г. Костюк, Г. Леушина, Т. Ріхтерман, А. Столяр, С. Рубінштейн).

Вивчення англійської мови починається ще з ранніх років життя, оскільки набирає трендів подорож за кордон, знайомство з новими людьми, пошук цікавих місць по всьому світу. І вивчення часу відповідно до правил англійської мови є частиною програми більшості загальноосвітніх шкіл України. Зрозуміло, що вивчення часу і так не є простим, а ще й англійська мова додає складнощів у розумінні часу. Проте, якщо ґрунтовно розібратися з матеріалом, а також з особливостями мови, то не виникне жодних труднощів в освоєнні орієнтації в часі іноземною мовою.

Є ряд правил, які потрібно першочергово зрозуміти та запам'ятати при вивченні часу відповідно правил англійської мови [4]:

1. Числівники англійської мови.

2. В англійській мові, зазвичай, не вживають в розмовах 24 годинний час, а зазвичай використовують 12 годинні позначення. Щоб не плутатися, вони

застосовують скорочення. Час з 00 до 12 позначають «AM». А час з 12 до 00 годин «PM».

3. На початку речення ставити «it's» .

4. До половини години потрібно використовувати «past» , а після половини – «to».

5. Замість слова «п'ятнадцять» треба використовувати «quarter» (чверть).

6. Слово «minutes» (хвилини) в англійській мові зазвичай опускається.

Також, замість «AM» можна використовувати вираз – «in the morning», а замість «PM» використовувати «in the evening». Наприклад, 05.15 (п'ятнадцять хвилин на шосту ранку) – «it's a quarter past five AM» або «it's a quarter past five in the morning».

Варто також згадати, що в англійській мові, зазвичай, вживаються скорочення, тому час буде вживатися без «it's» на початку, а «AM», «PM» і фрази «in the morning», «in the evening» вживається тільки тоді, коли без цього не буде зрозуміло. Тобто, на прикладі, на питання «When do you go to bed?» – «Ти коли спати лягаєш?», відповідь – «at ten» – «в десять» – не потребує додання «вечора», тому що це і так зрозуміло.

В американському англійському замість «past» досить часто використовується прийменник «after». А замість прийменника «to» можливе використання прийменників «of», «before» і «till».

Крім того, також важливим є вивчення часу на будь-якій мові світу, адже ці знання можуть стати доречними у відповідних ситуаціях. Англійська мова – одна з ключових, адже вона найбільш поширена, проте і інші мови теж є важливими.

Отже, час – дійсно одна з найважчих для вивчення величин. Часові уявлення в дітей розвиваються повільно. Ознайомлюючи учнів початкових класів з часовими одиницями, часто на уроках використовується дуже багато різноманітної наочності – це годинники-циферблати і картини, різні види годинників, таблиці, схеми, секундоміри, табель-календар. Використовуються

на уроках читання, бесіди, переказування цікавих фактів, віршів, перегляд картин–ілюстрацій; проводяться дидактичні ігри, розумові розминки.

Освіта XXI століття спрямована на формування людини нового типу, готової жити в інформаційному суспільстві, розв’язувати нестандартні ситуації, реагувати на стрімкий технічний прогрес. Відтак, нові завдання стоять і перед дітьми.

При вивченні часу доцільно застосувати програмоване навчання. Програмоване навчання — це різновидність репродуктивного підходу до навчання, коли передача змісту матеріалу здійснюється невеликими, логічно завершеними дозами. Після кожної дози здійснюється контроль, після чого повідомляється наступна доза. Інформація часто йде від програмованого посібника.

При лінійному програмуванні: інформація ділиться на дози, програма включає підказки, вказівки, учень дізнається, чи правильна відповідь, помилки виключені; учень задоволений успіхом; учні засвоюють один і той же матеріал, але в різному темпі.

При розгалуженому програмуванні: матеріал ділиться на дози, до кожної з яких дається завдання з кількома відповідями, учень вибирає правильну відповідь, потім приступає до наступної дози інформації. Якщо відповідь неправильна, учень доучується.

Позитивні сторони програмного навчання: виділяється головне, істотне в навчальному матеріалі, забезпечується оперативний контроль знань; матеріал подається в логічній послідовності; можливий самоконтроль, швидкий темп; є можливість для індивідуалізації навчання. Недоліком програмованого навчання є те, що роль учителя зводиться до інструктора і відсутня можливість для розвитку творчості учнів.

Різновидністю програмованого навчання є алгоритмізація, при якій будується модель розумових процесів, послідовність розумових дій, операцій. Наприклад, можна навчити учнів алгоритму роздумів при розв'язуванні задач

певного класу. Так, при розв'язанні задач із динаміки потрібно дотримуватися таких правил:

1. Виявити всі сили, які діють на тіло.

2. Скласти їх, маючи на увазі, що ця сума дорівнює прискоренню.

3. Скласти рівняння, знайти невідомі.

Застосування алгоритмічних принципів допомагає учням швидко розв'язувати задачі за допомогою раціональних розумових дій. Проте не можна побудувати алгоритм для кола завдань, тому не слід обмежуватися лише алгоритмізацією.

Знайти засоби для розвитку сучасних дітей не важко. Потрібно лише говорити з малятами мовою, яка найбільш їм зрозуміла, на яку діти найкраще відгукуються – мовою гри, фантазії, уяви, а ще – мовою сучасних гаджетів. Використання в діяльності педагога дошкільного навчального закладу комп’ютера, мультимедіа та інших технічних засобів дозволяють розширити можливості фахівця щодо розвитку інтелектуальної сфери, творчих здібностей дитини, формування її особистісних та морально-вольових якостей.

Малята із задоволенням працюють на заняттях, де використовуються електронні ігри, активно включаються у виконання завдань, адже технічні засоби викликають інтерес і стійку увагу дітей. Крім того, вони сприяють посиленню мотивації, розвитку творчих здібностей дітей, створенню благополучного емоційного фону, а також допомагають індивідуалізації освітнього процесу.

Звичайно, комп’ютерні ігри в жодному разі не замінюють звичайних ігор і занять, але за умови їх доцільного та дозованого використання – доповнюють їх, збагачуючи розвивальний процес новими можливостями. Отже, електронна гра може стати потужним «інтелектуальним знаряддям» у руках досвідченого та творчого педагога.

Сьогодні виграє той педагогічний працівник, який не лише може дати базові знання дитині, а й спрямувати її на самостійне оволодіння ними. Для розвитку у малюків стійкого пізнавального інтересу педагог має докладати

неабияких зусиль. Перед ним стоїть завдання зробити заняття цікавим і насиченим інформацією, яка спонукала б дітей до активної пізнавальної діяльності. Тобто пропонований матеріал для пізнання має містити елементи надзвичайної, дивовижної, несподіваної інформації. Такої, що викликає у дошкільників інтерес до освітнього процесу і допомагає створити позитивну емоційну атмосферу навчання. Лише такий підхід до організації освітнього процесу сприятиме розвитку розумових здібностей дітей. Адже саме здивування, інтерес будить думку й веде до розуміння.

За даними ЮНЕСКО, коли людина слухає, вона запам'ятовує 15% мовної інформації, коли дивиться – 25% видимої інформації, коли бачить і слухає – 65% усієї інформації.

Використання комп'ютерних ігор у освітній діяльності є природним для дитини і слугує ефективним засобом підвищення мотивації і забезпечення індивідуалізації навчання та розвитку особистісних здібностей. У грі дитина оперує своїми знаннями, досвідом, враженнями, що відображаються у формі ігрових способів дії, ігрових знаків, які набуває в смисловому полі гри.

Наукові дослідження щодо застосування ІКТ в освіті дітей дошкільного віку, які проводила у Франції протягом декількох років Асоціація педагогічних фахівців «Комп'ютер і дитинство» у співдружності з ученими багатьох науково-дослідних інститутів Франції, показали, що мультимедійний спосіб подання інформації сприяє:

1. Легшому і глибшому осягненню дітьми понять.
2. Швидкому формуванню вмінь орієнтуватися на площині і у просторі.
3. Підвищенню ефективності уваги і поліпшенню пам`яті.
4. Оволодінню навичками читання.
5. Активному поповненню словникового запасу.
6. Розвитку дрібної моторики рук.
7. Формуванню чіткої координації рухів очей.
8. Зменшення часу як на просту реакцію, так і на реакцію вибору.
9. Вихованню цілеспрямованості і зосереджуваності.

10. Удосконаленню уяви та творчих здібностей.

11. Розвитку елементів наочно-образного мислення.

У комп’ютерних іграх присутні елементи новизни, сюрпризності, незвичності, в них використовуються засоби заохочення, що так полюбляють діти. Застосування комп’ютерів, мультимедійних інформаційних технологій у якості дидактичних засобів використовуються для підвищення мотивації та індивідуалізації навчання, розвитку творчих здібностей дітей та для створення благополучного емоційного фону.

Використання електронних ігор у навчанні не тільки збільшує швидкість передачі інформації дітям та підвищує рівень її засвоєння, але й сприяє розвитку таких психічних процесів як увага, пам’ять, мислення, мовлення, розвиває почуття кольору, композиції, бере участь в інтелектуальному, емоційному та моральному розвитку дітей.

Розвиваючі комп’ютерні ігри можуть бути дуже різноманітними, розрахованими на різний вік і характер дитини. Певна гра тренує якісь конкретні навички, тому потрібно уважно продумати гру, в залежності від завдання, яке хоче поставити психолог перед конкретною дитиною чи групою дошкільників.

Під час створення комп’ютерних ігор важливо дотримуватися дидактичних принципів: вибирати ігри помірної складності, поступово пропонувати більш важкі завдання. Для рухливих непосидючих дітей краще готувати ігри коротші й динамічніші. Спокійним дітям можна пропонувати більш тривалі ігри.

Щоб комп’ютер не становив загрози для дитини, перед початком гри необхідно встановити обов’язкові правила, своєрідну техніку безпеки для дітей. Зокрема, не слід дозволяти дитині сидіти за комп’ютером більше 10-15 хв, не грати в комп’ютерні ігри безпосередньо перед денним сном. Якщо ви помічаєте, що дитина надмірно збуджується від якоїсь гри, краще «відмінити» її. Крім того, щоб зрозуміти, наскільки гра підходить для задач, які ви ставите, потрібно спочатку програти в неї самому.

Комп’ютерні ігри мають подвійний зміст:

1. Ігровий – що передбачає реалізацію творчого задуму та одержання дитиною винагороди.

2. Навчальний – придбання знань, умінь і навичок за допомогою діяльності за заданими правилами.

У процесі гри дитина подорожує за певним сюжетом, що супроводжується яскравими зображеннями відомих дитині персонажів, завдання з кожним кроком ускладнюються. Дитину це захоплює, їй цікаво, що буде далі, вона хоче виграти. Саме завдяки цьому малюк стає зосередженим і уважним. Крім того, під час виконання завдань, дошкільник вчиться контролювати, аналізувати свої дії, виправляти помилки.

Заняття – групові та індивідуальні – проводяться в ігровій формі. Зміст ігрових завдань направлений на підвищення впевненості, розвиток навичок саморегуляції емоційних станів, розвиток комунікативних навичок дитини. Крім того, комп’ютерні ігри вчать дітей переборювати труднощі. Дошкільник входить у сюжет гри, засвоює правила, відповідно діє і намагається досягнення результатів. Таким чином, комп’ютер допомагає розвивати не лише інтелектуальні здібності дитини, але й виховувати вольові якості: самостійність, зібраність, зосередженість, посидючість. Під час таких занять завдання педагога — допомогти малюкові подолати негативні емоції, нерішучості, хвилювання, сприяти створенню позитивного настрою.

Комп’ютерні ігри створені так, що дитина може собі уявити окремий предмет або конкретну ситуацію, одержати узагальнене поняття про всі схожі предмети або ситуації. У такий спосіб у дитини розвиваються такі важливі операції мислення як узагальнення і класифікація.

Застосування на корекційно-розвивальному занятті комп’ютерних ігор допомагає дитині зрозуміти, що на екрані присутнє зображення речей, а не реальні речі. То ж, у дітей починає розвиватися так звана знакова функція свідомості, тобто розуміння того, що є кілька рівнів навколишнього середовища — це реальні речі, і картинки, й схеми, слова або числа тощо.

Також комп’ютер просто незамінний для формування довільності психічних процесів, адже передає інформацію у привабливій для дитини формі, що не тільки прискорює запам’ятовування матеріалу, але й робить його осмисленим і довгостроковим.

Заняття дітей на комп’ютері мають велике значення не тільки для розвитку інтелекту, але й для розвитку їхньої дрібної моторики. У будь-яких іграх, від найпростіших до складних, дітям необхідно вчитися натискати пальцями на певні клавіші або користуватися тачпадом (сенсорною панеллю), що розвиває дрібну моторику. Вчені зазначають, що чим більше робиться дрібних і складних рухів пальцями, тим більше ділянок мозку включається в роботу.

Важливою також є робота зорових аналізаторів та відповідних ділянок у корі головного мозку. Чим уважніше ми вдивляємося в те, над чим працюємо, тим ефективнішим буде процес запам’ятовування. Ось чому так важливо формувати моторну координацію і координацію спільної діяльності зорового й моторного аналізаторів, що з успіхом досягається на заняттях, де діти працюють з комп’ютером.

Досвід показує, що спілкування з гаджетами викликає в дітей жвавий інтерес. Цей інтерес лежить в основі формування таких важливих структур, як пізнавальна мотивація, довільні пам’ять й увага. Незаперечним є той факт, що саме ці якості забезпечують психологічну готовність дошкільників до навчання в школі.

Новизна роботи з комп’ютером сама по собі сприяє підвищенню інтересу до навчання, а можливість регулювати навчальні завдання за ступенем складності позитивно позначаються на мотивації.

Крім того, комп’ютер дозволяє повністю усунути одну з найважливіших причин негативного ставлення дітей до навчання – проблеми нерозуміння матеріалу, адже новий матеріал подається в інтерактивній формі, що забезпечує та зберігає стійкий інтерес та увагу дітей [5].

Встановлено, що ігрова діяльність є провідним засобом соціальної адаптації дітей, що впливає на такі дезадаптуючі психічні стани, як тривожність, емоційна напруженість, депресія, психічна й соматична втомлюваність. Впливаючи на мотиваційно-потребну сферу, ігрова діяльність задовольняє потреби в самовираженні, у виході з напруженої соціальної ситуації, у підтриманні соціальних зв'язків [6].

Дитина у грі, яка для неї є провідним видом діяльності, оволодіває людською діяльністю, її ігрові дії завжди реальні та соціальні, бо спрямовані на реалізацію узагальненої соціальної функції дорослих, набуття досвіду, який трансформується із зовнішнього у внутрішній план психічної діяльності, визначає її розвиток як суб’єкта предметної діяльності. Під впливом соціально-психологічних феноменів: ефекту зараження, механізму звикання, вербальної орієнтації, позитивного підкріплення, відбувається переведення об’єктивних взаємин, які виникають у процесі ігрової діяльності дітей у норми соціальної моралі та поведінки [7].

Тобто гра допомагає дитині розвивати здібності до спостережень, навчає дітей виражати своє ставлення до інших людей, формувати ставдення до інших однолітків.

Розповсюджені думки щодо ігор:

1. Розвивають координацію та зосередженість; кмітливість; логічне мислення; швидкість прийняття рішень.
2. Підвищують агресивність.
3. Знижують поріг чутливості до вбивства.
4. Формують залежність.
5. Сприяють відстороненню від суспільного життя.
6. Викликають депресію і відчуття одинокості.
7. Знижують здатність до співпереживання.
8. Призводять до погіршення комунікативних навичок.
9. Консервують розвиток мови.

Основні причини популярності комп’ютерних ігор:

1. Спосіб реалізації інфантильної поведінки (позачовість, залежність від іншої думки).

2. Небажання ставати дорослим, довга соціалізація, прагнення безпеки.

3. Схожість комп'ютерних ігор з уявою, що відіграють роль дитячої гри.

4. Можливість задовольнити творчі здібності та спілкування. Відсутність можливості дітьми керувати своєю реальним життям.

5. Потреба в нових знаннях, нових зразках поведінки.

6. Відсутність досвіду управління своїм власним життям.

Висновки щодо ігор:

1. 40% учнів грають в ігри 1-2 рази на тиждень.

2. 11% учнів грають щодня.

3. Комп'ютерні ігри не мають безпосередньо негативного впливу на когнітивні установки підлітків.

4. Ігри впливають на розсудливість, більший інтерес до себе, гнучкістю установок, покращення логічного мислення, соціальну адаптацію.

5. Підвищення агресивності підлітків не завжди залежить від комп'ютерних ігор, а швидше залежать від психічних якостей та схильності підлітків.

6. Ігри з агресивним змістом здатні стимулювати агресивність у дітей, але тільки молодшого шкільного віку (6-9 років).

Відомо, що надмірне використання комп’ютерних ігор можуть викликати у дітей залежність. Основні ознаки залежності:

1. Поява почуття радості, ейфорії при контакті з комп'ютером або ігровою приставкою, а також в передчутті контакту.

2. Відсутність контролю за часом взаємодії з комп'ютером або ігровою приставкою, втрата відчуття часу.

3. Бажання збільшувати час взаємодії з комп'ютером або ігровою приставкою.

4. Поява почуття роздратування, гнівливості або пригнічення, порожнечі, депресії при відсутності контакту з комп'ютером чи ігровою приставкою.

5. Використання комп'ютера, ігрової приставки для зняття внутрішньої напруги, тривоги, депресії, тобто взагалі для поліпшення настрою.

6. Поява проблем у взаєминах з батьками, в школі тощо.

7. Зміни в інших видах поведінки (поява байдужості до того, що раніше хвилювало, недбалість у виконанні справ тощо).

Варто зазначити, що в іграх є як переваги, так і недоліки, і кожна з вище згаданих думок має право на існування. Проте, з ігор можна отримати максимум користі при правильному підході до їх використання.

Є багато методик по вивченню часу дітьми, проте подання матеріалу в ігровій формі сприятиме закріпленню отриманих знань в пізнанні часу дітьми. Використання комп’ютерних ігор в роботі педагога є не лише цікавим, але й дуже ефективним методом у підготовці дітей до шкільного навчання.

Зокрема, у результаті впровадження програмного комплексу «Time Teacher» з комп’ютерною грою можна очікувати такі результати:

1. Активізація пізнавальної діяльності дітей.
2. Покращення показників обізнаності матеріалу.
3. Позитивні зміни у емоційно-вольовій сфері дітей.

Отже, враховуючи викладеного вище, слід зазначити, що комп’ютерні ігри є потужним ресурсом для розвитку, навчання й виховання дітей і розробка програмного комплексу «Time Teacher» є доцільною.

1.2 Порівняльний аналіз аналогів

Проблема покращення пізнання дітей у часі не є новою. Існує ряд готових реалізацій, направлених на вирішення проблеми. Розглянемо кілька з них.

«Годинник із зозулею» – додаток, основна мета якого є вказати відповідний час, представлений на годиннику, у числовому вигляді. Мета –

вказати за певний проміжок часу максимум правильних відповідей. При кожній правильній відповіді з’являється зозуля у відповідній області [8].

Процес гри зображено на рисунку 1.1.



Рисунок 1.1 – Гра «Годинник із зозулею»

«Вивчаємо час» – додаток, основна мета якого є вказати розташування позицій чисел на годиннику. Мета – вказати всі правильні позиції розташування чисел [9].

Процес гри зображено на рисунку 1.2.



Рисунок 1.2 – Гра «Вивчаємо час»

«READING the clock» – додаток, основна мета якого є вказати розташування позицій стрілок на годиннику відповідно до заданого числового

значення. Мета – вказати за певний проміжок часу максимум правильних відповідей [10].

Процес гри зображено на рисунку 1.3.

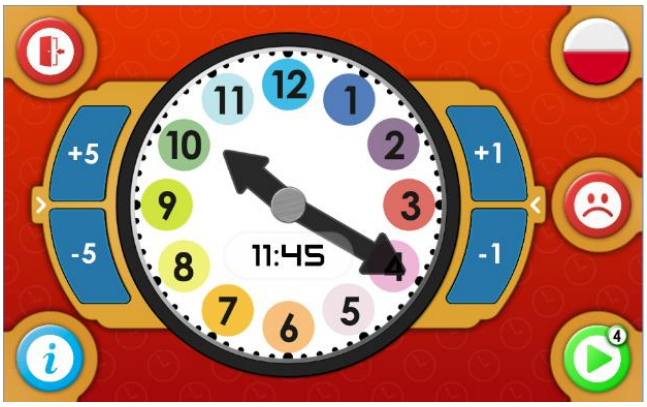


Рисунок 1.3 – Гра «READING the clock»

Розглянуто особливості ігрового процесу кожного з додатків. Визначено, що кожен з додатків має свій підхід до вирішення поставленої проблеми. Вирішено, що потрібно об’єднати принципи в один додаток для освоєння різних аспектів вивчення часу. Також розроблено власний метод вирішення проблеми – введення часу відповідно до правил мов світу.

«Time Teacher» є складною програмною системою та модифікацією дитячої комп’ютерної гри «TeachTime», що покладено за основу для розробки системи. Основний ігровий функціонал гри «TeachTime» перенесено у програмний комплекс та модифіковано.

Загальне порівння аналогів та ігрової частини програмного комплексу «Time Teacher» наведено у таблиці 1.1.

Таблиця 1.1 – Порівняння ігрового функціоналу «Time Teacher» з аналогами

	«Годинник із зозулею»	«Вивчаємо час»	«READING the clock»	«Time Teacher»
Перетягування арабських чисел на відповідні позиції годинника	–	+ (1.0)	–	+ (1.0)
Введення значення часу по відповідному розташуванню стрілок	+ (0.9)	–	–	+ (0.9)
Маніпуляції зі стрілками годинника.	–	–	+ (0.8)	+ (0.8)
Введення часу відповідно до правил мов світу	–	–	–	+ (1.0)
Загальна сума	0.9	1.0	0.8	3.6

Проаналізувавши таблицю 1.1, а також підсумувавши коефіцієнти у кожному додатку можна зробити висновок, що найбільше значення «3.6» у ігровій частині функціоналу програмного комплексу «Time Teacher», оскільки він поєднує функціонал усіх додатків. Тому розробка програмного комплексу є доцільною.

1.3 Постановка задачі

Ігрова частина функціоналу «Time Teacher» призначена для засвоєння знань у пізнанні часу.

Розглядаємо основні сценарії ігрового функціоналу, а також додаткові можливості системи, щоб визначити основні задачі для реалізації програмного комплексу «Time Teacher».

Після відкриття головної сторінки програмного комплексу «Time Teacher» користувачу надається можливість вибрати режим гри, який він хоче проходити. Основними режимами є:

1. «DragNumbers» – перетягування арабських чисел на відповідні позиції годинника.
2. «TimeArrow» – введення значення часу по відповідному розташуванню стрілок.
3. «ArrowManipulation» – маніпуляції зі стрілками годинника.
4. «LanguageTime» – введення часу в реченні відповідно до правил обраної мови, зокрема англійської.

Розглядаємо основні режими детальніше.

Характерним для кожного режиму є гра з часовим обмеженням, надання кількох спроб відповіді на поставлене завдання. Гра завершується, коли відповідне завдання виконано, або ж закінчено кількість спроб на помилку. В результаті, дані записуються в базу даних для фікмації результатів проходження гри.

Особливості кожного з режимів:

1. Перетягування арабських чисел на відповідні позиції годинника**.** Рівень є базовим, принцип якого наступний: на ігровому полі наявний годинник, на якому відсутні числа. Числа розташовані зліва від годинника і за певний проміжок часу користувач перетягує числа на відповідні позиції.

2. Введення значення часу за відповідним розташуванням стрілок**.** На ігровому полі зображено годинник, де випадковим чином задано на годиннику час. Внизу розташовано поле, в яке потрібно ввести час, який зображено на годиннику. Суть гри – вказати максимально можливу кількість вірного часу за обмежений проміжок часу.

3. Маніпуляції зі стрілками годинника**.** На ігровому полі зображено годинник. Випадковим чином задано час і його потрібно зобразити на годиннику, тобто шляхом маніпуляції стрілок вказати правильний час. Суть гри

– вказати максимально можливу кількість вірних відповідей за обмежений проміжок часу.

4. Введення часу відповідно до правил мов світу. На ігровому полі зображено годинник, на якому випадковим чином задано час. Поряд розташовано «блоки» зі словами, що формують речення – як вірно визначається час на обраній, зокрема англійській мові. Потрібно у відповідну область послідовно переносити блоки, щоб сформувати речення. Суть гри – вказати максимально можливу кількість вірних речень по визначенню годин та хвилин за обмежений проміжок часу.

Розглянемо особливості додаткового функціоналу програмного комплексу «Time Teacher». Складається система з двох частин – для користувачів (ігровий функціонал) та для адміністраторів. Адміністратори можуть керувати даними про користувачів системи, проглядати їх результати. Статистику проходження можуть бачити як користувачі, так і адміністратори – графіки прогресу користувача в залежності від режимів. Також, при проходженні одного з режимі гри, відбувається відправка результатів на пошту, вказану при реєстрації.

При запуску гри відкривається вікно авторизації. Користувач може ввійти за своїм персональним логіном та паролем, або ж зареєструватися.

Отже, визначено основні задачі, які необхідно реалізувати у грі «TeachTime»:

1. Авторизація \ реєстрація користувача.
2. Надання можливості вибору режиму гри.
3. Розробка режиму «Перетягування арабських чисел на відповідні позиції годинника».
4. Розробка режиму «Введення значення часу по відповідному розташуванню стрілок».
5. Розробка режиму «Маніпуляції зі стрілками годинника».
6. Розробка режиму «Введення часу відповідно до правил обраної мови».

7. Коректна робота при маніпуляції елементами програмного комплексу.
8. Наведення статистики проходження гри.
9. Розробка системи адміністрування ігрового функціоналу.

1.4 Висновки

В результаті, проаналізовано предметну область та виявлено доцільність розробки програмного комплексу «Time Teacher». Розглянуто методи вирішення проблеми навчання дітей пізнанню часу в навчальних закладах та 3 найбільш популярних гри.

Таким чином, виявлено проблему пізнання часу дітьми та розглянуто основні методи, що використовуються у навчальних закладах для її вирішення. Також враховано, що все більшої популярності набуває вивчення мов світу, і вивчення часу відповідно правил мов позитивно впливає на розвиток дитини. Крім того, метод програмованого навчання сприяє швидкому запам’ятовуванні матеріалу.

Досліджено, що в ігровій формі діти краще сприймають матеріал. Існують ігрові додатки, направлені на вирішення проблеми. Проте додатки не вирішують проблему, тому є необхідність у розробці програмного комплексу «Time Teacher», основою якого є дитяча комп’ютерна гра для пізнавання часу дітьми.

В результаті, виявлено доцільність розробки системи «Time Teacher». Розроблено сценарій гри, основні задачі до розробки програмного комплексу «Time Teacher».

2 РОЗРОБКА АРХІТЕКТУРИ ТА АЛГОРИТМІВ ПРОГРАМНОГО КОМПЛЕКСУ

2.1 Розробка загальної архітектури додатку

Для подальшого проектування програмного комплексу «Time Teacher», є необхідність сформувати його загальну архітектуру за допомогою діаграм варіантів використання, взаємодії об’єктів гри за часом, діаграми класів. Проектування діаграм здійснюється на основі ігрового функціоналу програмого комплексу, описаному в розділі 1.3.

Діаграма варіантів використання гри представлена на на рисунку 2.1.

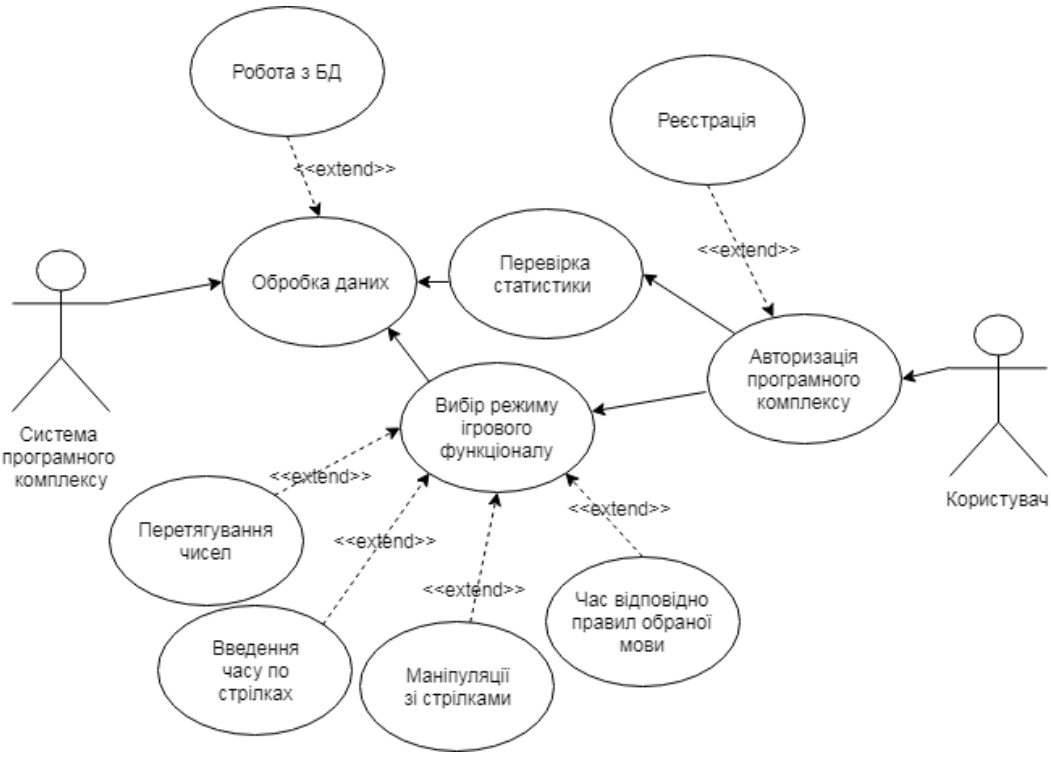


Рисунок 2.1 – Діаграма варіантів використання

Наявні 2 актори – система програмного комплексу та користувач, які виконують властиві для них прецеденти. Особливістю є те, що система здійснює оброблення всіх даних, в залежності від вибору певних дій користувачем.

Взаємодія об'єктів ігрового функціоналу програмного комплексу за часом представлено на діаграмі послідовності на рисунку 2.2.

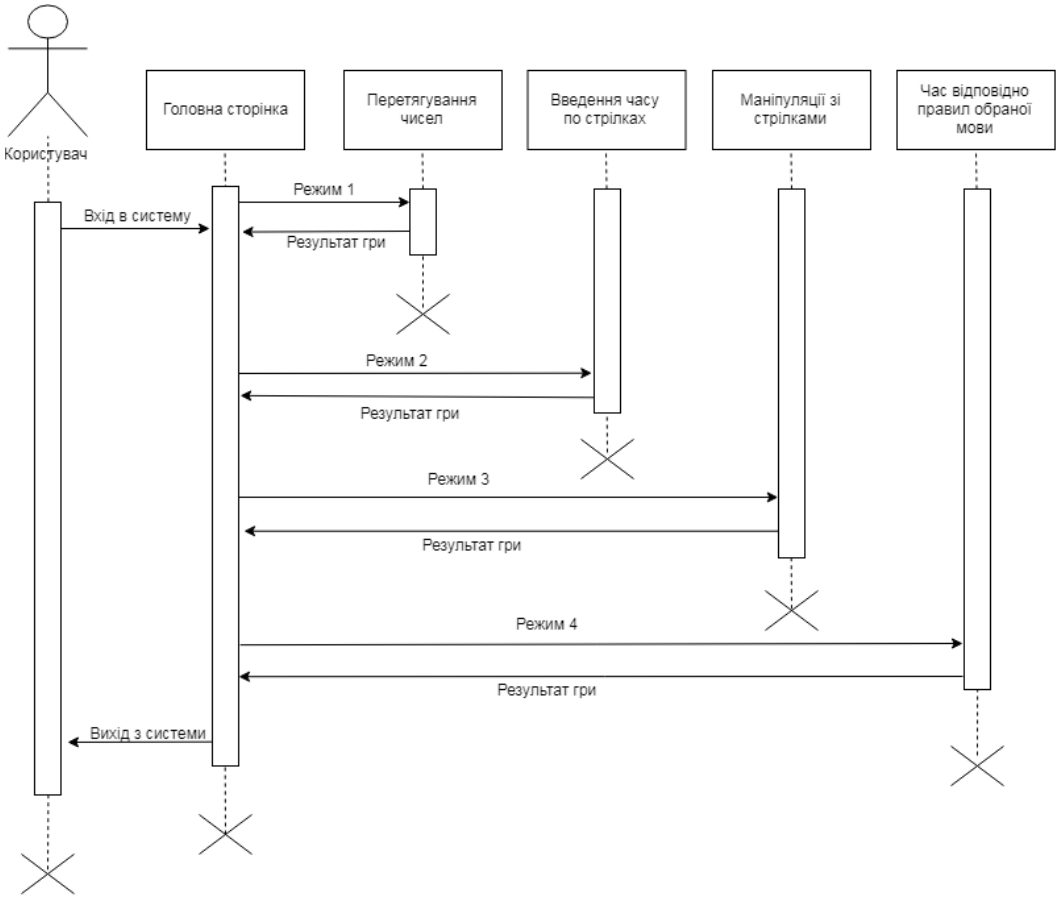


Рисунок 2.2 – Діаграма послідовності

Об'єктами ігрового процесу є користувач, головна сторінка та режими гри, які може обрати користувач. Продемонстровано послідовність взаємодії користувача з додатком у часі. Спочатку користувач входить у систему, щоб перейти в головну сторінку програмного комплексу, обирає бажаний режим ігрового функціоналу та починає грати у межах ігрової області. Вибір режиму є рівноправним, тобто з головної сторінки програми можна перейти в будь-який із режимів.

Структурна карта Константайна є моделлю відносин ієрархії між програмними модулями [11]. Вона представляє собою взаємодію модулів програмного комплексу «Time Teacher» і зображена на рисунку 2.3 у вигляді графу.

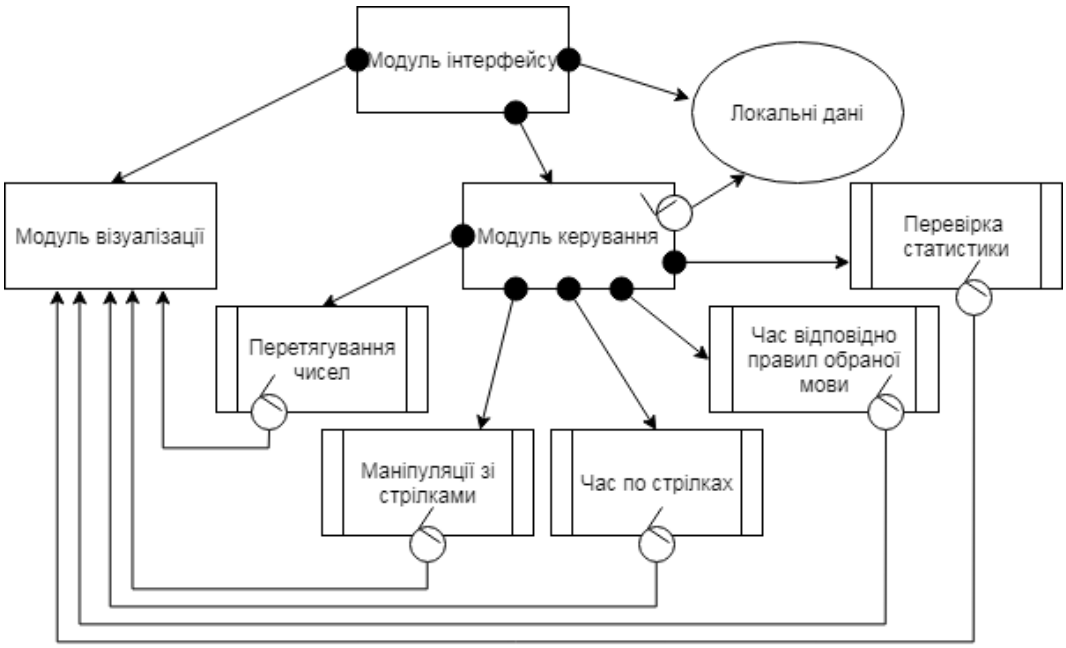


Рисунок 2.3 – Структурна карта Константайна програмного комплексу «Time Teacher»

Вершини – модулі, дуги – міжмодульні зв’язки [11]. Головним модулем є модуль інтерфейс. Він пов’язаний зв’язком за управлінням з модулями візуалізації та модулем керування програми. Також, відповідним зв’язком поєднується з локальними даними. Модуль керування пов’язаний зв’язком за даними з локальними даними, які включають в себе логін та пароль користувача, дані про вибір одного з режимів гри. Модуль керування використовує локальні дані та на їх основі виконує одну з підсистем – режими ігрового функціоналу «Time Teacher» та функцію перевірки статистики користувача. Кожна з підсистем взаємодіє з модулем візуалізації зв’язком за даними, оскільки відповідний модуль використовує дані як для відображення відповідних компонентів підсистем, так і дані результату їх виконання.

Загальна будова гри представлена на діаграмі класів на рисунку 2.4. Кожен з класів має ряд своїх полів та властивостей.

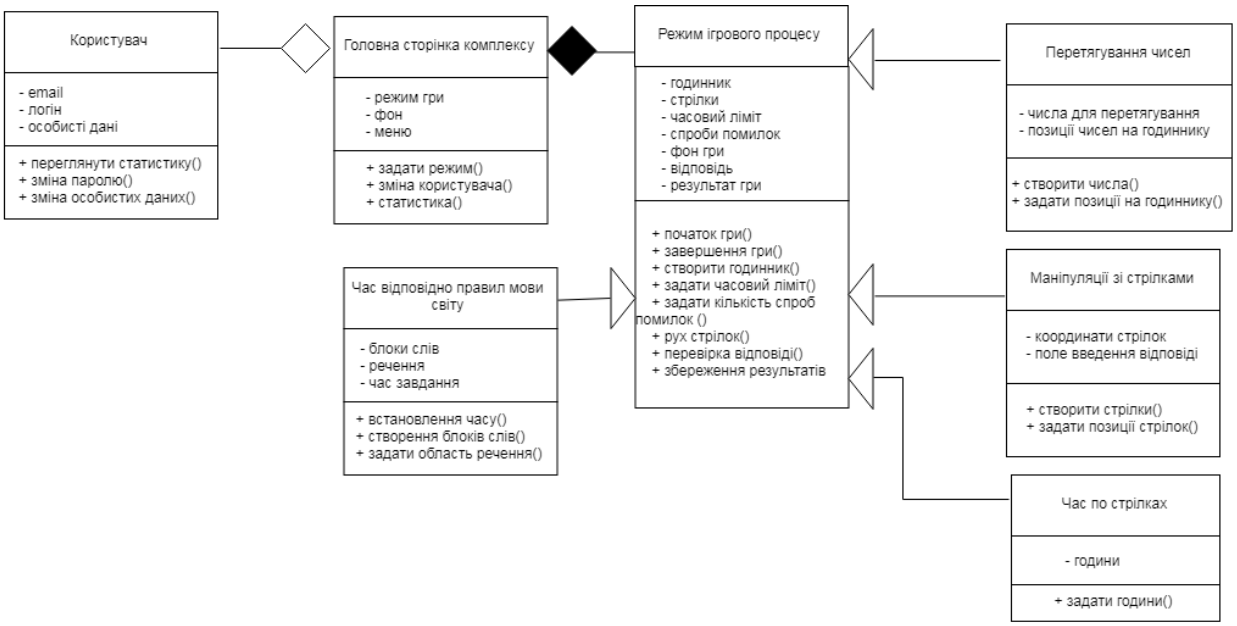


Рисунок 2.4 – Діаграма класів

Головним класом є «Головна сторінка комплексу», в якому задається загальне середовище програми.

Клас «Користувач» описує характеристику користувача. До класу «Головна сторінка комплексу» знаходиться в відношенні агрегації.

Клас «Режим ігрового процесу» описує безпосередньо загальну структуру процесу гри. До класу «Головна сторінка комплексу» перебуває у відношенні композиції.

Класи «Перетягування чисел», «Час відповідно до правил мови світу», «Маніпуляції зі стрілками», «Визначення часу по стрілках» уточнюють вид гри залежрно від вибору користувача та до класу «Режим ігрового процесу» перебувають у відношенні узагальнення.

Позначення режимів доступу:

1. + доступ public.
2. # доступ protected.
3. – доступ private.

Структурна схема компонентів програмного комплексу «Time Teacher» розглянуто на рисунку 2.5.

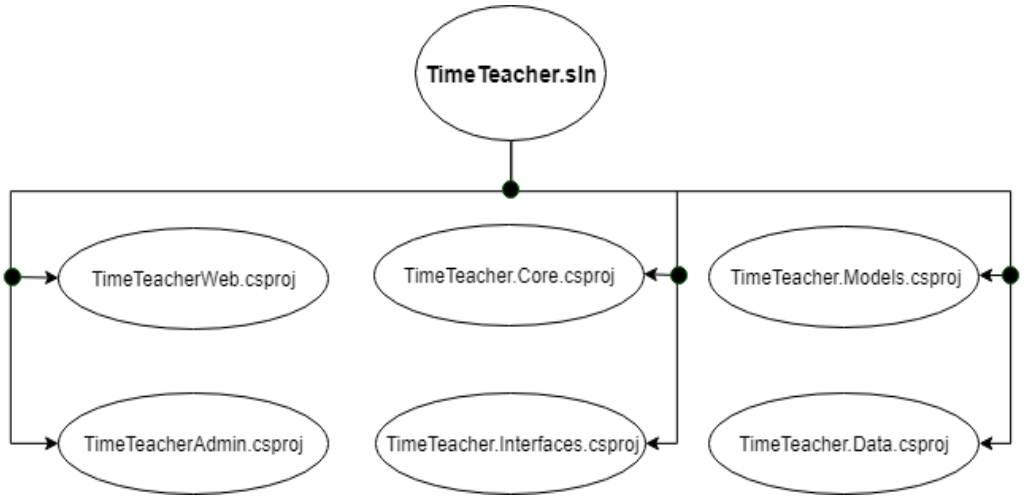


Рисунок 2.5 – Структурна схема компонентів програмного комплексу «Time Teacher»

Основними компонентами програмного комплексу є:

1. TimeTeacher.sln – .sln-файл, головне рішення програмного комплексу, що містить у собі всі підсистеми комплексу.
2. TimeTeacher.Web.csproj – .csproj-файл, реалізовує користувацький інтерфес програмного комплексу, ігровий функціонал.
3. TimeTeacher.Admin.csproj – .csproj-файл, складає функціонал для адміністрування, менеджменту програмним комплексом.
4. TimeTeacher.Core.csproj – .csproj-файл, що реалізує основну логіку по обробці запитів до бази даних (у вигляді сервісів та репозиторіїв) та повертає відповідні результати роботи системи.
5. TimeTeacher.Interfaces.csproj – .csproj-файл з інтерфейсами сервісів та репозиторіїв програмного комплексу.
6. TimeTeacher.Data.csproj – .csproj-файл, реалізує формування базу даних системи, надає функціонал по взаємодії з базою даних.
7. TimeTeacher.Models.csproj – .csproj-файл, містить моделі, що використовуються для представлення структури таблиць в базі даних.

2.2 Проектування бази даних програмного комплексу

Схема баз даних – це структура системи баз даних описана формальною мовою, яка підтримується системою управління баз даних і відноситься до організації даних для створення плану побудови бази даних з розподілом на таблиці [12].

Схема бази даних «TimeTeacherDb» програмного комплексу, призначеного для покращення існуючих методів у вирішенні проблеми вивчення та засвоєння дітьми пізнавання часу, розроблена за нотацією Microsot SQL Server Management Studio, зображена на рисунку 2.6.

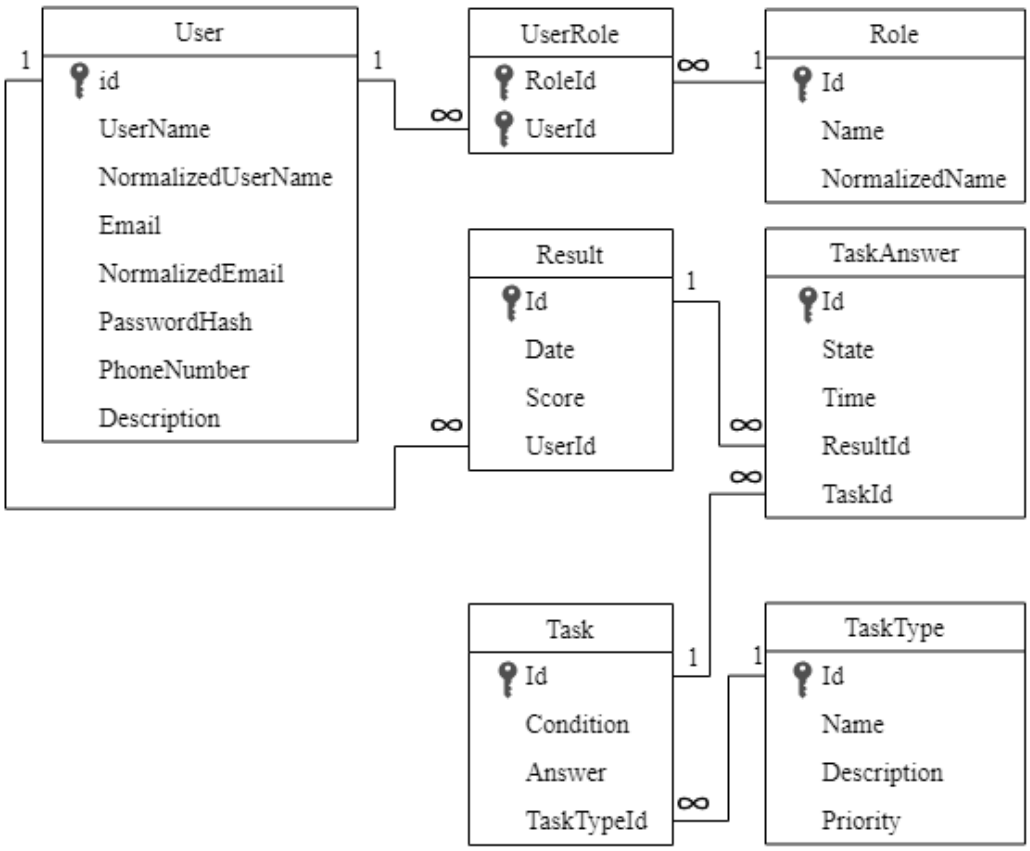


Рисунок 2.6 – Діаграма бази даних програмного комплексу

База даних «TimeTeacherDb» призначена для зберігання інформації про користувача – User, роль користувача – Role, завдання ігрового функціоналу – Task, інформацію про результати ігрового функціоналу – Results. Також

зберігається інформація про відповідь користувача – TaskAnswer – на кожне завдання, що має свій тип – TaskType.

У відношенні 1 : N знаходяться таблиці User : Result, Result : TaskAnswer, Task : TaskAnswer, TaskType : Task. Тому за 4 правилом нормалізації методом «суть-зв'язок» у кожній з таблиць, відношення якої N, з'явилося додаткове поле – значення ідентифікатора таблиці, відношення якої 1.

У відношенні N : N знаходяться таблиці User : Role, тому за 5 правилом нормалізації методом «суть-зв'язок» виникає проміжна таблиця UserRole, що містить складений ключ з ключів відповідних таблиць.

Структура полів таблиці User представлена у таблиці 2.1.

Таблиця 2.1 – Структура полів таблиці User

Назва поля	Тип	Інформаційний зміст
Id	int	Ключове поле для ідентифікації запису
UserName	nvarchar(256)	Ім'я користувача
NormalizedUserName	nvarchar(256)	Нормалізоване ім'я користувача
Email	nvarchar(256)	Пошта користувача
NormalizedEmail	nvarchar(256)	Нормалізована пошта користувача
PasswordHash	nvarchar(MAX)	Зашифрований пароль користувача
PhoneNumber	nvarchar(MAX)	Телефон користувача
Description	nvarchar(MAX)	Опис користувача

Структура полів таблиці Role представлена у таблиці 2.2.

Таблиця 2.2 – Структура полів таблиці Role

Назва поля	Тип	Інформаційний зміст
Id	int	Ключове поле для ідентифікації запису
Name	nvarchar(256)	Назва ролі
NormalizedName	nvarchar(256)	Нормалізована назва ролі

Структура полів таблиці UserRole представлена у таблиці 2.3.

Таблиця 2.3 – Структура полів таблиці UserRole

Назва поля	Тип	Інформаційний зміст
RoleId	int	Ключове поле – код ролі
UserId	int	Ключове поле – код користувача

Структура полів таблиці Result представлена у таблиці 2.4.

Таблиця 2.4 – Структура полів таблиці Result

Назва поля	Тип	Інформаційний зміст
Id	int	Ключове поле для ідентифікації запису
Date	datetime(7)	Час отримання результату
Score	int	Набрані очки
UserId	int	Код користувача

Структура полів таблиці Task представлена у таблиці 2.5.

Таблиця 2.5 – Структура полів таблиці Task

Назва поля	Тип	Інформаційний зміст
Id	int	Ключове поле для ідентифікації запису
Condition	nvarchar(MAX)	Початкові умови завдання
Answer	nvarchar(MAX)	Відповідь на завдання
TaskTypeId	int	Код типу завдання

Структура полів таблиці TaskType представлена у таблиці 2.6.

Таблиця 2.6 – Структура полів таблиці TaskType

Назва поля	Тип	Інформаційний зміст
Id	int	Ключове поле для ідентифікації запису
Name	nvarchar(256)	Назва типу завдання
Description	nvarchar(MAX)	Опис завдання
Priority	int	Складність завдання

Структура полів таблиці TaskAnswer представлена у таблиці 2.7.

Таблиця 2.7 – Структура полів таблиці TaskAnswer

Назва поля	Тип	Інформаційний зміст
Id	int	Ключове поле для ідентифікації запису
State	bit	Стан відповіді
Time	int	Час отримання відповіді
ResultId	int	Код результату завдання
TaskId	int	Код завдання

В результаті, розроблена структура бази даних для програмного комплексу «Time Teacher».

2.3 Розробка інтерфейсу програмного комплексу

Програмний комплек «Time Teacher» проектується як веб додаток, тому слід розглянути основні принципи побудови веб-інтерфейсу.

Ефективний інтерфейс користувача об'єднує в собі дві складові: приємний зовнішній вигляд і зручність використання. Щоб створити такий інтерфейс, потрібно розуміти основи: як користувач взаємодіє з UI, чого він очікує і що може стати джерелом потенційних проблем.

Інтерфейс - не просто кнопки і меню, кольори і шрифти, тобто зовнішній вигляд, а набір інструментів, які допомагають вирішувати проблеми користувача.

Розглянемо основні принципи побудови користувальницького веб інтерфейсу:

1. Знайти свого користувача. Чим детальніше його описати, тим краще. Тільки точно знаючи, хто користувач, як і чому він діє, можна створити веб інтерфейс, який буде працювати так, як потрібно. Інтерфейс не повинен змушувати користувача думати. Потрібно прийняти рішення виходячи з того, хто користувач, і реалізувати його потреби в своєму інтерфейсі.

2. Використовувати вже відомі моделі. Про це легко забути, але користувачі вже проводять багато часу, користуючись іншими інтерфейсами, наприклад Facebook, Gmail тощо. Потрібно використовувати вже відомі моделі побудови веб інтерфейсів, адже це допоможе користувачам швидко розібратися в новому продукті і завоювати довіру користувача.

3. Бути послідовним. Побачивши певний функціонал в одному вигляді, користувач очікує, що і в подальшому це буде відбуватися саме так.

4. Створити візуальну ієрархію. Показати користувачу, що найважливіше на веб сторінці. Розмір, колір, розміщення і вільний простір навколо кожного елемента повинні відповідати його ролі, важливою або другорядною. Правильно побудована ієрархія спрощує розуміння навіть складних концепцій.

5. Надавати фідбек і захищати користувача від випадкових дій. Говорити користувачеві, що відбувається. Чи відправлено лист? Чи завантажився файл? Чи підтримується цей формат зображень? Переконайтися, що не можна видалити всі дані одним кліком. Користувачеві важливо знати, що він не залишився один на один зі своїми проблемами. Подбати, щоб йому було комфортно користуватися системою.

6. Не показувати всі елементи управління відразу. Це відлякає недосвідченого користувача. Відмінний хід - заховати додаткові функції, які

можуть ніколи не знадобитися, на вкладці «Додаткові» або в спеціальному меню.

7. Надавати більше можливостей досвідченим користувачам. Ті, хто вже в сотий раз використовує сервіс, будуть вдячні за можливість виконувати рутинні дії за допомогою гарячих клавіш.

8. Не показувати користувачеві порожню сторінку. Коли користувач вперше заходить в новий інструмент, потрібна певна інструкція або інформація про функціонал.

9. Не ускладнювати. Перш, ніж додати ще один елемент UI, потрібно перевірити, чи полегшить він роботу з системою або тільки забере у користувача цінний час.

10. Тестування. Чим раніше почати і чим більше тестів провести, тим якісніший буде кінцевий результат. Потрібно залучати звичайних користувачів. Вони не працювали над проектом тижнями і нічого не знають про те, як все влаштовано, тому їх зауваження будуть особливо корисні.

Коли користувач переходить на сторінку, він не читає її, а всього лише пробігається по ній очима. Тому веб-дизайн сучасного інтерфейсу користувача створюється з урахуванням F і Z-патернів читання.

F-патерн працює для сайтів з великою кількістю тексту - користувач переглядає верхню частину сторінки в пошуку ключових слів, потім опускається нижче і пробігається по підзаголовкам. Тому на «важких» сайтах важлива інформація розміщується зліва, там, де на неї чекає побачити користувач. Цей патерн використовують CNN і New York Times.

На сторінках без великих фрагментів тексту, наприклад, на «Landing page», працює Z-патерн читання. Спочатку відвідувачі переглядають шапку з ліва на право, потім опускають погляд вниз вліво і переглядають нижню частину вікна зліва на право.

Описані вище принципи побудови користувальницького веб інтерфейсу зводяться до наступного: UI повинен вирішувати проблеми користувача найпростішим з можливих способів [13].

Основною частиною програмного комплексу є ігровий функціонал, тому слід розглянути побудову ігового інтерфейсу.

Ігровий інтерфейс – це змога управління персонажем, взаємодія персонажів один з одним, спілкування гравців між собою і т.д. Практично всі ігри мають складний інтерфейс, що дозволяє управляти персонажами за допомогою різних способів – мишкою, клавіатурними кнопками, віртуальними кнопками на екрані. Ігри, де користувач за допомогою різних способів взаємодіє з об’єктами ігрового світу, набирає все більшої популярності. Більшість дій персонажів гри реалізуються стандартними способами, однаковими для всіх ігор. Часто гравець може змінити налаштування інтерфейсу так, щоб йому було більш звично і зручно. У той же час з використанням сенсорних екранів з’явилися і нові способи управління за допомогою рухів пальців [14].

Детальний опис інтерфейсу допомагає користувачу освоїтися в грі і максимально швидко отримати доступ до інформації, що його цікавить, та ігрових налаштувань. Для реалізації програмного комплексу «Time Teach» прийнято рішення реалізовувати інтерфейс з мінімальною кількістю елементів, оскільки користувачами є, в основному, діти. Тому для них дуже важливим є простий інтерфейс, оскільки при великому нагромадженні компонентів на екрані дитина важко орієнтуватиметься та втратить інтерес.

Для проектування програмного комплексу необхідним є створення графічних схем інтерфейсу. Розглянемо проектування інтерфейсу головної сторінки програмного комплексу, кожного з модулів ігрового функціоналу.

Структурна схема інтерфейсу додатку «Time Teacher» зображена на рисунку 2.7. Програмний комплекс «Time Teacher» складається з «Account page» – сторінка авторизації та реєстрації користувача, та «Main page» – сторінка, що містить вибір одного з режимів ігрового процесу.

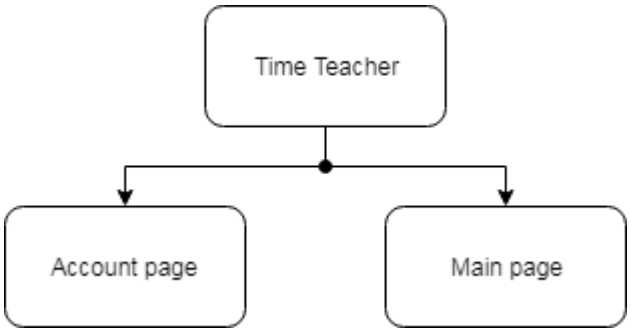


Рисунок 2.7 – Структурна схема інтерфейсу додатку «Time Teacher»

Графічна схема інтерфейсу головної сторінки програмного комплексу «Time Teacher» зображено на рисунку 2.8.

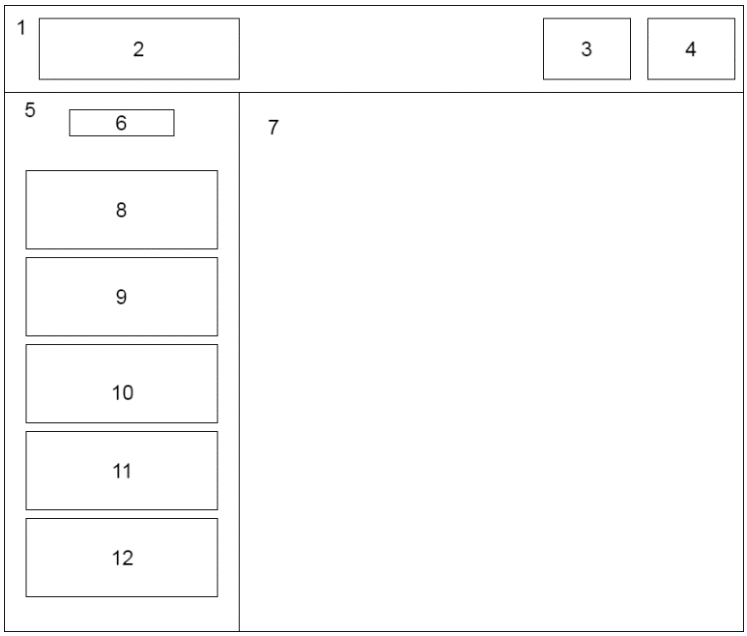


Рисунок 2.8 – Графічна схема інтерфейсу головної сторінки програмного комплексу «Time Teacher»

Головна сторінка програмного складається з таких елементів:

1. Header сторінки
2. Назва програмного комплексу.
3. Зміна мови програмного комплексу.
4. Кнопка «Вихід».
5. Меню сторінки.
6. Текст «Вибір режиму гри».

7. Ігрова область – на головній сторінці на області відображені основні правила ігрового функціюналу програмного комплексу.
8. Модуль «Перетягування чисел».
9. Модуль «Час по стрілках».
10. Модуль «Маніпуляції зі стрілками».
11. Модуль «Час відповідно до правил мови світу».
12. Результати.

Кожен з модулів ігрового функціоналу розташований в «ігровій області» головної сторінки програмного комплексу.

Графічна схема інтерфейсу модуля «DragNumbers» зображена на рисунку 2.9.

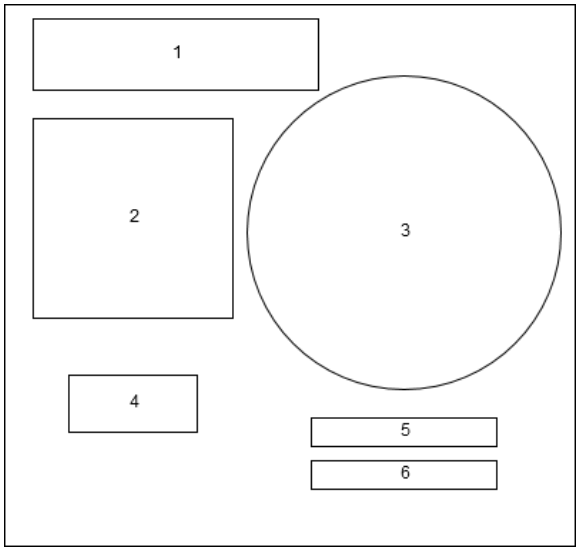


Рисунок 2.9 – Графічна схема інтерфейсу модуля «DragNumbers»

Модуль «DragNumbers» складається з таких елементів:

1. Відомості про користувача та правила гри.
2. Область чисел, які потрібно перетягувати.
3. Годинник, що містить відповідні позиції для чисел.
4. Кнопка «Далі».
5. Залишок часу.

6. Рахунок користувача

Графічна схема інтерфейсу модуля «TimeArrow» зображена на рисунку 2.10.

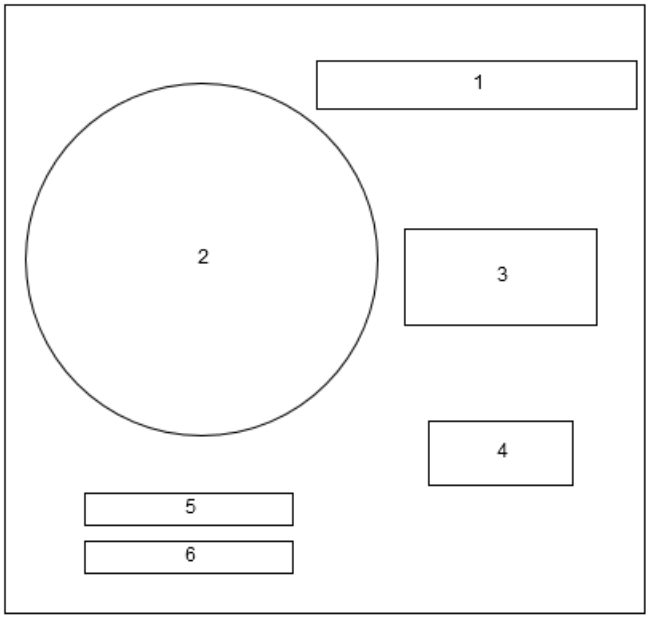


Рисунок 2.10 – Графічна схема інтерфейсу модуля «TimeArrow»

Модуль «TimeArrow» складається з таких елементів:

1. Відомості про користувача та правила гри.
2. Годинник зі стрілками, час яких потрібно вказати.
3. Область, де вказується час, що зображено на годиннику.
4. Кнопка «Далі».
5. Залишок часу.
6. Рахунок користувача

Графічна схема інтерфейсу модуля «ArrowManipulation» зображена на рисунку 2.11.

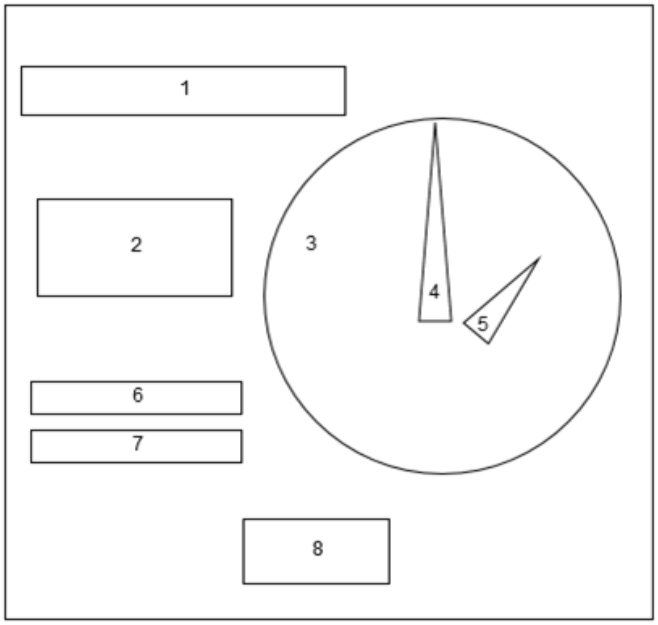


Рисунок 2.11 – Графічна схема інтерфейсу модуля «ArrowManipulation»

Модуль «ArrowManipulation» складається з таких елементів:

1. Відомості про користувача та правила гри.
2. Час, який потрібно задати на годиннику
3. Годинник.
4. Хвилинна стрілка
5. Годинна стрілка
6. Залишок часу.
7. Рахунок користувача
8. Кнопка «Далі».

Для модуля «LanguageTime» додатково обирається користувачем мова, згідно якої формуються відповідні завдання. Графічна схема інтерфейсу модуля «LanguageTime» під час ігрового процесу зображена на рисунку 2.12.

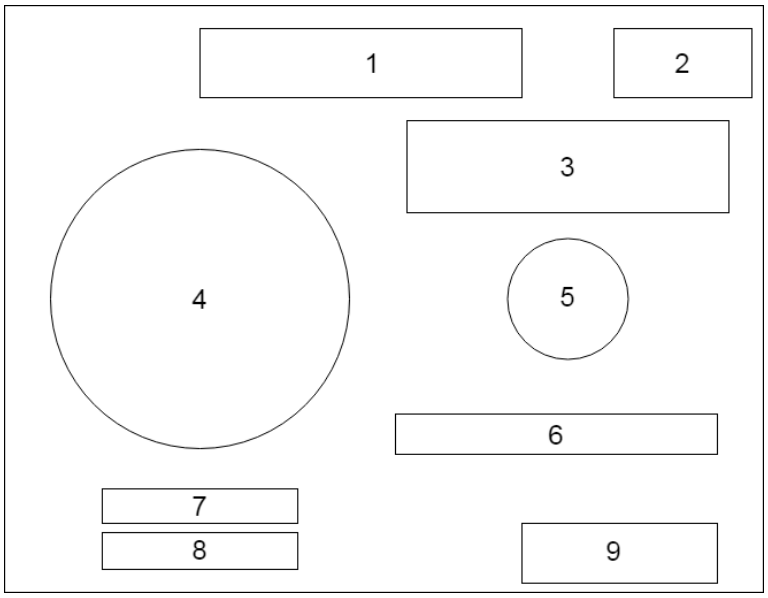


Рисунок 2.12 – Графічна схема інтерфейсу модуля «LanguageTime»

Модуль «LanguageTime» складається з таких елементів:

1. Відомості про користувача та правила гри.
2. Відомості про мову, згідно якої формуються завдання.
3. Блоки слів обраної мови, що вказують час відповідно правил мови.
4. Годинник з заданим часом.
5. Область, у яку необхідно перетягувати блоки зі словами
6. Речення, яке формується при перетягуванні блоків зі словами.
7. Залишок часу.
8. Рахунок користувача
9. Кнопка «Далі».

2.4 Вибір кольорової гами

Феномен кольору зовсім не простий. Емоційний, або психологічний, вплив кольору не так легко проаналізувати, як фізіологічні процеси, що виникають у результаті сприйняття кольору, а тим часом більшість людей віддає перевагу певним кольорам і вважає, що колір впливає на настрій.

Кольори розділяють на сильні й слабкі, заспокійливі й збудливі, навіть на важкі й легкі.

Колір змінює враження про дійсні розміри предметів, причому кольори, які здаються важкими, зменшують ці розміри. З рівновеликих квадратів найменшим здається червоний, синій – більшим, білий – найбільшим. Поширена думка про збудливу дію червоного світла є правильною, тому що він підвищує електричну провідність шкіри, що визначає потовиділення. У той же час червоне світло не здійснює значного впливу на частоту пульсу або дихання. У багатьох країнах білий колір – традиційно весільний, а в деяких – колір жалоби.

Червоний, жовтий, зелений і синій – «фокусні» кольори для людства. Саме цим кольорам віддають перевагу діти, поки не починають говорити, і уникають граничних кольорів, що лежать між ними. Сірий, жовтий і білий, як правило, уважають слабкими кольорами, а червоний – сильним і активним. Синій майже всюди розцінюють як «гарний» колір. У своєму сприйнятті кольору і відношенні до нього люди, очевидно, дуже схожі [15].

При розробці дизайну додатку важливою є кольорова гамма. Адже кількість, контрастність, насиченість кольорів впливає на роботу користувача з програмним продуктом. Правильно підібрана кольорова гамма робить інтерфейс додатку легким у розумінні та сприйнятті (рисунок 2.13).

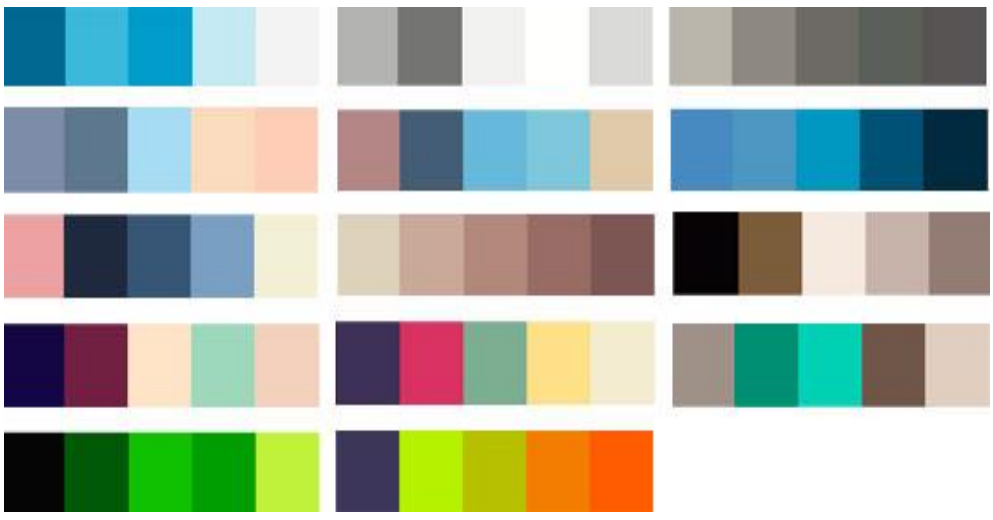


Рисунок 2.13 – Вдалі поєднання кольорів

Розглянемо кольорову гамму ігрової області програмного комплексу «Time Teacher». Основний колір тексту – темно-сірий (код «#2D2D2F»). Фон задається текстурою, зображеною на рисунку 2.14, основний колір якої – світло-коричневий (код «#E5D88E»).

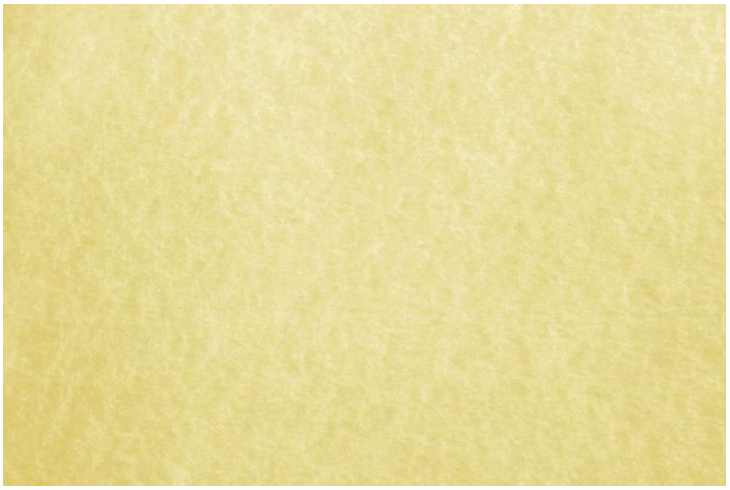


Рисунок 2.14 – Фон ігрового функціоналу «Time Teacher»

Текст обраного кольору добре виділяється на кольорі фону ігрової області і, до того ж, не надто контрастний, що могло б викликати роздратування у користувача.

Кожен з режимів ігрового функціоналу мають як схожі, так і характерні лише для них елементи. Поля вводу інформації, а також кнопка «Далі», що дозволяє перевірити введені дані на правильність та створити наступне завдання, мають світло–сірий колір (код #F5F5F5). Поєднання світло-сірого та фонового кольорів є вдалим, оскільки розглянуті відтінки не контрастні. В результаті чого, користувач не відволікається і ефективно працює з програмою.

На годинниках наявні ряд кольорів, як для цифр, так і для стрілок, фону годинника, основними з яких є зелений, червоний, рожевий, блакитний, помаранчевий. При одночасному поєднанні даних кольорів може виникати великий контраст, що негативно вплине на стан користувача, оскільки він буде дратуватися, відволікатися. Проте, оскільки годинник є одним із ключових елементів ігрової сцени і займає приблизно 1/3 частини вікна ігрової області,

контраст є вдалим. Годинник та його елементи виділяються і їх легко можна знайти на вікні програми. Також дана палітра гармонійно поєднуються з загальним фоном програми.

2.5 Розробка блок-схеми алгоритму роботи програмного комплексу

Наступним кроком є створення загального алгоритму роботи програми «Time Teacher». Алгоритм є ключовим при проектуванні, оскільки він буде основою для подальшої розробки програмного комплексу.

Блок–схема алгориту роботи програмного комплексу «Time Teacher» представлена на рисунку 2.15.

1 – Початок роботи програми.

2 – Блок для авторизації\реєстрації користувача.

3 – StartGame – завантаження основних налаштувань ігрового функціоналу.

4 – Task – змінна, що слугує вибором певної роботи програми. При запуску вона рівна 0.

5 – ChooseTask – метод, що обробляє зміну значення Task і викликає метод відповідно до її значення.

6 – Якщо Task=1, то 7 – виклик методу Exit. Вихід з програми при закритті вікна браузера, або виходу із системи. Завершення програми (блок 18).

8 – Якщо Task=2, то 9 – виклик методу ShowStatistic, що призначений для демонстрації статистики користувача.

10 – Якщо Task=3, то 11 – виклик методуDragNumbers, що призначений для запуску рівня «Перетягування чисел».

12 – Якщо Task=4, то 13 – виклик методу TimeArrow, що призначений для запуску рівня «Час по стрілках».

14 – Якщо Task=5, то 15 – виклик методу ArrowManipulation, що призначений для запуску рівня «Маніпуляція зі стрілками».

16 – Якщо Task=6, то 17 – виклик методу LanguageTime, що призначений для запуску рівня «Час відповідно правил мови світу».

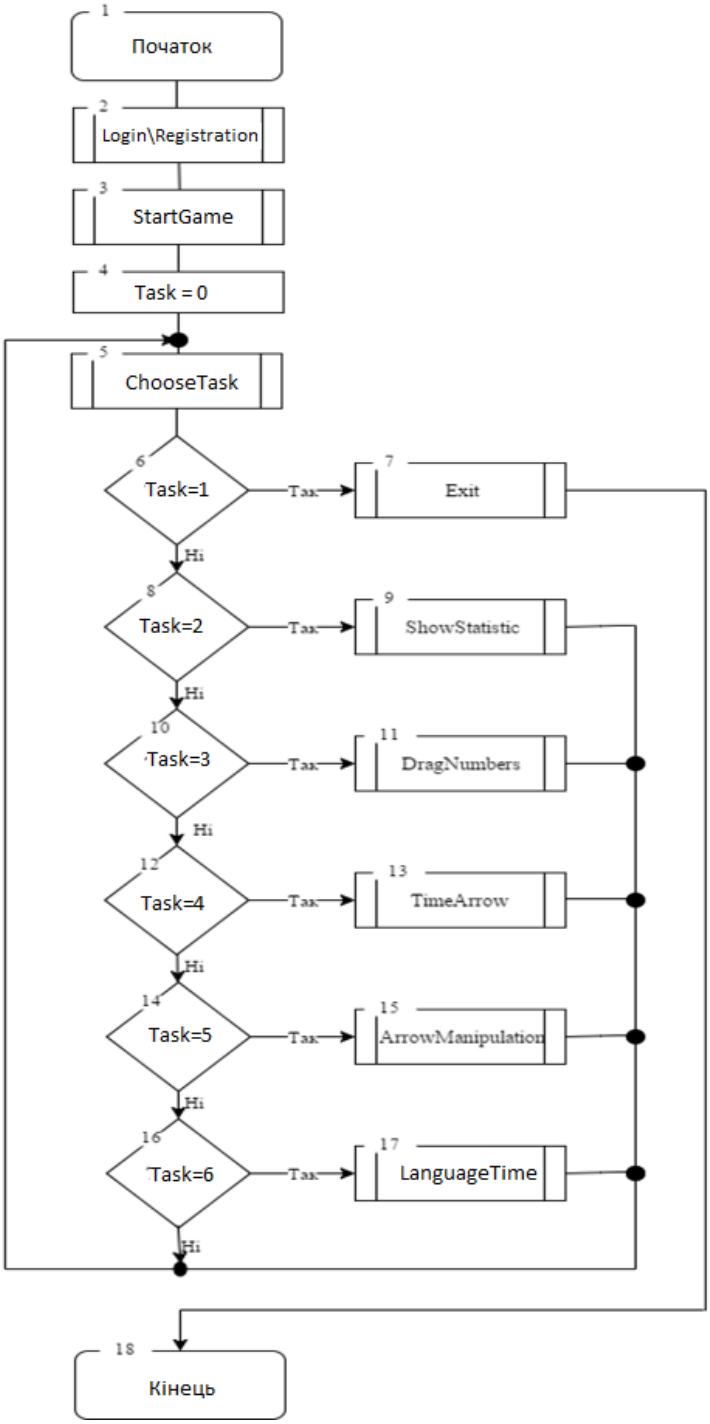


Рисунок 2.15 – Блок–схема алгориту роботи програми «Time Teacher»

В результаті, створена блок–схема алгоритму програми буде використана при написанні коду програмного комплексу.

2.6 Висновки

Отже, розроблено загальну архітектуру додатку за допомогою відповідних діаграм варіантів використання, послідовності, класів, структурної карти Константайна, структурної схеми компонентів додатку. Кожна з діаграм розглядає програмний комплекс «Time Teacher» як цілісну систему, та описує різні її аспекти.

Також спроектовано базу даних, що призначена для фіксації результатів ігрового процесу користувача. Вона необхідна для формування статистики ігрового функціоналу, щоб відслідковувати прогрес у освоєнні того, чи іншого модуля.

Розроблено графічні схеми інтерфейсу головної сторінки та основних модулів ігрового процесу.

Виконано підбір кольорової гамми інтерфейсу, а також розроблено загальний алгоритм роботи комплексу. Відповідні матеріали будуть використані при розробці програмного продукту.

3 РОЗРОБКА ПРОГРАМНОГО КОМПЛЕКСУ

3.1 Аналіз вибору мови програмування та технологію розробки

Програмний комплекс представлено у вигляді веб додатку, що має front-end (клієнську частину) та back-end (серверну частину).

Розглянемо мови програмування та відповідні технології (фреймворки), що використовуються для розробки back-end веб додатків – C# (ASP.NET Core), JavaScript (Node.js), Phython (Django).

Мова **C#** є відомою в області мов програмування. З'явившись в Microsoft, будучи спадкоємцем C++, вона одержала потужну підтримку. Однак цього недостатньо для дійсного визнання переваг мови. Винахідник мови - співробітник Microsoft Андреас Хейлсберг у червні 2000 року. Мова C# придатна як для швидкого створення прототипів додатків, так і для розробки великомасштабних проектів.

Особливості мови C# [16]:

1. Повний доступ до бібліотеки базових класів .NET.
2. Підтримка структурного програмування.
3. Можливість використання C# для написання динамічних web–сторінок ASP.NET.
4. Автоматичне звільнення динамічно розподіленої пам’яті.
5. Повна підтримка класів та об’єктно-орієнтованого програмування.
6. Вказівники і прямий доступ до пам’яті, якщо вони необхідні.

Платформа **ASP.NET Core** – технологія від компанії Microsoft, призначена для створення веб-додатків: від невеликих веб-сайтів до великих веб-порталів і веб-сервісів. З одного боку, ASP.NET Core є продовженням розвитку платформи ASP.NET. Але з іншого боку, вихід ASP.NET Core фактично означає революцію всієї платформи, її якісна зміна.

Розробка над платформою почалася в 2014 році. Тоді платформа умовно називалася ASP.NET vNext. У червні 2016 року вийшов перший реліз платформи, а в травні 2018 року побачила версія ASP.NET Core 2.1.

ASP.NET Core є opensource-фреймворком. ASP.NET Core може працювати на основних популярних операційних системах: Windows, Mac OS X, Linux. Отже, за допомогою ASP.NET Core можливо створювати крос-платформні додатки. А для розгортання веб-додатків можна використовувати традиційний IIS, або крос-платформний веб-сервер Kestrel.

Хоча ASP.NET Core переважно націлене на використання .NET Core, але фреймворк також може працювати і з повною версією фреймворка .NET. Завдяки модульності фреймворка всі необхідні компоненти веб-додатки можуть завантажуватися як окремі модулі через пакетний менеджер Nuget. ASP.NET Core включає в себе фреймворк MVC, який об'єднує функціональність MVC, Web API і Web Pages.

У ASP.NET Core наявно ряд функцій. Однією з таких є тег-хелпери (tag helper), які дозволяють поєднувати синтаксис html з кодом C #. Спрощено управління залежностями і конфігурація проекту. Фреймворк має свій легкий контейнер для впровадження залежностей, і більше немає необхідності застосовувати сторонні контейнери, такі як Autofac, Ninject. Хоча при бажанні їх також можна продовжувати використовувати. В якості інструментарію розробки використовується останні випуски Visual Studio, починаючи з версії Visual Studio 2015. Крім того, створюються додатки в середовищі Visual Studio Code, яка є крос-платформнною.

Для обробки запитів використовується новий конвеєр HTTP, який заснований на компонентах Katana і специфікації OWIN. А його модульність дозволяє легко додати свої власні компоненти.

У підсумку, виділимо ключові особливосты ASP.NET Core від попередніх версій ASP.NET [17]:

1. Новий легкий і модульний конвеєр HTTP-запитів.

2. Можливість розгортати додаток як на IIS, так і в рамках свого власного процесу.

3. Використання платформи .NET Core і її функціональності.

4. Поширення пакетів платформи через NuGet.

5. Інтегрована підтримка для створення та використання пакетів NuGet.

6. Єдиний стек веб-розробки, що поєднує Web UI і Web API.

7. Конфігурація для спрощеного використання в хмарі.

8. Вбудована підтримка для впровадження залежностей.

9. Можливість розширення.

10. Кросплатформеність: можливість розробки і розгортання додатків ASP.NET на Windows, Mac і Linux.

11. Розвиток як open source, відкритість до змін.

JavaScript створено в 1995 році в компанії Netscape в якості мови сценаріїв в браузері Netscape Navigator 2. Сьогоднішній світ веб-сайтів важко уявити без мови JavaScript, оскільки вона робить «живими» веб-сторінки, які користувачі переглядають у своєму веб-браузері.

Спочатку мова називалася LiveScript, але на хвилі популярності в той момент іншої мови Java LiveScript був перейменований в JavaScript. Спочатку JavaScript мав досить невеликі можливості. Його мета полягала в тому, щоб додати трохи поведінки на веб-сторінку. Наприклад, обробити натискання кнопок на веб-сторінці, провести які-небудь інші дії, пов'язані перш за все з елементами управління.

Однак розвиток веб-середовища, поява HTML5 і технології Node.js відкрило перед JavaScript набагато більші можливості. Зараз JavaScript продовжує використовуватися для створення веб-сайтів, але тепер надає набагато більше можливостей.

JavaScript є мовою, що інтерпретується. Інтерпретатор отримує інструкції мови JavaScript, які визначені на веб-сторінці, виконує їх (або інтерпретує).

З самого початку існувало кілька веб-браузерів (Netscape, Internet Explorer), які надавали різні реалізації мови. І щоб звести різні реалізації до

загальної основи і стандартизувати мову під керівництвом організації ECMA був розроблений стандарт ECMAScript. Терміни JavaScript і ECMAScript є багато в чому взаємозамінними і відносяться до однієї і тієї ж мови [18].

Платформа **Node.js** призначена для виконання високопродуктивних мережевих додатків, написаних мовою програмування JavaScript. Платформа окрім роботи із серверними скриптами для веб-запитів, також використовується для створення клієнтських та серверних програм.

В платформі використовується розроблений компанією Google рушій V8.

Для забезпечення обробки великої кількості паралельних запитів у Node.js використовується асинхронна модель запуску коду, заснована на обробці подій в неблокуючому режимі та визначенні обробників зворотніх викликів (callback). Як способи мультиплексування з'єднань підтримується epoll, kqueue, /dev/poll і select. Для мультиплексування з'єднань використовується бібліотека libuv, для створення пулу потоків (thread pool) задіяна бібліотека libeio, для виконання DNS-запитів у неблокуючому режимі інтегрований c-ares. Всі системні виклики, що спричиняють блокування, виконуються всередині пула потоків і потім, як і обробники сигналів, передають результат своєї роботи назад через неіменовані канали (pipe).

Цикл обробки подій (event loop) у Node.js прихований від розробника і нагадує обробку подій у веб-застосунку, що працює в браузері. При написанні програм для Node.js необхідно враховувати специфіку подієво-орієнтованого програмування.

Жодна функція в Node.js не повинна безпосередньо виконувати блокуючі операції вводу/виводу - для отримання даних з диска, від іншого процесу або з мережі потрібна установка callback-обробника.

Для розширення функціональності застосунків на базі Node.js підготовлена велика колекція модулів, в якій можна знайти реалізацію HTTP, SMTP, XMPP, DNS, FTP, IMAP, POP3 серверів і клієнтів, модулі для інтеграції з різними веб-фреймворками, обробники WebSocket і AJAX, конектори до СКБД (MySQL, PostgreSQL, SQLite, MongoDB), шаблонізатори, CSS-рушії,

реалізації криптоалгоритмів і систем авторизації (наприклад, OAuth), XML-парсери [19].

Python є популярна високорівнева мова програмування, яка призначена для створення додатків різних типів. Це і веб-додатки, і гри, і настільні програми, і робота з базами даних. Досить велике поширення Python отримав в області машинного навчання і досліджень штучного інтелекту.

Вперше мова Python була анонсована в 1991 році голландським розробником Гвідо Ван Россум. З тих пір ця мова виконала великий шлях розвитку. У 2000 році була видана версія 2.0, а в 2008 році - версія 3.0. Незважаючи на такі великі проміжки між версіями постійно виходять підверсії.

Основні особливості мови програмування Python:

1. Скриптова мова. Код програм визначається у вигляді скриптів.
2. Підтримка різних парадигм програмування, в тому числі об'єктно-орієнтованої та функціональної парадигм.
3. Інтерпретація програм. Для роботи зі скриптами необхідний інтерпретатор, який запускає і виконує скрипт. Виконання програми на Python виглядає наступним чином. Спочатку пишуть у текстовому редакторі скрипт з набором виразів на даній мові програмування. Передається цей скрипт на виконання інтерпретатора. Інтерпретатор транслює код в проміжний байткод, а потім віртуальна машина переводить отриманий байткод в набір інструкцій, які виконуються операційною системою. Тут варто зазначити, що хоча формально трансляція інтерпретатором вихідного коду в байткод і переклад байткода віртуальною машиною в набір машинних команд представляють два різних процеси, але фактично вони об'єднані в сам інтерпретатор.
4. Портативність і платформонезалежність. Не має значення, яка операційна система - Windows, Mac OS, Linux, достатньо написати скрипт, який буде запускатися на всіх ОС при наявності інтерпретатора.
5. Автоматичне управління пам'яті.
6. Динамічна типізація.

Python вважається простою мовою програмування, лаконічна і в той же час досить проста і зрозуміла у синтаксисі. Відповідно її легко вивчати, і власне це одна з причин, по якій вона є однією з найпопулярніших мов програмування для навчання. Зокрема, в 2014 році вона був визнана найпопулярнішою мовою програмування для навчання в США.

Python також популярний не тільки в сфері навчання, але в написанні конкретних програм в тому числі комерційного характеру. У чималому ступені тому для цієї мови написано безліч бібліотек, які можна використовувати.

Крім того, у даної мови програмування дуже велике ком'юніті, в інтернеті можна знайти по даній мові безліч корисних матеріалів, прикладів, отримати кваліфіковану допомогу фахівців [20].

Django - це фреймворк для створення веб-додатків за допомогою мови програмування Python.

Django був створений в 2005 році, коли веб-розробники з газети Lawrence Journal-World стали використовувати Python в якості мови для створення веб-сайтів. А в 2008 році вийшов публічний перший реліз фреймворку. На сьогоднішній день він продовжує розвиватися.

Django досить популярний. Він використовується на багатьох сайтах, в тому числі таких, як Pinterest, PBS, Instagram, BitBucket, Washington Times, Mozilla і багатьох інших.

Фреймворк є безкоштовним, він розвивається як open source. Фреймворк Django реалізує архітектурний патерн Model-View-Template або скорочено MVT, який за фактом є модифікацією распростанение в веб-програмуванні паттерна MVC (Model = -View-Controller).

Основні елементи паттерну:

1. «URL dispatcher»: при отримання запиту на основі запитаної адреси URL визначає, який ресурс повинен обробляти даний запит.

2. «View»: отримує запит, обробляє його і відправляє користувачеві деяку відповідь. Якщо для обробки запиту необхідно звернення до моделі і бази даних, то View взаємодіє з ними. Для створення відповіді може застосовувати

Template або шаблони. В архітектурі MVC цьому компоненту відповідають контролери.

3. «Model»: описує дані, які використовуються в додатку. Окремі класи, як правило, відповідають таблицям в базі даних.

4. «Template»: представляє логіку у вигляді згенерованої розмітки html. У MVC цього компоненту відповідає View, тобто уявлення.

Коли в додаток приходить запит, то «URL dispatcher» визначає, з яким ресурсом зіставляється даний запит і передає цей запит обраному ресурсу. Ресурс фактично являє функцію або «View», який отримує запит і певним чином обробляє його. В процесі обробки «View» може звертатися до моделей і базі даних, отримувати з неї дані, або, навпаки, зберігати в неї дані. Результат обробки запиту відправляється назад, і цей результат користувач бачить в своєму браузері. Як правило, результат обробки запиту представляє згенерований html-код, для генерації якого застосовуються шаблони («Template») [21].

Проаналізувавши кожну з мов програмування та відповідні технології для розробки серверної частини веб додатків, обрано мову програмування C# та технологію ASP.NET Core, оскільки надаються всі необхідні засоби для швидкої реалізації програмного комплексу «Time Teacher».

Для розробки front-end веб додатків використовується мова програмування JavaScript, описана вище, осільки її використовують для написання як back-end, так і front-end. Сучасними технологіями, що використовують JavaScript для написання клієнтської частини, є React.js, Vue.js, Angular.js.

Також існує технологія Angular 7, яка є нащадком технології AngularJS. Використовує мову програмування TypeScript, яка після побудови проекту автоматично перетворюється в мову JavaScript, зрозумілу для браузерів мову програмування.

React.js - це бібліотека JavaScript, яка використовується для створення призначеного для користувача інтерфейсу. React був створений компанією Facebook, а перший реліз бібліотеки з’явився у березні 2013 року.

Спочатку React призначався для створення веб-сайтів, проте пізніше з'явилася платформа React Native, яка вже призначалася для мобільних пристроїв.

React представляється як ідеальний інструмент для створення масштабованих веб-додатків (в даному випадку мова йде про фронтенд), особливо в тих ситуаціях, коли додаток являє SPA (односторінковий додаток).

React відносно простий в освоєнні, має зрозумілий та лаконічний синтаксис.

Вся структура веб-сторінки може бути представлена за допомогою DOM («Document Object Model») - організація елементів html, якою можна маніпулювати, змінювати, видаляти або додавати. Для взаємодії з DOM застосовується мова JavaScript. Однак коли відбуваються маніпулювання з html-елементами за допомогою JavaScript, може виникнути зниження продуктивності, особливо при зміні великої кількості елементів. А операції над елементами можуть зайняти деякий час. Однак якби використовувались об'єкти JavaScript замість сторінки html, то операції проводилися б швидше. Для вирішення проблеми продуктивності якраз і з'явилася концепція віртуального DOM.

Віртуальний DOM представляє легковажну копію звичайного DOM. І відмінною рисою React є те, що дана бібліотека працює саме з віртуальним DOM, а не звичайним.

Якщо через додаток потрібно дізнатися інформацію про стан елементів, то відбувається звернення до віртуального DOM.

Якщо необхідно змінити елементи веб-сторінки, то зміни спочатку вносяться в віртуальний DOM. Потім новий стан віртуального DOM порівнюється з поточним станом. І якщо ці стани розрізняються, то React

знаходить мінімальну кількість маніпуляцій, які необхідні до поновлення реального DOM до нового стану і виконує їх.

У підсумку така схема взаємодії з елементами веб-сторінки працює набагато швидше і ефективніше, ніж якби ми працювали з JavaScript з DOM безпосередньо.

Іншою відмінною рисою бібліотеки є концентрація на компонентах – можливим є створення окремих компонентів і потім вони легко переносяться з проекту в проект.

Ще одна особливість React - використання JSX. JSX представляє комбінацію коду JavaScript і XML і надає простий і інтуїтивно зрозумілий спосіб для визначення коду візуального інтерфейсу [].

Angular.js є opensource JavaScript-фреймворк, який використовує шаблон MVC. Власне використання MVC є його однією з відмінних рис.

Для опису інтерфейсу використовується декларативне програмування, а бізнес-логіка відокремлена від коду інтерфейсу, що дозволяє поліпшити тестованість та розширюваність додатків.

Іншою відмінною рисою фреймворка є двостороннє зв'язування, що дозволяє динамічно змінювати дані в одному місці інтерфейсу при зміні даних моделі в другом. Таким чином, AngularJS синхронізує модель і уявлення.

Крім того, Angular.js підтримує такі функціональності, як Ajax, управління структорою DOM, анімація, шаблони, маршрутизація тощо. Наявність великого функціоналу вплинуло на те, що він знаходить своє застосування у все більшій кількості веб-додатків, будучи одним з найпопулярніших javascript-фреймворків [23].

Vue.js представляє сучасний прогресивний фреймворк, написаний на мові JavaScript і призначений для створення веб-додатків клієнтського рівня. Основна сфера застосування даного фреймворка - це створення і організація призначеного для користувача інтерфейсу.

Перший реліз фреймворку - лютий 2014 року. Його творцем є Еван Ю (Evan You), який до цього працював в Google над Angular.js. З тих пір фреймфорк динамічно розвивається.

Vue.js має досить невеликий розмір - не більше 20 кБ, і при цьому володіє хорошою продуктивністю. Даний фреймворк стає все більш популярним.

Одним з ключових моментів в роботі Vue.js є віртуальний DOM. Структура веб-сторінки, як правило, описується за допомогою DOM (Document Object Model), яка представляє організацію елементів html на сторінці. Для взаємодії з DOM (додавання, зміни, видалення html-елементів) застосовується JavaScript. Завдяки віртуальному DOM підвищується продуктивність програми.

Vue.js підтримується всіма браузерами, які сумісні з ECMAScript 5. На даний момент це все сучасні браузери, в тому числі IE11 [24].

TypeScript це мова програмування на основі JavaScript.

Розвиток TypeScript почалося в кінці 2012 року. Хоча він зародився в компанії Microsoft, і його фактичним творцем є програміст Андерс Хейлсберг, але даний проект відразу став розвиватися як OpenSource. І вже з самого початку нова мова стала швидко поширюватися в силу своєї гнучкості і продуктивності. Чимало проектів, які були написані на JavaScript, стали переноситися на TypeScript. Популярність і актуальність ідей нової мови привела до того, що ряд з цих ідей в подальшому стануть частиною нового стандарту JavaScript. Нова версія одного з найпопулярніших фреймворків для Web - Angular 2/4/5/6/7 повністю написана на TypeScript спільно компаніями Microsoft і Google.

Однак, здавалося б, навіщо потрібно ще одну мову програмування для клієнтської сторони в середовищі Web, якщо з усією тією ж самою роботою справляється і традиційний JavaScript, який використовується практично на кожному сайті, яким володіє безліч розробників і підтримка якого в співтоваристві програмістів досить висока . Але TypeScript це не просто новий JavaScript.

По-перше, слід зазначити, що TypeScript - це строго типізована і компільована мова, схожа до C # та іншої строго типізованої мови. Хоча на виході компілятор створює все той же JavaScript, який потім виконується браузером, однак сувора типізація зменшує кількість потенційних помилок, які могли б виникнути при розробці на JavaScript.

По-друге, TypeScript реалізує багато концепції, які властиві об'єктно-орієнтованим мовам, як, наприклад, спадкування, поліморфізм, інкапсуляція і модифікатори доступу тощо.

По-третє, потенціал TypeScript дозволяє швидше і простіше писати великі складні комплексні програми, відповідно їх легше підтримувати, розвивати, масштабувати і тестувати, ніж на стандартному JavaScript.

По-четверте, TypeScript розвивається як opensource-проект. Крім того, він є кросплатформним, а це означає, що для розробки ми можемо використовувати як Windows, так і MacOS або Linux.

У той же час TypeScript є надмножиною JavaScript, а це означає, що будь-яка програма на JS є програмою на TypeScript. В TS можна використовувати всі ті конструкції, які застосовуються в JS - ті ж оператори, умовні, циклічні конструкції. Більш того код на TS компілюється в javascript. В кінцевому рахунку, TS - це всього лише інструмент, який покликаний полегшити розробку додатків.

Генерується компілятором TypeScript в код JS та підтримується переважною більшістю браузерів, так як орієнтується насамперед на стандарт ECMAScript 3, хоча TS також підтримує і стандарти ECMAScript 5 і ECMAScript 2015/2017. Хоча в процесі розробки можна задати цільовий стандарт ECMAScript [25].

Angular 7 це фреймворк від компанії Google для створення клієнтських додатків. Перш за все він націлений на розробку SPA-рішень (Single Page Application), тобто односторінкових додатків. Angular є спадкоємцем іншого фреймворка Angular.js. У той же час Angular це не нова версія Angular.js, а принципово новий фреймворк.

Angular надає таку функціональність, як двостороннє зв'язування, що дозволяє динамічно змінювати дані в одному місці інтерфейсу при зміні даних моделі в іншому, шаблони, маршрутизація і так далі.

Однією з ключових особливостей Angular є те, що він використовує в якості мови програмування TypeScript. Але nevf обмеження мовою TypeScript. Можливо писати програми на Angular за допомогою таких мов як Dart або JavaScript. Однак TypeScript все таки є основною мовою для Angular [26].

Проаналізувавши кожну з технологій для розробки клієнської частини веб додатків, обрано мову програмування TypeScript та технологію Angular 7, оскільки особливість мови TypeScript – типізованість, якої немає у мові JavaScript, є ключовою перевагою при розробці складних програмних комплексів.

3.2 Програмна реалізація

При розробці візуальної частини на технології Angular 7 використано редактор Visual Studio Code, а при розробці серверної частини на мові C# – середовище розробки Visual Studio 2017.

Розглянемо розробку серверної частини.

При проектуванні бази даних програмного продукту використано Entity Framework Core та принцип побудови БД – «Code First».

Entity Framework (EF) Core - це спрощена, розширювана і кросплатформенна версія технології доступу до даних Entity Framework для .NET Core. EF Core може використовуватися як об'єктно реляційний модуль зіставлення, дозволяючи розробникам .NET працювати з базою даних за допомогою об'єктів .NET і усуваючи необхідність в написанні коду, необхідного для доступу до даних. EF Core підтримує безліч систем баз даних. У EF Core доступ до даних здійснюється за допомогою моделі. Модель складається з класів сутностей і похідного контексту, який представляє сеанс взаємодії з базою даних, дозволяючи запитувати і зберігати дані. Можливим є

створення моделі з існуючої бази даних, вручну складання коду моделі відповідній базі даних, або використовувати міграції EF, щоб створити базу даних з моделі (і модифікувати її в міру зміни моделі) [27]. Реалізується принцип побудови БД – «Code First». При його використанні формуються моделі, які являють собою таблиці БД, визначаються зв'язки між моделями. З використанням EF Core моделі і з’язки перетворюються в відповідні таблиці та зв'язки між цими таблицями. Такий принцип є гнучким, адже щоб провести маніпуляції над полями, таблицями бази даних, не потрібно щось змінювати в самій базі даних, а достатньо лише змінити моделі. База даних зміниться автоматично при її обновленні. Фрагмент використання Entity Framework Core (підключення до БД, всановлення ключових полів таблиць) наведено на рисунку 3.1.

```
namespace TimeTeacher.Data.DataBaseContext
{
    public class DataContext : IdentityDbContext<User>
    {
        private readonly IConfiguration _configuration;
        public virtual DbSet<Result> Results { get; set; }
        public virtual DbSet<TaskAnswer> TaskAnswers { get; set; }
        public virtual DbSet<Task> Tasks { get; set; }
        public virtual DbSet<TaskType> TaskTypes { get; set; }

        public DataContext(IConfiguration configuration)
        {
            _configuration = configuration;
        }
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(_configuration.GetConnectionString("DefaultConnection"));
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Result>()
                .HasKey(e => e.Id);

            modelBuilder.Entity<Task>()
                .HasKey(e => e.Id);

            modelBuilder.Entity<TaskAnswer>()
                .HasKey(e => e.Id);
        }
    }
}
```

```
"ConnectionStrings": {
  "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=TimeTeacherDb;Trusted_Connection=True;MultipleActiveResultSets=true"
}
```

Рисунок 3.1 – Фрагмент використання Entity Framework Core

Для розробки ідентифікації користувача використано «Microsoft AspNetCore Identity». Identity це система членства, що розширює функціональні можливості входу для додатків ASP.NET Core. Користувачі можуть створювати

обліковий запис входу відомостей, що зберігаються в identity або вони можуть використовувати зовнішнього постачальника входу. Постачальниками підтримуваних зовнішніх облікових записів є Facebook, Google, обліковий запис Майкрософт та Twitter. Фрагменти реалізації AspNetCore Identity наведено на рисунку 3.2.

```
namespace TimeTeacher.Models
{
    public class User : IdentityUser
    {
        public string Description {get; set;}
        public ICollection<Result> Results { get; set; }
    }
}
```

```
namespace Microsoft.AspNetCore.Identity
{
    public class IdentityUser<TKey> where TKey : IEquatable<TKey>
    {
        public IdentityUser();

        public IdentityUser(string userName);

        public virtual DateTimeOffset? LockoutEnd { get; set; }

        [PersonalData]
        public virtual bool TwoFactorEnabled { get; set; }

        [PersonalData]
        public virtual bool PhoneNumberConfirmed { get; set; }

        [ProtectedPersonalData]
        public virtual string PhoneNumber { get; set; }

        public virtual string ConcurrencyStamp { get; set; }

        public virtual string SecurityStamp { get; set; }

        public virtual string PasswordHash { get; set; }

        [PersonalData]
        public virtual bool EmailConfirmed { get; set; }

        public virtual string NormalizedEmail { get; set; }

        [ProtectedPersonalData]
        public virtual string Email { get; set; }

        public virtual string NormalizedUserName { get; set; }

        [ProtectedPersonalData]
        public virtual string UserName { get; set; }

        [PersonalData]
        public virtual TKey Id { get; set; }

        public virtual bool LockoutEnabled { get; set; }

        public virtual int AccessFailedCount { get; set; }

        public override string ToString();
    }
}
```

Рисунок 3.2 – Фрагменти реалізації AspNetCore Identity

Identity можна налаштувати за допомогою бази даних SQL Server для зберігання імен користувачів, паролі та дані профілю. Крім того, за допомогою інших постійних сховищ, наприклад, таблиці Azure. [28].

При авторизації зареєстрованого користувача на клієнську частину відправляється token, що використовується на клієнті для перевірки, чи авторизований користувач. У програмному комплексі «Time Teacher» використано JWT (JSON Web Tokens) – це JSON об'єкт, який визначений у стандарті RFC 7519. Вважається одним з безпечних способів передачі інформації. Для його створення необхідно визначити заголовок (header) із загальною інформацією про токен, корисні дані (payload), такі як id користувача, його роль та інше, і підписи (signature) [29]. Фрагмент реалізації JWT наведено на рисунку 3.3.

```
private string CreateJWTToken(UserViewModel user)
    {
      var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["JwtSettings:JwtKey"]));
      var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

      DateTime issuedAt = DateTime.UtcNow;
      DateTime expireDays = DateTime.Now.AddDays(Convert.ToInt64(_config["JwtSettings:ExpireDays"]));

      var claims = new[]
      {
                new Claim(ClaimTypes.Name, user.UserName),
                new Claim(JwtRegisteredClaimNames.NameId, user.UserId),

            }.ToList();
      claims.AddRange(user.UserRoles.Select(_ => new Claim(ClaimTypes.Role, _)));

      ClaimsIdentity claimsIdentity = new ClaimsIdentity(claims);

      var tokenHandler = new JwtSecurityTokenHandler();

      var token = (JwtSecurityToken)tokenHandler.CreateJwtSecurityToken(
          subject: claimsIdentity,
          notBefore: issuedAt,
          expires: expireDays,
          signingCredentials: creds);

      return tokenHandler.WriteToken(token);
    }
```

```
"JwtSettings": {
  "JwtKey": "52rezK02NW3Tw1PE3uuQqdsPbw4RO0ATCfYJovaNo8LrKlM80WgdwVG2b6abF0djr2efbXoFWshipnUU4QX1pFFjY6xX61cVJ0huaUeQf4G0VbI4zfUJFpuLflo68epG",
  "ExpireDays": "30"
}
```

Рисунок 3.3 – Фрагмент реалізації JWT

Клієнтська частина будується у папку проекту «wwwroot», а back-end використовує її для запуску програмного комплексу. Це відбувається за допомогою головних файлів «Program.cs» та «Startup.cs», а доступ до папки з

клієнтською чатиною описано в «HomeController.cs». Фрагменти реалізації відповідних модулів наведено на рисунку 3.4 а, б, в.

```
namespace TimeTeacherWeb
{
    public class Program
    {
    public static void Main(string[] args)
    {
        CreateWebHostBuilder(args)
          .Build().Run();
    }

    public static IWebHostBuilder CreateWebHostBuilder(string[] args) =>
        WebHost.CreateDefaultBuilder(args)
            .UseStartup<Startup>();
    }
}
```

а

```
namespace TimeTeacherWeb
{
    public class Startup
    {
        public void ConfigureServices(IServiceCollection services)
        {
          services.AddMvc();
          services.AddCore();
          services.AddCors(options =>
          {
            options.AddPolicy("CorsPolicy",
                builder => builder.AllowAnyOrigin()
                    .AllowAnyMethod()
                    .AllowAnyHeader()
                    .AllowCredentials()
                    .Build());
          });
        }

        public void Configure(IApplicationBuilder app,
            IHostingEnvironment env, IDbInitializer dbInitializer)
        {

          if (env.IsDevelopment())
          {
            app.UseDeveloperExceptionPage();
          }

          app.UseStaticFiles();
          app.UseDefaultFiles();

          app.UseMvc(routes =>
          {
            routes.MapRoute(
              name: "home",
              template: "{*url}",
              defaults: new { controller = "Home", action = "Index" });
          });

          Task.Run(async () => { await dbInitializer.Initialize(); }).Wait();
        }
    }
}
```

б

```
namespace TimeTeacherWeb.Controllers.Mvc
{
    public class HomeController : Controller
    {
        [AllowAnonymous]
        [HttpGet]
        public IActionResult Index()
        {
          return View("~/wwwroot/index.html");
        }
    }
```

в

Рисунок 3.4 – Фрагменти реалізації модулів для запуску сервера
а – «Program.cs»; б – «Startup.cs»; в – «HomeController.cs»

При розробці використано шаблон проектування «Dependency Injection», що дає програмному компоненту зовнішні залежності, використовуючи «Inversion of control» для отримання залежностей, що реалізований у проекті back-end за допомогою «IServiceCollection». Фрагменти реалізації «Dependency Injection» наведено на рисунку 3.5.

```
namespace TimeTeacher.Data.Extensions
{
    public static class DependencyInjection
    {
        public static void AddData(this IServiceCollection services)
        {
            services.AddDbContext<DataContext>();
            services.AddTransient<IObjectiveRepository, ObjectiveRepository>();
            services.AddTransient<IResultRepository, ResultRepository>();
            services.AddScoped<IDbInitializer, DbInitializer>();
        }
    }
}

namespace TimeTeacher.Core.Extensions
{
    public static class DependencyInjection
    {
        public static void AddCore(this IServiceCollection services)
        {
            services.AddData();
            services.AddIdentity<User, IdentityRole>(option =>
                { option.User.RequireUniqueEmail = true; })
                .AddEntityFrameworkStores<DataContext>()
                .AddDefaultTokenProviders();
            services.AddTransient<IObjectiveService, ObjectiveService>();
            services.AddTransient<IResultService, ResultService>();
        }
    }
}

namespace TimeTeacherWeb
{
    public class Startup
    {
        public void ConfigureServices(IServiceCollection services)
        {
          services.AddMvc();
          services.AddCore();

          ...
}
```

Рисунок 3.5 – Фрагменти реалізації «Dependency Injection»

Розглянемо розробку клієнтської частини, де використано ряд сучасних засобів при проектуванні програмних систем за допомогою Angular 7.

Для стилізації основних компонентів використано Angular Material, що надає пакет вбудованих стилізованих елементів сторінки браузера, що найчастіше використовуються. Також використано SASS - скриптову метамову, що інтерпретується в CSS для конкретизації стилів елементів системи.

«Token Interceptor» - клас, що призначений для роботи з JWT, що передається на клієнта від сервера. Відбувається перевірка чи юзер валідний, визначається його роль та загальні дані про користувача.

«JWT» зберігається в «Session Storage» за допомогою відповідного «Storage Service». При перезавантаженні сторінки відбувається доступ до «Session Storage», перевіряється токен і спрацьовує «Login Guard», що прописаний в налаштуваннях «routes» в секції «Can Active» для необхідних «route», – якщо користувач валідний, то залишається на тій же сторінці браузера, якщо ж ні – повертається на сторінку авторизації. Фрагмент реалізації «Session Storage» наведено на рисунку 3.6.

```
@Injectable()
export class StorageService {
    constructor() { }

    set(id: string, val: string, inLocalScope: boolean = false) {
      if (inLocalScope) {
        localStorage.setItem(id, JSON.stringify(val));
      } else {
        sessionStorage.setItem(id, JSON.stringify(val));
      }
    }

    get(id: string) {
      let val;
      val = localStorage.getItem(id);
      if (!val) {
        val = sessionStorage.getItem(id);
      }
      try {
          return val ? JSON.parse(val) : null;
      } catch (e) {
          return null;
      }
    }

    remove(id: any) {
      localStorage.removeItem(id);
      sessionStorage.removeItem(id);
    }

    clear() {
        localStorage.clear();
        sessionStorage.clear();
    }
}
```

Рисунок 3.6 – Фрагмент реалізації «Storage Service»

«Error Handler Interceptor» перевизначений клас призначений для опрацювання помилок користувача. Використовує «Notification Service» і за допомогою «Snack Bar» демонструє результат виконання операцій – як успіх операції, так і помилку виконання. Фрагмент реалізації «Error Handler Interceptor» наведено на рисунку 3.7, реалізація «Notification Service» на рисунку 3.8.

```
@Injectable()
export class ErrorHandlerInterceptor implements ErrorHandler {

  public ErrorSubject: Subject<string> = new Subject<string>();

  constructor(private inject: Injector) { }

  handleError(exception: any): void {

    const authService = this.inject.get(AuthService);
    const router = this.inject.get(Router);
    const zona = this.inject.get(NgZone);

    const status = exception.status;
    switch (status) {
      case 401: {
        this.ErrorSubject.next(exception.error);
        authService.logOut();
        zona.run(() => {
          router.navigate(['/account']);
        });
      }break;
      case 404: {
        this.ErrorSubject.next(exception.error);
      }break;
      case 403: {
        this.ErrorSubject.next(exception.error);
      }break;
      case 422: {
        this.ErrorSubject.next(exception.error);
      }break;
      case 504: {
        this.ErrorSubject.next(exception.error);
      }break;
      default: {
        console.error(exception);
      }
    }
  }
}
```

Рисунок 3.7 – Фрагмент реалізації «Error Handler Interceptor»

```
@Injectable()
export class NotificationService {
    constructor (private snackBar: MatSnackBar,
                 private errorHandler: ErrorHandler,
                 private zone: NgZone,
                 private matDialog: MatDialog,
                 private overlay: Overlay
    ) {
      (this.errorHandler as ErrorHandlerInterceptor).ErrorSubject.subscribe(error => {
        this.openNotification({message: error, type: 'error'});
      });
    }

    openNotification(data: BaseNotificationInterface) {
      this.zone.run(() => {
        this.snackBar.open(data.message, '', {
          duration: 2000,
          panelClass: [data.type ? data.type : ''],
          verticalPosition: 'bottom',
          horizontalPosition: 'center',
        });
      });
    }

    openModalWindow<T>(data: ModalWindowModel, constr: Type<T>) {
      const dialogRef = this.matDialog.open(constr, {
          height: data.height ? data.height : '100%',
          width: data.width ? data.width : '90%',
          maxWidth: data.maxWidth ? data.maxWidth : '100%',
          data: data.data ? data.data : null,
          panelClass: 'mat-dialog-panel'
      });

      return dialogRef.afterClosed();
    }
}
```

Рисунок 3.8 – Фрагмент реалізації «Notification Service»

Для визначення мови системи використано «Translate Service», що використовує тексти, написані у відповідному файлі «i18n.json». Фрагмент реалізації «Translate Service» наведено на рисунку 3.9.

```
...
import {TranslateModule, TranslateLoader} from '@ngx-translate/core';
import {TranslateHttpLoader} from '@ngx-translate/http-loader';
import {TranslateService} from '@ngx-translate/core';

export function HttpLoaderFactory(http: HttpClient) {
  return new TranslateHttpLoader(http, './assets/il8n/', '.json');
}

@NgModule({
  declarations: [
  ],
  imports: [
    ...
    TranslateModule.forRoot({
      loader: {
          provide: TranslateLoader,
          useFactory: HttpLoaderFactory,
          deps: [HttpClient]
      }
    })
    ...
   ],
  exports: [
  ],
  providers: [
    ...
    TranslateService,
    ...
   ],
  bootstrap: [ ]
})
export class CoreModule {
  // This guards against the CoreModule from being imported into any
    // module except the root-module (AppModule).
    constructor( @Optional() @SkipSelf() parentModule: CoreModule) {
      if (parentModule) {
          throw new Error('CoreModule is already loaded. Import it in the AppModule only');
      }
  }
 }
```

Рисунок 3.9 – Фрагмент реалізації «Translate Service»

Отже, виконано аналіз розробки функціональних особливостей та принципів при розробці програмного комплексу «TimeTeacher».

Фрагменти коду реалізації програмного комплексу наведено у додатку Б.

3.3 Висновки

В результаті, проведено аналіз мов програмування та технології розробки front-end, back-end програмного комплексу Time Teacher.

Серед мов програмування та технологій для back-end веб додатків – C# (ASP.NET Core), JavaScript (Node.js), Phython (Django) обрано C# (ASP.NET Core), оскільки надаються всі необхідні засоби для швидкої реалізації програмного комплексу «Time Teacher».

Серед мов програмування та технологій для front-end веб додатків – JavaScript (ReactJS, VueJS, AngularJS) та TypeScript (Angular 7) обрано

TypeScript (Angular 7), оскільки типізованість є ключовою для ефективної розробки front-end у програмному комплексі «Time Teacher». Використано середовище розробки середовище розробки Visual Studio 2017 та редактор Visual Studio Code.

Розглянуто реалізацію основного функціоналу програмного комплексу, особливості і підходи до розробки. Продемонстровано частини коду основних функцій та особливостей, що використовуються при реалізації програмного комплексу.

4 ТЕСТУВАННЯ ПРОГРАМИ

4.1 Методи тестування

Тестування програмного забезпечення – це:

1. Процес дослідження ПЗ з метою отримання інформації про якість продукту.

2. Процес перевірки відповідності заявлених до продукту вимог і реально реалізованої функціональності, здійснений шляхом спостереження за його роботою в штучно створених ситуаціях і на обмеженому наборі тестів, обраних певним чином.

3. Оцінка системи для того, щоб знайти відмінності між тим, якою система повинна бути і якою вона є.

У широкому сенсі, тестування – це одна з технік контролю якості (Quality Control), яка включає планування, складання тестів, безпосередньо виконання тестування і аналіз отриманих результатів [30].

Важливо розуміти, що тестування ПЗ включає в себе не тільки проведення тестів, але і багато інших дій, пов’язаних з процесом забезпечення якості:

1. Аналіз і планування.
2. Розробку тестових сценаріїв.
3. Оцінку критеріїв закінчення тестування.
4. Написання звітів.
5. Рецензування документації (в тому числі і вихідного коду).
6. Проведення статичного аналізу.

Розглянемо наступні методи тестування: «біла скринька» і «чорна скринька».

Метод тестування «біла скринька» виконується розробником (так як необхідно знати внутрішні принципи роботи програми) для перевірки внутрішньої структури програмного засобу. Об’єктом тестування є дані,

отримані шляхом аналізу логіки програми. Перевіряється коректність побудови всіх елементів програми та правильність їхньої взаємодії один з одним. Зазвичай аналізуються керуючі зв'язки елементів, рідше – інформаційні зв'язки. Тестування за принципом «білої скриньки» характеризується ступенем, в якому тести виконують або покривають логіку (вихідний текст) програми. Зазвичай тестування «білої скриньки» засноване на аналізі керуючої структури програми. Програма вважається повністю перевіреною, якщо проведено вичерпне тестування маршрутів (шляхів) її графа управління [30].

Принцип «білої скриньки» дозволяє врахувати особливості програмних помилок:

1. Попередні припущення про ймовірність потоку керування або даних у програмі часто бувають некоректними. У результаті типовим може стати маршрут, модель обчислень за яким опрацьована слабо.

2. Кількість помилок мінімально в «центрі» і максимально на «периферії» програми.

3. Деякі результати в програмі залежать не від вихідних даних, а від внутрішніх станів програми.

4. При записі алгоритму програмного забезпечення у вигляді тексту на мові програмування можливе внесення типових помилок трансляції (синтаксичних та семантичних).

Метод тестування «чорна скринька» використовується, коли тестувальнику не потрібно знати внутрішні властивості програми. При тестуванні «чорної скриньки» розглядаються системні характеристики програм, ігнорується їхня внутрішня логічна структура. Вичерпне тестування, як правило, неможливе. Тестування «чорної скриньки» не реагує на багато особливостей програмних помилок [31].

Тести демонструють:

1. Як виконуються функції програми.
2. Як приймаються вихідні дані.
3. Як виробляються результати.

4. Як зберігається цілісність зовнішньої інформації.

Тестування «чорної скриньки» (функціональне тестування) дозволяє отримати комбінації вхідних даних, які забезпечують повну перевірку всіх функціональних вимог до програми. Програмний виріб тут розглядається як «чорна скринька», чию поведінку можна визначити тільки дослідженням його входів та відповідних виходів. Принцип «чорної скриньки» – не альтернативний принципу «білої скриньки». Скоріше це доповнює підхід, який виявляє інший клас помилок.

Тестування «чорної скриньки» забезпечує пошук наступних категорій помилок:

1. Некоректних чи відсутніх функцій.
2. Помилок інтерфейсу.
3. Помилок у зовнішніх структурах даних або в доступі до зовнішньої бази даних.
4. Помилок характеристик (необхідна ємність пам'яті і т. д.).
5. Помилок ініціалізації та завершення.

Існує метод тестування «сіра скринька», яка створена на основі розглянутих методів. При роботі з даним методом тестувальник має доступ до коду програми, проте тестування проводить з точки зору кінцевого користувача. Суть даних методів не є складною, проте ефективність тестування за допомогою кожного з них вимагає хороших знань та навичок [31]. В результаті, обрано метод тестування «чорна скринька», оскільки за даним методом тестуються лише вхідні/вихідні дані.

Оскільки розроблено комп’ютерну гру, доцільним є провести тестування графічного інтерфейсу користувача для забезпечення його відповідності до даної специфікації. Завданням тестування графічного інтерфейсу користувача є виявлення помилок наступного характеру:

1. Помилки у функціональності за допомогою інтерфейсу.
2. Необроблені виключення при взаємодії з інтерфейсом.

3. Втрата або перекручування даних, переданих через елементи інтерфейсу.

4. Помилки в інтерфейсі (невідповідність проектної документації, відсутність елементів інтерфейсу).

Важливим для тестування інтерфейсу користувача є оцінка його зручності. Зручність використання графічного інтерфейсу виявляє міру простоти доступу користувача до функцій системи, наданих через інтерфейс програми. Тестування зручності використання інтерфейсу не належить до класичного методу тестування програмних систем. Фахівець по тестуванню користувальницького інтерфейсу має поєднувати у собі знання в галузі як програмної інженерії, так і фізіології, психології та ергономіки [31].

На зручність використання користувальницького інтерфейсу впливають такі чинники:

1. Легкість навчання – чи швидко людина навчається використовувати систему.
2. Ефективність навчання – чи швидко людина працює після навчання.
3. Помилки – чи часто людина припускається помилок у роботі.
4. Загальна задоволеність – чи є загальне враження від співпраці з системою позитивним.

Усі ці фактори, незважаючи на неформальність, можуть бути виміряні. Для таких вимірів вибирається група типових користувачів. Під час їх роботи вимірюються показники роботи системи, і їм пропонується висловити враження від інтерфейсу за допомогою заповнення опитувальних аркушів.

4.2 Інструкція тестування програми

Для програмного комплексу «Time Teacher» проводиться тестування графічного інтерфейсу користувача з використанням методу «чорна скринька». Для перевірки правильності роботи програмного комплексу проводимо

тестування загальних функцій та процесу ігрового функціоналу кожного з модулів.

Після запуску програмного комплексу відкривається сторінка авторизації. Сторінка призначена для авторизації користувача. Користувачу надається можливість ввести email, пароль та ввійти в систему за допомогою відповідної кнопки «Вхід». На сторінці є «header», що складається з назви програмного комплексу, зміни мови системи.

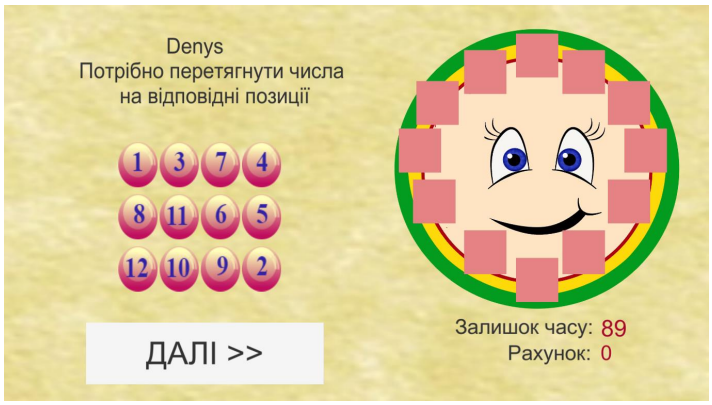
При натисненні кнопки «Реєстрація» відкриється відповідне вікно реєстрації. Користувач може ввести дані у відповідні поля та натиснути «Реєстрація». Якщо email уже наявний в базі даних, то з'явиться відповідне повідомлення про дубльованість email.

Після завершення реєстрації користувач повертається у вікно авторизації та вводить відповідні email та пароль, введені при реєстрації. При відсутності введених даних, або якщо дані введені некоректно, система відправить користувачу відповідне повідомлення. Коли користувач зареєструвався і ввійшов в систему з коректними email та паролем, відкривається сторінка програмного комплексу «Time Teacher».

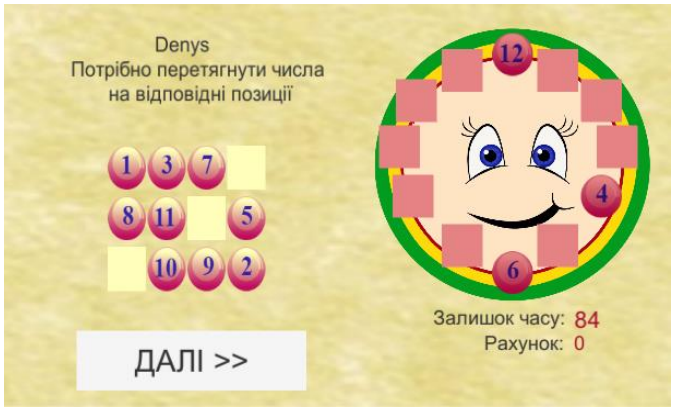
Коли користувач авторизований, у «header» сторінки додано кнопку «Вихід», що призначена для виходу користувача з програмного комплексу. При цьому відбувається повернення на сторінку авторизації. Головна інформація про користувача відображається у головному меню програмного комплексу.

Розглянемо роботу кожного з модулів ігрового комплексу.

Модуль «Перетягування чисел» («DragNumbers») зображено на рисунку 4.1 (а – початок гри, б – процес проходження гри).



а



б

Рисунок 4.1 – Інтерфейс модуля «DragNumbers»

а – початок ігри; б – процес проходження гри

Після запуску на ігровому полі відображена умова завдання. Числа, які розташовані в випадковому порядку, потрібно розмістити на відповідні позиції на годиннику. Після того, як всі числа розташовані на своїх місцях, небхідно натиснути кнопку «ДАЛІ». Якщо розставлено вірно, то нарахується 10 очок, і після цього з’явиться нова випадкова послідовність чисел, яку потрібно буде розмістити на годиннику. Якщо ж певні числа розставлені невірно, то запускається наступна послідовність чисел без додання очок. На гру дається 90 секунд. Протягом гри потрібно набрати максимально можливу кількість очок, поки час не закінчився.

Модуль «Час по стрілках» («TimeArrow») зображено на рисунку 4.2 (а – початок гри, б – процес проходження гри).



а



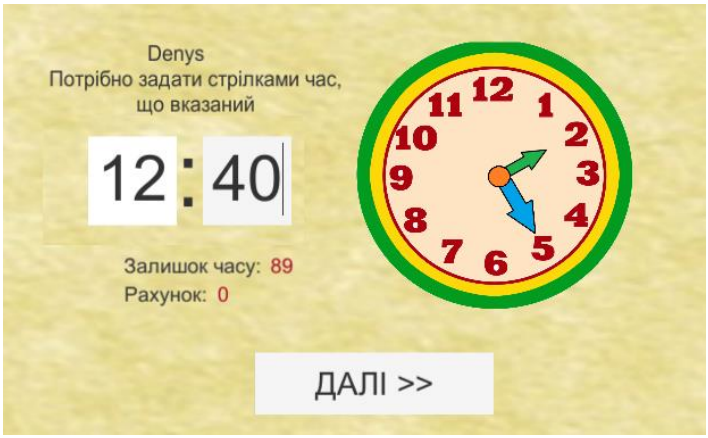
б

Рисунок 4.2 – Інтерфейс модуля «TimeArrow»

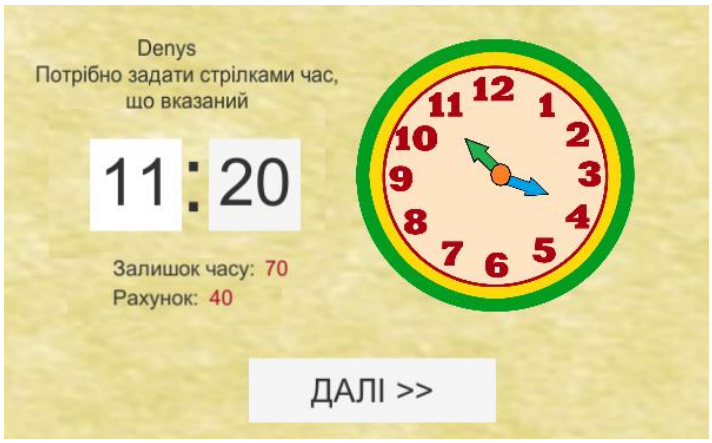
а – початок ігри; б – процес проходження гри

Після запуску модуля «TimeArrow» відображено умову завдання на ігровому полі. На годиннику за допомогою стрілок вказано час. Хвилинна стрілка – синього кольору, а годинна – зеленого. Необхідно у відповідні поля ввести значення годин та хвилин. Після введення небхідно натиснути кнопку «ДАЛІ». Якщо вказано час вірно, то нарахується 10 очок, і після цього з’явиться новий час на годиннику, який потрібно ввести. Якщо ж час було вказано невірно, то запускається наступна час на годиннику без додавння очок. На гру дається 90 секунд. Протягом гри потрібно набрати максимально можливу кількість очок, поки час не закінчився.

Модуль «Маніпуляції зі стрілками» («ArrowManipulation») зображено на рисунку 4.3 (а – початок гри, б – процес проходження гри).



а



б

Рисунок 4.3 – Інтерфейс модуля «ArrowManipulation»
а – початок ігри; б – процес проходження гри

Після запуску на ігровому полі відображено умову завдання. У відповідних полях вказано час, який необхідно виставити на годиннику за допомогою стрілок.

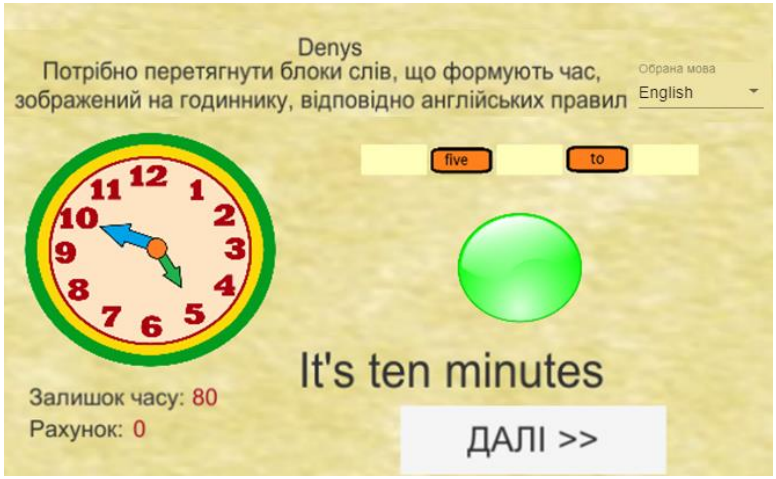
Хвилинна стрілка – синього кольору, а годинна – зеленого. Спочатку стрілки знаходяться у випадкових позиціях. Після налаштувань положення стрілок небхідно натиснути кнопку «ДАЛІ». Якщо вказано час на годиннику вірно, то нарахується 10 очок, і після цього з’явиться новий час, який потрібно

встановити. Якщо ж час було вказано невірно, то запускається наступна час на годиннику без додавння очок. На гру дається 90 секунд. Протягом гри потрібно набрати максимально можливу кількість очок, поки час не закінчився.

Модуль «Час відповідно правил обраної мови» («LanguageTime») зображено на рисунку 4.4 (а – початок гри, б – процес проходження гри).



а



б

Рисунок 4.4 – Інтерфейс модуля «LanguageTime»
а – початок ігри; б – процес проходження гри

Після запуску відображено умову завдання на ігровому полі. На годиннику зображено час. Хвилинна стрілка – синього кольору, а годинна –

зеленого. У відповідних блоках вказано слова на обраній мові світу, які в поєднанні формують речення, як вірно говорити час.

Потрібно переміщувати блоки у відповідну зелену круглу область, після чого блок зникає та формується відповідне речення. Після переміщення усіх блоків небхідно натиснути кнопку «ДАЛІ». Якщо вірно вказано час відповідно часу на годиннику, то нарахується 10 очок, і після цього з’явиться новий час. Якщо ж час було вказано невірно, то запускається наступна час на годиннику без додавння очок. На гру дається 90 секунд. Протягом гри потрібно набрати максимально можливу кількість очок, поки час не закінчився.

Після того, як час вичерпано, автоматично з’являється відповідний результат із проходження гри.

Після проведення тестування графічного інтерфейсу користувача з використанням методу «чорна скринька» виявлено повну відповідність між вхідними та вихідними даними, помилок при роботі програми не виявлено. Наявна система оброблення помилок у невідповідності введених даних. Тому методика тестування «чорна скринька» підтверджує нормальний режим роботи додатку.

Зображення інтерфейсу гри наведено у додатку В на рисунках В.1 – В.4.

4.3 Висновки

В результаті, розглянуто методи тестування «чорної скриньки» та «білої скриньки». Оскільки для програмного комплексу «Time Teacher» необхідно тестування вхідних і вихідних даних, обрано метод тестування «чорна скринька». Оскільки розробляється програмний комплекс з веб інтерфейсом, необхідним є тестування інтерфейсу користувача, щоб виявити та виправити помилки, що виникають під час роботи програми.

Проведено тестування модулів ігрового процесу програмного комплексу. В результаті виконання тестування помилок не було виявлено, цим самим підтверджено якісне функціонування додатку.

5 ЕКОНОМІЧНА ЧАСТИНА

5.1 Оцінювання комерційного потенціалу розробки

Метою проведення технологічного аудиту є оцінювання комерційного потенціалу розробки. Для проведення технологічного аудиту залучено 2-х незалежних експертів. Експерти – к.т.н., ст. викладач Бабюк Н. П. та к.т.н., доц. Черноволик Г. О.

Здійснюємо оцінювання комерційного потенціалу розробки за 12-ма критеріями за 5-ти бальною шкалою.

Результати оцінювання комерційного потенціалу розробки наведено в таблиці 5.1.

Таблиця 5.1 – Результати оцінювання комерційного потенціалу розробки

Критерії	Прізвище, ініціали, посада експерта	
	1. Черноволик Г. О.	2. Бабюк Н. П.
	Бали, виставлені експертами:	
1	4	4
2	3	3
3	3	4
4	4	4
5	3	4
6	3	4
7	3	3
8	3	4
9	4	4
10	4	3
11	3	4
12	3	4
Сума балів	$СБ_1 = 41$	$СБ_2 = 45$
Середньоарифметична сума балів $\overline{СБ}$	$\overline{СБ} = \frac{\sum_{1}^{3} СБ_i}{2} = 43$	

Отже, з отриманих даних таблиці 5.1 видно, що нова розробка має високий рівень комерційного потенціалу.

5.2 Прогнозування витрат на виконання науково-дослідної та конструкторсько–технологічної роботи.

Для розробки нового програмного продукту необхідні такі витрати.

Основна заробітна плата для розробників визначається за формулою (5.1):

$$З_о = \frac{М}{Т_р} \cdot t, \quad (5.1)$$

де М- місячний посадовий оклад конкретного розробника;

$Т_р$ - кількість робочих днів у місяці, $Т_р$ = 21 день;

t - число днів роботи розробника, t = 50 днів.

Розрахунки заробітних плат для керівника і програміста наведені в таблиці 5.2.

Таблиця 5.2 – Розрахунки основної заробітної плати

Працівник	Оклад М, грн.	Оплата за робочий день, грн.	Число днів роботи, t	Витрати на оплату праці, грн.
Науковий керівник	6000	285,71	5	1428,55
Інженер-програміст	3500	166,66	50	8333
Всього:				9761,55

Розрахуємо додаткову заробітну плату:

$$З_{дод} = 0,1 \cdot 9761,55 = 976,15 \text{ (грн.)}$$

Нарахування на заробітну плату операторів НЗП розраховується як 37,5…40% від суми їхньої основної та додаткової заробітної плати (за формулою 5.2):

$$Н_{зп}= (З_о + З_р)\cdot \frac{\beta}{100}, \quad (5.2)$$

$$Н_{зп}= (9761,55+976,15)\cdot \frac{36,3}{100} = 3897,81 \text{ (грн.)}.$$

Розрахунок амортизаційних витрат для програмного забезпечення виконується за формулою 5.3:

$$А= \frac{Ц\cdot Н_а}{100}\cdot \frac{Т}{12}, \quad (5.3)$$

де Ц – балансова вартість обладнання, грн;

$Н_а$ – річна норма амортизаційних відрахувань % (для програмного забезпечення 25%);

Т – Термін використання (Т=3 міс.).

Таблиця 5.3 – Розрахунок амортизаційних відрахувань

Найменування програмного забезпечення	Балансова вартість, грн.	Норма амортизації, %	Термін використання, міс.	Величина амортизаційних відрахувань,грн
Персональний комп’ютер	9000	25	3	562,5
Всього:				562,5

Розрахуємо витрати на комплектуючі. Витрати на комплектуючі розрахуємо за формулою 5.4:

$$К = \sum_{1}^{n} Н_i \cdot Ц_i \cdot К_i, \quad (5.4)$$

де n – кількість комплектуючих;

Ні - кількість комплектуючих і-го виду;

Ці – покупна ціна комплектуючих і-го виду, грн;

Кі – коефіцієнт транспортних витрат (приймемо Кі = 1,1).

Таблиця 5.4 - Витрати на комплектуючі, що були використані для розробки ПЗ

Найменування матеріалу	Одиниці виміру	Ціна, грн.	Витрачено	Вартість витрачених матеріалів, грн.
Флешка	шт.	200	1	200
Пачка паперу	уп.	120	1	120
Ручка	шт.	5	1	5
Всього з урахуванням транспортних витрат				357,5

Витрати на силову електроенергію розраховуються за формулою 5.5:

$$В_е = В \cdot П \cdot Ф \cdot К_п, \quad (5.5)$$

де В – вартість 1кВт-години електроенергії (В=1,7 грн/кВт);

П – установлена потужність комп’ютера (П=0,6кВт);

Ф – фактична кількість годин роботи комп’ютера (Ф=200 год.);

$К_п$ – коефіцієнт використання потужності ($К_п < 1$, $К_п = 0,9$).

$$В_е = 1,7 \cdot 0,6\ 200 \cdot 0,9 = 183,6 \text{ (грн.)}$$

Розрахуємо інші витрати $В_{ін}$.

Інші витрати $І_в$ можна прийняти як (100…300)% від суми основної заробітної плати розробників та робітників, які були виконували дану роботу, тобто (формула 5.6):

$$В_{ін} = (1..3) \cdot (З_о + З_р) \quad (5.6)$$

Отже, розрахуємо інші витрати:

$$В_{ін} = 1 * (9761,55 + 976,15) = 16106,65 \text{ (грн).}$$

Сума всіх попередніх статей витрат дає витрати на виконання даної частини роботи (формула 5.7):

$$В = З_о + З_д + Н_{зп} + А + К + В_е + І_в \quad (5.7)$$

$$В = 9761,55 + 976,15 + 3897,81 + 562,5 + 183,6 + 357,5 + 16106,65 = 31845,76 \text{ (грн.)}$$

Розрахуємо загальну вартість наукової роботи $B_{заг}$ за формулою 5.8:

$$В_{заг} = \frac{В_{ін}}{\alpha} \quad (5.8)$$

де α – частка витрат, які безпосередньо здійснює виконавець даного етапу роботи, у відн. одиницях = 1.

$$В_{заг} = \frac{31845,76}{1} = 31845,76$$

Прогнозування загальних витрат ЗВ на виконання та впровадження результатів виконаної наукової роботи здійснюється за формулою 5.9:

$$ЗВ = \frac{В_{заг}}{\beta}, \quad (5.9)$$

де β – коефіцієнт, який характеризує етап (стадію) виконання даної роботи.

Отже, розрахуємо загальні витрати:

$$ЗВ = \frac{31845,76}{0,9} = 35384,17 \text{ (грн.)}$$

5.3 Прогнозування комерційних ефектів від реалізації результатів розробки.

Спрогнозуємо отримання прибутку від реалізації результатів нашої розробки. Зростання чистого прибутку можна оцінити у теперішній вартості грошей. Це забезпечить підприємству (організації) надходження додаткових коштів, які дозволять покращити фінансові результати діяльності .

Оцінка зростання чистого прибутку підприємства від впровадження результатів наукової розробки. У цьому випадку збільшення чистого прибутку підприємства $\Delta П_i$ для кожного із років, протягом яких очікується отримання позитивних результатів від впровадження розробки, розраховується за формулою 5.10:

$$\Delta П_i = \sum_1^n (\Delta П_я \cdot N + П_я \Delta N)_i \quad (5.10)$$

де $\Delta П_я$ – покращення основного якісного показника від впровадження результатів розробки у даному році;

N – основний кількісний показник, який визначає діяльність підприємства у даному році до впровадження результатів наукової розробки;

ΔN – покращення основного кількісного показника діяльності підприємства від впровадження результатів розробки;

$П_я$ – основний якісний показник, який визначає діяльність підприємства у даному році після впровадження результатів наукової розробки;

n – кількість років, протягом яких очікується отримання позитивних результатів від впровадження розробки.

В результаті впровадження результатів наукової розробки витрати на виготовлення програмного продукту зменшаться на 15 грн (що автоматично спричинить збільшення чистого прибутку підприємства на 15 грн), а кількість користувачів, які будуть користуватись збільшиться: протягом першого року – на 300 користувачів, протягом другого року – на 250 користувачів, протягом третього року – 175 користувачів. Реалізація програмного продукту до впровадження результатів наукової розробки складала 800 користувачів, а прибуток, що отримував розробник до впровадження результатів наукової розробки – 250 грн.

Спрогнозуємо збільшення чистого прибутку від впровадження результатів наукової розробки у кожному році відносно базового.

Отже, збільшення чистого продукту $\Delta П_1$ протягом першого року складатиме:

$$\Delta П_1 = 15 \cdot 800 + (250 + 15) \cdot 300 = 91500 \text{ грн.}$$

Протягом другого року:

$$\Delta П_2 = 15 \cdot 800 + (250 + 15) \cdot (300 + 250) = 157750 \text{ грн.}$$

Протягом третього року:

$$\Delta П_3 = 15 \cdot 800 + (250 + 15) \cdot (300 + 250 + 175) = 204125 \text{ грн.}$$

5.4 Розрахунок ефективності вкладених інвестицій та період їх окупності

Визначимо абсолютну і відносну ефективність вкладених інвестором інвестицій та розрахуємо термін окупності.

Абсолютна ефективність $Е_{абс}$ вкладених інвестицій розраховується за формулою 5.11:

$$Е_{абс} = (ПП - PV), \quad (5.11)$$

де $\Delta П_i$ – збільшення чистого прибутку у кожному із років, протягом яких виявляються результати виконаної та впровадженої НДДКР, грн;

т – період часу, протягом якого виявляються результати впровадженої НДДКР, 3 роки;

τ – ставка дисконтування, за яку можна взяти щорічний прогнозований рівень інфляції в країні; для України цей показник знаходиться на рівні 0,1;

t – період часу (в роках) від моменту отримання чистого прибутку до точки 2, 3,4.

Рисунок, що характеризує рух платежів (інвестицій та додаткових прибутків) буде мати вигляд, рисунок 5.1.

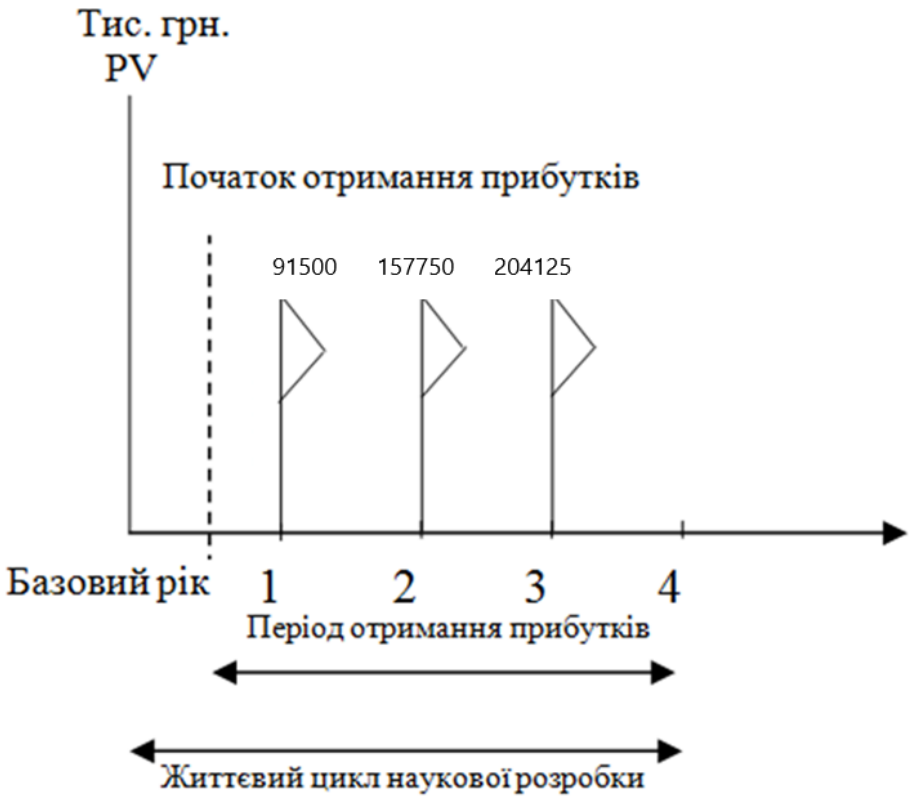


Рисунок 5.1 – Вісь часу з фіксацією платежів, що мають місце під час розробки та впровадження результатів НДДКР

Розрахуємо вартість чистих прибутків за формулою 5.12:

$$ПП = \sum_1^m \frac{\Delta П_i}{(1+\tau)^t}\ , \qquad (5.12)$$

де $\Delta П_i$ – збільшення чистого прибутку у кожному із років, протягом яких виявляються результати виконаної та впровадженої НДДКР, грн;

т – період часу, протягом якого виявляються результати впровадженої НДДКР, роки;

τ – ставка дисконтування, за яку можна взяти щорічний прогнозований рівень інфляції в країні; для України цей показник знаходиться на рівні 0,1;

t – період часу (в роках) від моменту отримання чистого прибутку до точки.

Отже, розрахуємо вартість чистого прибутку:

$$ПП = \frac{35384,17}{(1+0,1)^0} + \frac{91500}{(1+0,1)^2} + \frac{157750}{(1+0,1)^3} + \frac{204125}{(1+0,1)^4} = 368944,03 \text{ (грн.)}$$

Тоді розрахуємо $E_{абс}$:

$$Е_{абс} = 368944,03 - 35384,17 = 333559,86 \text{ грн.}$$

Оскільки $E_{абс} > 0$, то вкладання коштів на виконання та впровадження результатів НДДКР буде доцільним.

Розрахуємо відносну (щорічну) ефективність вкладених в наукову розробку інвестицій $Е_в$ за формулою 5.13:

$$Е_в = \sqrt[Т]{1 + \frac{Е_{абс}}{PV}} - 1, \qquad (5.13)$$

де $Е_{абс}$ – абсолютна ефективність вкладених інвестицій, грн;

PV – теперішня вартість інвестицій PV = ЗВ, грн;

$Т_ж$ – життєвий цикл наукової розробки, роки.

Тоді будемо мати:

$$Е_в = \sqrt[3]{1 + \frac{333559,86}{35384,17}} - 1 = 1,18 \text{ або } 118\ \%$$

Далі, розраховану величина $Е_в$ порівнюємо з мінімальною (бар'єрною) ставкою дисконтування $\tau_{мін}$, яка визначає ту мінімальну дохідність, нижче за яку інвестиції вкладатися не будуть. У загальному вигляді мінімальна (бар'єрна) ставка дисконтування $\tau_{мін}$ визначається за формулою 5.14:

$$\tau = d + f, \tag{5.14}$$

де d – середньозважена ставка за депозитними операціями в комерційних банках; в 2018 році в Україні d = 0,2;

f – показник, що характеризує ризикованість вкладень, величина f = 0,1.

$$\tau=0,2+0,1=0,3$$

Оскільки $Е_в$ = 118% > $\tau_{мін}$ = 0,3 = 30%, то у інвестор буде зацікавлений вкладати гроші в дану наукову розробку.

Термін окупності вкладених у реалізацію наукового проекту інвестицій. Термін окупності вкладених у реалізацію наукового проекту інвестицій $Т_{ок}$ розраховується за формулою 5.15:

$$Т_{ок} = \frac{1}{Е_в} \tag{5.15}$$

$$Т_{ок} = \frac{1}{1,18} = 0,84 \text{ року}$$

Отже, обрахувавши термін окупності наукової розробки «Time Teacher» виявлено, що фінансування даної наукової розробки є доцільним.

5.5 Висновки

Після оцінювання розробки експертами виявлено, що розробка програмного комплексу має високий рівень комерційного потенціалу.

Загальні витрати на розробку складають 35 384,17 грн.

Вартість чистого прибутку – 368 944,03 грн (>0).

Абсолютна ефективність вкладених інвестицій Еабс – 333 559,86 грн (>0).

Відносна (щорічна) ефективність вкладених інвестицій Ев – 118% (>30%).

Період окупності – 0,84 року (< 3 років).

Враховуючи всі фактори, фінансування наукової розробки є доцільним.

ВИСНОВКИ

У результаті написання магістерської кваліфікаційної роботи розроблено програмний комплекс «Time Teacher». Проведено аналіз предметної області та виявлено, що проблема пізнання часу дітьми є актуальною. Для її вирішення вже існує велика кількість методик при навчанні дітей у закладах освіти. Також потрібно враховувати відповідності віку дитини та інформації, яку вона може сприймати при покращенні пізнання часу.

Оскільки зростає популярність навчальних комп'ютерних ігор для дітей, виявлено, що за їх допомогою вирішується розглянута проблема. У іграх-аналогах застосовуються різні підходи, тому вирішено поєднати їх у один програмний комплекс, додати вдосконалений підхід – вивчення часу відповідно до правил обраної мови – для максимального охоплення основних аспектів у вивченні пізнання часу. У програмному комплексі при навчанні застосовано метод програмованого навчання, що також впливає на швидкість засвоєння матеріалу. Таким чином, доведено доцільність розробки програмного комплексу «Time Teacher». Розроблено загальний сценарій ігрового процесу та додаткові можливості програми. Також спроектовано основні задачі, які потрібно виконати при розробці додатку, сформовано відповідне технічне завдання.

Наступним кроком створено архітектуру додатку за допомогою діаграм варіантів використання, послідовності, класів, структурної карти Константайна, структурної схема компонентів. Спроектовано базу даних програмного комплексу, а також розроблено графічні схеми інтерфейсів головної сторінки та основних модулів програмного комплексу, підібрано кольорову гамму та розроблено загальний алгоритм роботи.

Проаналізовано засоби та методи для розробки веб додатків. Кожен з них має свої особливості. Проте, для реалізації програмного комплексу «Time Teacher» обрано мовою програмування та технологією для back-end веб додаику – C# (ASP.NET Core), а мовою програмування та технологію для front-

end веб додатку – TypeScript (Angular 7). Наведено фрагмент реалізації основного функціоналу та принципів при розробці програмного комплексу.

Виконано тестування графічного інтерфейсу додатку з використанням методу тестування – «чорна скринька». Внаслідок перевірки роботи додатку на відповідність очікуваного та фактичного результатів не виявлено жодних помилок.

Проведено економічне обгрунтування розробки програмного комплексу «Time Teacher». Обрахувавши термін окупності наукової розробки (0,84 року) досліджено, що фінансування наукової розробки є доцільним.

Економічне обгрунтування виконано згідно методичних вказівок [32].

Ілюстративний матеріал до захисту магістерської кваліфікаційної роботи розглянуто у додатку Г на плакатах 1 – 20.

ПЕРЕЛІК ПОСИЛАНЬ

1. Подогова А. Є. Вплив комп'ютерних ігор на рівень агресивності підлітків. / А. Є. Подогова. – 5 видавництво. – Санкт–Петербурзький державний університет, 2004. – 88 с.

2. Слободяник Д. С., Бабюк Н.П. Розробка дитячої комп'ютерної гри «TeachTime» / Д. С. Слободяник, Н.П. Бабюк – Вінниця: ВНТУ, 2017.

3. Слободяник Д. С., Бабюк Н.П. Розробка методу вивчення часу на англійській мові для дитячої комп'ютерної гри «TeachTime» / Д. С. Слободяник, Н.П. Бабюк – Вінниця: ВНТУ, 2017.

4. Державний стандарт початкової загальної освіти // Початкова школа. – 2011 – №7. – Січень. – 18 с.

5. Кутузова В.В. Використання комп'ютерних дидактичних ігор для розвитку когнітивної сфери дошкільників / В.В. Кутузова. – Миколаївська обл., м. Первомайськ, 2017.

6. Кравченко Т. В. Соціалізація особистості і соціальне середовище // Теоретико-методологічні проблеми дітей та учнівської молоді: Зб. Наук. пр. - К., 2006 - Вип.9. Кн.2. - С. 23-29.

7. Лукша Т. Крокуємо з малечею дорогою адаптації / Т. Лукша, С. Немержицька, Н. Соколова // Психолог дошкілля. – 2012. – 150 с.

8. Гра «Годинник із зозулею» [Електронний ресурс]. Режим доступу: https://www.igraemsa.ru/igry-dlja-detej/poznavatelnye-igry/igra-chasy-s-kukushkoj.

9. Гра «Вивчаємо час» [Електронний ресурс]. Режим доступу: http://www.igraemsa.ru/igry-dlja-detej/poznavatelnye-igry/uchim-vremya.

10. Гра «READING the clock» [Електронний ресурс]. Режим доступу: https://play.google.com/store/apps/details?id=pl.multipro.clockforkids&hl=uk.

11. Структурные карты Константайна – [Електронний ресурс]. – Режим доступу: http://www.interface.ru/fset.asp?Url=/case/defs71.htm.

12. Кузін А. В. Базы данных, 5-е издание / А. В. Кузін, С. В. Левонисова – К. : «Академия», 2012. – 317 с.

13. Принципи побудови користувальницького веб інтерфейсу [Електронний ресурс]. Режим доступу: https://stfalcon.com/ru/blog/post/user-interface-web-design-principles.

14. Терещенко П. В. Інтерфейси інформаційних систем. / П. В. Терещенко. – Київ, 2012. – 650 с.

15. Романюк О. Н., Катєльніков Д. І. Веб-дизайн і комп'ютерна графіка. / О.Н. Романюк, Д. І. Катєльніков. – Вінниця: ВНТУ, 2007. – 144 с.

16. Рихтер Д. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. / Д. Рихтер. – Москва, 2017. – 896 с.

17. Особливості технології ASP.NET Core [Електронний ресурс]. Режим доступу: https://metanit.com/sharp/aspnet5/1.1.php.

18. Ніксон Р. Создаем динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5. 3-е издание / Р. Ніксон – К. : «Питер», 2015. – 688 с.

19. Закас Николас. ECMAScript 6 для разработчиков. / Н. Закас. – СПб: Питер, 2017. – 352 с.

20. Особливості мови Python [Електронний ресурс]. Режим доступу: https://metanit.com/python/tutorial/1.1.php.

21. Adrian Holovaty, Jacob K. Moss. Django The Definitive Guide to Django: Web Development Done Right. / A. Holovaty, J. K. Moss. – Taxas, 2018. – 358 pages.

22. Особливості технології React.js [Електронний ресурс]. Режим доступу: https://metanit.com/web/react/1.1.php.

23. Козловский П., Дарвин П. Б. Разработка веб-приложений с использованием AngularJS. / П. Козловский, П. Б. Дарвин. – Пер. с англ. Киселева А. Н., М.: ДМК Пресс, 2014. – 394 с.

24. Macrae Callum. Vue.js: Up and Running. / C. Macrae. – O'Reilly Media; February 2018. – 435 pages.

25. Basarat Ali Syed. TypeScript Deep Dive. / A. S. Basarat. – LA, 2018. – 334 pages.

26. Murray Nate, Coury Felipe, Lerner Ari, Taborda Carlos. Ng-book 2 The Complete Book to Angular. / N. Murray, F. Coury, A. Lerner, C. Taborda. – San Francisco, California, 2017. – 655 pages.

27. Entity Framework (EF) Core [Електронний ресурс]. Режим доступу: https://docs.microsoft.com/ru-ru/ef/core/.

28. Barbettini Nate. Little ASP.NET Core Book. / N. Barbettini. – New York, 2018. – 124 pages.

29. JWT (JSON Web Tokens) [Електронний ресурс]. Режим доступу: https://habr.com/post/340146/.

30. Степанченко И. В. Методы тестирования программного обеспечения: Учеб. пособие / И. В. Степанчеко. – ВолгГТУ. – Волгоград, 2006. – 74 с.

31. Бейзер Б. Тестирование чёрного ящика. Технологии функционального тестирования программного обеспечения и систем. / Б. Бейзер – СПб.: Питер, 2004. – 320 с.

32. Методичні вказівки до виконання студентами-магістрантами економічної частини магістерських кваліфікаційних робіт / Уклад. В. О. Козловський – Вінниця: ВНТУ, 2012. – 22 с.

ДОДАТКИ

Додаток А. Технічне завдання

Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

ЗАТВЕРДЖУЮ
Завідувач кафедрою ПЗ
_____А. М. Пєтух
"____"_____ _____ р.

ТЕХНІЧНЕ ЗАВДАННЯ
на магістерську кваліфікаційну роботу за спеціальністю
121 – Інженерія програмного забезпечення

Керівник магістерської кваліфікаційної роботи:
_____ ст. викладач, к.т.н. Бабюк Н.П.
"____"_____ _____ р.

Виконав:
_____студент гр.1ПІ-17м Слободяник Д. С.
"____"_____ _____ р.

ВНТУ –

1. Вступ

1.1. При освоєнні пізнання часу учнями молодших класів виникає велика кількість проблем, оскільки не всі засвоюють отримані знання. До того ж, на дану проблему виділяється занадто мало часу, що також є одним з ключових факторів. Крім того, вивчення часу на різних мовах світу позитивно впливає на розвиток дитини. Отже, розробка програмного комплексу, що складається з ігрового підходу до вирішення проблеми пізняння користувачів

1.2. Мета – вирішення проблеми вивчення та засвоєння користувачів пізнання часу.

1.3. Ціль – аналіз існуючих методів вивчення пізнання часу дітьми та винайдення кращого методу, порівняно з існуючими.

1.4. Задачі:

1.4.1. Аналіз проблеми навчання орієнтації користувачів в часі та пошук шляхів її вирішення.

1.4.2. Ознайомитись з можливостями базових функцій для розробки програми.

1.4.3. Створення загального алгоритму роботи програми.

1.4.4. Розробка UML-діаграм для демонстрації роботи програми.

1.4.5. Розробка графічного інтерфейсу програми.

1.4.6. Вибір методів та засобів розробки програми.

1.4.7. Програмування логіки програми.

1.4.8. Тестування й відлагодження продукту.

1.4.9. Економічне обгрунтування розробки програмного комплексу.

1.4.10. Випуск готової версії програмного комплексу «Time Teacher».

2. Основа для розробки

2.1. Програмний комплекс розробляється на основі індивідуального завдання до магістерської кваліфікаційної роботи, яке затверджене на засіданні кафедри програмного забезпечення – протокол № _____ від _____.

2.2. Назва програми.

Програмний комплекс «Time Teacher» призначений для навчання

користувачів пізнання часу.

2.3. Виконавець: ст.гр. 1ПІ–17м Слободяник Денис Сергійович.

3. Призначення

3.1. Цільова аудиторія.

3.1.1. Програмний комплекс «Time Teacher» призначений для користувачів віком від 6 до 10 років (дітей), яких навчають, як використовувати годинник і пізнавати часовий простір. Крім того, користувачами можуть бути люди молодшого/старшого віку, проте їх частка значно менша, ніж основна вікова категорія.

3.1.2. Використовується як в закладах освіти, так і як для персонального використання для поглиблення знань часу.

3.2. Вимоги до програмного виробу.

3.2.1. Основні функції.

3.2.1.1. Авторизація \ реєстрація користувача.

3.2.1.2. Надання можливості вибору режиму гри.

3.2.1.3. Демонстрація завдань відповідного режиму.

3.2.1.3.1. Перетягування арабських чисел на відповідні позиції годинника.

3.2.1.3.2. Введення значення часу по відповідному розташуванню стрілок.

3.2.1.3.3. Маніпуляції зі стрілками годинника.

3.2.1.3.4. Введення часу відповідно до правил обраної мови, зокрема англійської.

3.2.1.4. Коректна робота при маніпуляції елементами програмного комплексу.

3.2.1.5. Наведення статистики проходження гри.

3.2.1.6. Розробка системи адміністрування ігрового функціоналу.

3.3. Початкові дані.

3.3.1. Вхідними даними є вибір режиму гри.

3.3.2. Вихідними даними є результати пройденого рівня гри, а також наведена статистика по результатам відповідного рівня.

Загальна схема програмного комплексу «Time Teacher» (рисунок А.1). «Account page» – сторінка авторизації/реєстрації користувача, «Main page» – сторінка, що містить вибір одного з режимів ігрового процесу.

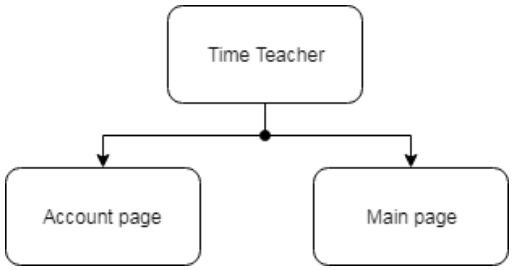


Рисунок А.1 – Загальна схема додатку «Time Teacher»

3.4. Вимоги до функціонування та надійності програмного продукту.

3.4.1. Програма повинна функціонувати при безперебійній роботі браузера, на якому її запущено.

3.4.2. Час відновлення після відмови має складатися з: часу запиту користувачем лінку до програмного комлплексу; часу завантаження стану елементів інтерфейсу програми.

3.4.3. Наявність оброблення виключень та попереджувальних повідомлень.

3.4.4. Захищеність програми логіном та паролем.

3.5. Вимоги до програмної сумісності.

3.5.1. Програма повинна бути сумісна з версіями браузерів: Chrome 21+, Safari 6.1+, Firefox 22+, Opera 12+, IE 11+, Edge 17+.

3.6. Вимоги до параметрів технічних засобів.

3.6.1. Мінімальні вимоги.

3.6.1.1. Процесор з частотою до 1.6 ГГц.

3.6.1.2. Об’єм оперативної пам’яті 512 МБ.

3.6.1.3. Швидкість інтернет з’єднання – 1 МБ/с.

3.6.2. Номінальні вимоги.

3.6.2.1. Процесор з частотою 1.6 ГГц.

3.6.2.2. Об’єм оперативної пам’яті 1 ГБ.

3.6.2.3. Швидкість інтернет з’єднання – 5 МБ/с.

3.7. Спеціальні вимоги.

3.7.1. Мова програмування та технологія розробки back-end – C# (ASP.NET Core 2.1).

3.7.2. Мова програмування та технологія розробки front-end – TypeScript (Angular 7).

4. Вимоги до програмної документації

4.1. Розроблена програма повинна включати довідкову інформацію про роботу програми.Текст програми повинен містити всі необхідні коментарі.

4.3. Наявне «Технічне завдання».

5. Стадії і етапи розробки

1.1 Завдання на роботу видане _____ _____ року. Розробка та дослідження повинні бути завершеними до _____ _____ року.

№ з/п	Назва етапів кваліфікаційної магістерської роботи	Строк виконання етапів роботи	Примітка
1	Аналіз проблеми, обґрунтування актуальності розробки програмного комплексу та постановка задачі		Вик.
2	Розробка архітектури та алгоритмів роботи програмнго комплексу, проектування інтерфейсу		Вик.
3	Вибір мови та технології розробки		Вик.
4	Розробка програмного продукту		Вик.
5	Тестування роботи додатку		Вик.
6	Економічне обгрунтування розробки програмного комплексу		Вик.
7	Оформлення матеріалів до захисту МКР		Вик.

1.2 Порядок контролю і приймання роботи регламентується відповідними документами ВНТУ і державними стандартами.

Додаток Б. Лістинг програми

«appsettings.json»

```
{
  "JwtSettings": {
    "JwtKey": "52rezK02NW3Tw1PE3uuQqdsPbw4RO0ATCfYJovaNo8LrK1M80WgdwVG2b6abF0djr2efbXoFWshipnUU4QX1pFFjY6xX61cVJ0huaUeQf4G0VbI4zfUJFpuLf1o68epG",
    "ExpireDays": "30"
  },
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=TimeTeacherDb;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

«DataContext.cs»

```
namespace TimeTeacher.Data.DataBaseContext
{
    public class DataContext : IdentityDbContext<User>
    {
        private readonly IConfiguration _configuration;
        public virtual DbSet<Result> Results { get; set; }
        public virtual DbSet<TaskAnswer> TaskAnswers { get; set; }
        public virtual DbSet<Task> Tasks { get; set; }
        public virtual DbSet<TaskType> TaskTypes { get; set; }
        public DataContext(IConfiguration configuration)
        {
            _configuration = configuration;
        }
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(_configuration.GetConnectionString("DefaultConnection"));
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<Result>()
```

```
            .HasKey(e => e.Id);
        modelBuilder.Entity<Task>()
            .HasKey(e => e.Id);
        modelBuilder.Entity<TaskAnswer>()
            .HasKey(e => e.Id);
    }
  }
}
```

«DbInitializer.cs»

```
namespace TimeTeacher.Data.DataBaseContext
{
  public class DbInitializer : IDbInitializer
  {
    private readonly DataContext _context;
    private readonly UserManager<User> _userManager;
    private readonly RoleManager<IdentityRole> _roleManager;

    public DbInitializer(DataContext context, UserManager<User> userManager, RoleManager<IdentityRole>
roleManager)
    {
      _context = context;
      _userManager = userManager;
      _roleManager = roleManager;
    }

    public async Task Initialize()
    {
      _context.Database.EnsureCreated();

      await SeedRoles(_roleManager);
      await SeedUsers(_userManager, _roleManager);
    }
```

```
        private async Task SeedRoles(RoleManager<IdentityRole> roleManager)
        {
            //Create Admin role
            bool x = await roleManager.RoleExistsAsync("Admin");
            if (!x)
            {
                var role = new IdentityRole { Name = "Admin" };
                await roleManager.CreateAsync(role);
            }

            //Creating Player role
            x = await roleManager.RoleExistsAsync("Player");
            if (!x)
            {
                var role = new IdentityRole { Name = "Player" };
                await roleManager.CreateAsync(role);
            }
        }

        private async Task SeedUsers(UserManager<User> userManager, RoleManager<IdentityRole> roleManager)
        {
            var roles = roleManager.Roles;

            var adminUser = new User { UserName = "DenysSlobodianyk", Email = "denys.slobodianyk@gmail.com" };
            var adminUserPassword = "0ZAQ!1qaz0";
            var adminUserCreating = await userManager.CreateAsync(adminUser, adminUserPassword);
            if (adminUserCreating.Succeeded)
            {
                var role = roles.FirstOrDefault(r => r.Name == "Admin");
                var res = await userManager.AddToRoleAsync(adminUser, role.Name);
            }
        }
    }
}
```

«DependencyInjection.cs»

```
namespace TimeTeacher.Data.Extensions
{
   public static class DependencyInjection
   {
      public static void AddData(this IServiceCollection services)
      {
         services.AddDbContext<DataContext>();
         services.AddTransient<IObjectiveRepository, ObjectiveRepository>();
         services.AddTransient<IResultRepository, ResultRepository>();
         services.AddScoped<IDbInitializer, DbInitializer>();
      }
   }
}
```

«User.cs»

```
namespace TimeTeacher.Models
{
   public class User : IdentityUser
   {
      public string Description {get; set;}
      public ICollection<Result> Results { get; set; }
   }
}
```

«IdentityUser»

```
namespace Microsoft.AspNetCore.Identity
{

   public class IdentityUser<TKey> where TKey : IEquatable<TKey>
   {
      public IdentityUser();

      public IdentityUser(string userName);

      public virtual DateTimeOffset? LockoutEnd { get; set; }
```

```
[PersonalData]
public virtual bool TwoFactorEnabled { get; set; }

[PersonalData]
public virtual bool PhoneNumberConfirmed { get; set; }

[ProtectedPersonalData]
public virtual string PhoneNumber { get; set; }

public virtual string ConcurrencyStamp { get; set; }

public virtual string SecurityStamp { get; set; }

public virtual string PasswordHash { get; set; }

[PersonalData]
public virtual bool EmailConfirmed { get; set; }

public virtual string NormalizedEmail { get; set; }

[ProtectedPersonalData]
public virtual string Email { get; set; }

public virtual string NormalizedUserName { get; set; }

[ProtectedPersonalData]
public virtual string UserName { get; set; }

[PersonalData]
public virtual TKey Id { get; set; }

public virtual bool LockoutEnabled { get; set; }

public virtual int AccessFailedCount { get; set; }
```

```
        public override string ToString();
    }
}
```

«BaseController.cs»

```
namespace TimeTeacherWeb.Controllers.Api
{
  [Authorize(Roles = "Player,Admin")]
  public class BaseController : Controller
    {
    }
}
```

«AccountController.cs»

```
namespace TimeTeacherWeb.Controllers.Api
{

  [Route("api/account")]
  public class AccountController : BaseController
    {
      private readonly SignInManager<User> _signInManager;
      private readonly UserManager<User> _userManager;
      private readonly RoleManager<IdentityRole> _roleManager;
      private readonly IConfiguration _config;

      public AccountController(SignInManager<User> signInManager, UserManager<User> userManager,
RoleManager<IdentityRole> roleManager, IConfiguration config)
      {
        _signInManager = signInManager;
        _userManager = userManager;
        _roleManager = roleManager;
        _config = config;
      }

    public async Task<Microsoft.AspNetCore.Identity.SignInResult> SignInUser(UserViewModel user)
    {
```

```
  return await _signInManager.PasswordSignInAsync(user.UserName, user.UserPassword, false, false);
}

[AllowAnonymous]
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] UserViewModel user)
{
  if (user == null)
    return Ok(new BaseViewModel
    {
      Errors = new List<ErrorViewModel> {
                new ErrorViewModel {PropertyName = "Login" ,PropertyErrorMessage = "User is null"}
              }
    });

  if (!ModelState.IsValid)
    return ModelStateNotValid(ModelState);

  var userExist = await SignInUser(user);

  if (userExist.Succeeded)
  {
    var currentUserInfo = await _userManager.FindByNameAsync(user.UserName);
    var currentUserRoles = (List<string>)(await _userManager.GetRolesAsync(currentUserInfo));

    var userForToken = new UserViewModel
    {
      UserName = currentUserInfo.UserName,
      UserId = currentUserInfo.Id,
      UserRoles = currentUserRoles
    };

    string token = CreateJWTToken(userForToken);

    return Ok(new JwtTokenViewModel
```

```
    {
      Token = token
    });
  }

  return Ok(new BaseViewModel
  {
    Errors = new List<ErrorViewModel> {
              new ErrorViewModel {PropertyName = "Login" ,PropertyErrorMessage = "User is not exist"}
            }
  });
}

private string CreateJWTToken(UserViewModel user)
{
  var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["JwtSettings:JwtKey"]));
  var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

  DateTime issuedAt = DateTime.UtcNow;
  DateTime expireDays = DateTime.Now.AddDays(Convert.ToInt64(_config["JwtSettings:ExpireDays"]));

  var claims = new[]
  {
          new Claim(ClaimTypes.Name, user.UserName),
          new Claim(JwtRegisteredClaimNames.NameId, user.UserId),

      }.ToList();
  claims.AddRange(user.UserRoles.Select(_ => new Claim(ClaimTypes.Role, _)));

  ClaimsIdentity claimsIdentity = new ClaimsIdentity(claims);

  var tokenHandler = new JwtSecurityTokenHandler();

  var token = (JwtSecurityToken)tokenHandler.CreateJwtSecurityToken(
     subject: claimsIdentity,
```

```
            notBefore: issuedAt,
            expires: expireDays,
            signingCredentials: creds);

        return tokenHandler.WriteToken(token);
    }

    private IActionResult ModelStateNotValid(ModelStateDictionary modelState)
    {
        List<ErrorViewModel> errors = new List<ErrorViewModel>();
        foreach (var modelError in modelState)
        {
            string propertyName = modelError.Key;
            if (modelError.Value.Errors.Count > 0)
            {
                foreach (var error in modelError.Value.Errors)
                    errors.Add(new ErrorViewModel
                    {
                        PropertyName = propertyName,
                        PropertyErrorMessage = error.ErrorMessage
                    });
            }
        }
        return Ok(new BaseViewModel { Errors = errors });
    }
  }
}
```

«UserController.cs»

```
namespace TimeTeacherWeb.Controllers.Api
{
    [Route("api/users")]
    [Authorize(Roles = "Admin")]
    public class UserController : BaseController
    {
```

```
        private readonly ITaskService _taskService;
        private readonly UserManager<User> _userManager;

        public UserController(ITaskervice taskService,UserManager<User> userManager)
        {
            _taskService = taskService;
            _userManager = userManager;
        }

        [HttpGet("getAllUsers")]
        public async Task<IActionResult> GetAllUsersList()
        {
            var users = _taskService.GetAllUsers();
            var usersForViewModel = new List<UserViewModel>();

            foreach (var user in users)
                usersForViewModel.Add(new UserViewModel
                {
                    UserId = user.Id,
                    UserName = user.UserName,
                    UserRoles = (List<string>)(await _userManager.GetRolesAsync(user))
                });

            return Ok(new UsersViewModel { Users = usersForViewModel });
        }

        [HttpGet("existUserRoles")]
        public IActionResult Roles()
        {
            return Ok(new RolesViewModel { Roles = _taskService.GetNameRoles() });
        }

    }
}
```

«UserTaskController.cs»

```
namespace TimeTeacherWeb.Web.Controllers.Api
{
    [Route("api/userTasks") ]
    public class UserTaskController : BaseController
    {
        private readonly ITaskService _taskService;

        private readonly UserManager<User> _userManager;
        public UserTaskController(ITaskService taskService, UserManager<User> userManager)
        {
            _taskService = taskService;
            _userManager = userManager;
        }

        [Authorize(Roles = "Admin")]
        [HttpGet("allUsersTask")]
        public IActionResult GetAllTasks()
        {
            List<Task> listTasks = _taskService.GetAllTasks();
            List<TaskViewModel> userTasks = new List<TaskViewModel>();

            foreach (var task in listTasks)
            {
                userTasks.Add(new TaskViewModel()
                {
                    Id = task.Id,
                    Description = task.Description,
                    Date = task.Date,
                    UserId = task.UserId
                });
            }

            return Ok(new UserTasksViewModel { Tasks = userTasks });
        }
```

```
[Authorize(Roles = "Admin")]
[HttpPost("selectedUserTasks")]
public async Task<IActionResult> GetUserTasks([FromBody]UserViewModel user)
{
    if (user == null)
        return Ok(new List<ErrorViewModel> {
            new ErrorViewModel {PropertyName = "GetUserTasks" ,PropertyErrorMessage = "User is null"}
        });

    var currentUserInfo = await _userManager.FindByNameAsync(user.UserName);

    if (string.IsNullOrEmpty(currentUserInfo.Id))
        return Ok(new List<ErrorViewModel> {
        new ErrorViewModel {PropertyName = "GetUserTasks" ,PropertyErrorMessage = "User is not exist in DB"}
    });

    List<Task> listTasks = _taskService.GetUserTasks(currentUserInfo.Id);
    List<TaskViewModel> userTaskss = new List<TaskViewModel>();

    foreach (var task in listTasks)
    {
        userTasks.Add(new TaskViewModel()
        {
            Id = task.Id,
            Description = task.Description,
            Date = task.Date,
            UserId = task.UserId
        });
    }
    return Ok(new UserTasksViewModel { Tasks = userTasks });
}

[HttpPost("addTask")]
public async Task<IActionResult> AddTask([FromBody] TaskViewModel task)
```

```
{
    if (task == null)
        return Ok(new List<ErrorViewModel> {
            new ErrorViewModel {PropertyName = "AddTask" ,PropertyErrorMessage = "Task is null"}
        });
    var  user = await _userManager.GetUserAsync(HttpContext.User);
    var userId = task.UserId ?? user.Id;

    _taskService.AddTask(new Task
    {
        Id = task.Id,
        Date = task.Date,
        Description = task.Description,
        UserId = userId
    });
    return Ok(new BaseViewModel());
}

[HttpGet]
public async Task<IActionResult> GetCurrentUserTasks()
{
    var currentUser = await _userManager.GetUserAsync(HttpContext.User);
    List<Task> listTasks = _taskService.GetUserTasks(currentUser.Id);
    List<TaskViewModel> userTasks = new List<TaskViewModel>();

    foreach (var task in listTasks)
    {
        userTasks.Add(new TaskViewModel()
        {
            Id = task.Id,
            Description = task.Description,
            Date = task.Date,
            UserId = task.UserId
        });
    }
```

```
            return Ok(new UserTasksViewModel { Tasks = userTasks });
        }

        [HttpPost("deleteTasks")]
        public IActionResult DeleteTasks([FromBody] UserTasksViewModel tasks)
        {
            if (tasks == null)
                return Ok(new List<ErrorViewModel> {
                new ErrorViewModel {PropertyName = "DeleteTasks" ,PropertyErrorMessage = "Tasks is null"}
            });

            List<Task> listTasks = new List<Task>();
            foreach (var task in tasks.Tasks)
            {
                listTasks.Add(new Task
                {
                    Id = task.Id,
                    Description = task.Description,
                    Date = task.Date,
                    UserId = task.UserId
                });
            }

            _taskService.DeleteTasks(listTasks);
            return Ok(new BaseViewModel());
        }

    }
}
```

«Program.cs»

```
namespace TimeTeacherWeb
{
    public class Program
    {
        public static void Main(string[] args)
```

```
        {
            CreateWebHostBuilder(args).Build().Run();
        }

        public static IWebHostBuilder CreateWebHostBuilder(string[] args) =>
            WebHost.CreateDefaultBuilder(args)
                .UseStartup<Startup>();
    }
}
```

«Startup.cs»

```
namespace TimeTeacherWeb
{
    public class Startup
    {
        public void ConfigureServices(IServiceCollection services)
        {
      services.AddMvc();
      services.AddCore();
      services.AddCors(options =>
      {
        options.AddPolicy("CorsPolicy",
            builder => builder.AllowAnyOrigin()
                .AllowAnyMethod()
                .AllowAnyHeader()
                .AllowCredentials()
                .Build());
      });
    }
        public void Configure(IApplicationBuilder app, IHostingEnvironment env, IDbInitializer dbInitializer)
        {
      if (env.IsDevelopment())
      {
        app.UseDeveloperExceptionPage();
      }
```

```
    app.UseStaticFiles();
    app.UseDefaultFiles();
    app.UseMvc(routes =>
    {
      routes.MapRoute(
        name: "home",
        template: "{*url}",
        defaults: new { controller = "Home", action = "Index" });
    });

    Task.Run(async () => { await dbInitializer.Initialize(); }).Wait();
  }
  }
}
```

«HomeController.cs»

```
namespace AltaCrowd.FrontOffice.Web.Controllers
{
  [AllowAnonymous]
  public class HomeController : Controller
  {
    public IActionResult Index()
    {
      return View("~/wwwroot/index.html");
    }
  }
}
```

«index.html»

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>TimeTeacherWeb</title>
  <base href="/">

  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <tt-root></tt-root>
</body>
</html>
```

«header.component.ts»

```
@Component({
  selector: 'tt-header',
  templateUrl: './header.component.html',
  styleUrls: ['./header.component.scss']
})
export class HeaderComponent implements OnInit {

  constructor(public languageService: LanguageService,
              private router: Router) { }

  languages = Languages;
  mobileMenuClosed = true;

  ngOnInit() {

    this.router.events.subscribe((evt) => {
      if (!(evt instanceof NavigationEnd)) {
          return;
      }
      window.scrollTo({ top: 0, behavior: 'smooth' });
    });

  }

  toggleLang(lang) {
    this.languageService.setLanguage(lang);
  }
```

```
  getKeys( obj: Object): string[] {
    if (obj) {
      return Object.keys(obj);
    }
  }

}
```

«header.component.html»

```
<section id="header_section" class="header-section">
  <div class="container-fluid h-100">
    <div class="row h-100">

      <div class="col-12 w-100 h-100 header-mobile-menu">
        <div class="row h-100">

          <div class="col-4 d-flex align-items-center justify-content-center">
            <div class="site-logo">
              <a [routerLink]="['/home']">
                <img src="/assets/images/header-logo.svg" alt="Alta Club">
              </a>
            </div>
          </div>

          <div class="col-8">
            <div class="row h-100">
              <div class="col-4 d-flex align-items-center justify-content-end">
                <div class="d-block" style="white-space: nowrap;">
                  <div [ngClass]="languageService.getLanguage() == languages[0]?'lang__active':"" class="d-inline-block
translate__lang"  (click)="toggleLang(languages[0]); $event.stopPropagation();">
                    EN
                  </div>
                  <div class="d-inline-block lang__delimeter">
                    /
                  </div>
```

```
                <div [ngClass]="languageService.getLanguage() == languages[1]?'lang__active':""  class="d-inline-block translate__lang" (click)="toggleLang(languages[1]); $event.stopPropagation();">
                  UA
                </div>
              </div>
            </div>

            <div class="col-8 d-flex align-items-center justify-content-center">
              <tt-logined class="w-100"></tt-logined>
            </div>
          </div>
        </div>

      </div>
    </div>
  </div>
</section>
```

«account.module.ts»

```
const storageService: StorageService = new StorageService();

@NgModule({
  declarations: [
    LoginComponent,
    RegistrationComponent,
    AccountComponent
  ],
  imports: [
    AccountRouting,
    SharedModule,
    RecaptchaFormsModule,
    RecaptchaModule.forRoot(),
  ],
  providers: [
    AccountService,
    {
```

```
    provide: RECAPTCHA_LANGUAGE,
    useValue: storageService.get('TimeTeacherWebLanguage') ? storageService.get('TimeTeacherWebLanguage') : 'en',
  },
  {
    provide: RECAPTCHA_SETTINGS,
    useValue: { siteKey: environment.reCaptchaKey  } as RecaptchaSettings
  },
 ]
})
export class AccountModule { }
```

«header.component.scss»

```
.header-section {
  height: 80px;
  width: 100%;
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  display: block !important;
  z-index: 10;
  border-bottom: 1px solid #EEEDF0;
  overflow: unset;
  background-color: #fff;
  transition: background-color .5s ease-out;
  color: #000;

  .site-logo {
    width: 100%;
    height: 100%;
    max-width: 150px;
    min-width: 130px;
    max-height: 30px;
    display: block;
    img {
      width: 100%;
```

```
        height: 100%;
      }
    }

    .translate__lang {
      padding: 10px;
      &:hover {
        cursor: pointer;
      }
    }

    .lang__delimeter {
      color: blue;
    }

    .lang__active {
      color: blue!important;
      font-weight: 700;
    }
}
```

«login.component.html»

```
<section  id="loginSection" class="main-section main-page-height d-flex align-items-center account-section" >
  <div class="container">
     <div class="row">
        <div class="col-12 text-center">
           <div class="account-form login-form">
              <form  [formGroup]="loginForm" (ngSubmit)="onSubmitForm()" novalidate>
                <div class="row">
                   <div class="col-12 text-uppercase">
                      <h4>{{'loginPage.title' | translate}}</h4>
                   </div>
                </div>
                <div class="row form-group">
                   <div class="col-12">
                      <label for="login_email" type="text">{{'loginPage.email' | translate}}</label>
```

```
            </div>
            <div class="col-12">
              <input id="login_email" type="email" class="form-control" pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$" formControlName="email" placeholder="{{'loginPage.emailPlaceholder' | translate}}">
              <div class="invalid-feedback" role="alert" [style.display]="(!buttonClick || isValidFormField('email'))?'none':'block'">{{'loginPage.emailValidation' | translate}}</div>
            </div>
          </div>
          <div class="row form-group">
            <div class="col-12">
              <label for="login_password" type="text">{{'loginPage.password' | translate}}</label>
            </div>
            <div class="col-12">
              <input id="login_password" type="password" class="form-control" formControlName="password" placeholder="{{'loginPage.passwordPlaceholder' | translate}}">
              <div class="invalid-feedback" role="alert" [style.display]="(!buttonClick || isValidFormField('password'))?'none':'block'">{{'loginPage.passwordValidarion' | translate}}</div>
            </div>
          </div>
          <div class="row form-group">
            <div class="col-6 text-left">
              <label for="rememberMe">{{'loginPage.rememberMe' | translate}}</label>
            </div>
            <div class="col-6 text-right">
              <div  class="checkSlide">
                <input id="rememberMe" type="checkbox" formControlName="rememberMe"/>
                <label for="rememberMe"></label>
              </div>
            </div>
          </div>
          <div class="row form-group">
            <div class="col-12">
              <a [routerLink]="['/account/forgot-password']">{{'loginPage.forgotPassword' | translate}}</a>
            </div>
          </div>
          <div class="row form-group">
            <div class="col-12">
```

```
                    <button type="submit" [disabled]="loginForm.pristine" class="btn lg-btn text-uppercase">{{'loginPage.submit' | translate}}</button>
                  </div>
                </div>

                <div class="row form-group">
                  <div class="col-12">
                    <p>{{'loginPage.haveAccount' | translate}} <a [routerLink]="['/account/registration']">{{'loginPage.registration' | translate}}</a></p>
                  </div>
                </div>
              </form>
            </div>
          </div>
        </div>
      </div>
    </section>
```

«login.component.ts»

```
@Component({
  selector: 'tt-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent implements OnInit, OnDestroy {
  transparent = false;
  buttonClick = false;
  loginForm: FormGroup;
  phone_number: string;

  constructor(private fb: FormBuilder,
              private accountService: AccountService,
              private router: Router,
              private notificationService: NotificationService) {}

  ngOnInit() {
```

```
  this.createForm();
}

createForm() {
  this.loginForm = this.fb.group({
    email: ['', [Validators.required, Validators.email] ],
    password: ['', [Validators.required] ],
    rememberMe: [false]
  });
}

ngOnDestroy() {
}

private validateAllFormFields(formGroup: FormGroup) {
  Object.keys(formGroup.controls).forEach(field => {
    const control = formGroup.get(field);
    if (control instanceof FormControl) {
      control.markAsDirty({ onlySelf: true });
    } else if (control instanceof FormGroup) {
      this.validateAllFormFields(control);
    }
  });
}

isValidFormField(name: string): boolean {
  return !(this.loginForm.get(name).invalid && this.loginForm.get(name).dirty);
}

onSubmitForm() {
  this.buttonClick = true;
  if (this.loginForm.valid) {
    this.accountService.loginUser(this.loginForm.value).subscribe((response: any) => {
      this.loginForm.reset();
      this.router.navigate(['home']);
```

```
      });
    } else {
      this.validateAllFormFields(this.loginForm);
    }
  }
}
```

«dashboard.component.html»

```
<section  id="dashboardSection" class="main-section dashboard-section main-page-height d-flex">
   <div class="vertical-menu text-semibold">
      <ul class="menu-content">
         <ng-container *ngFor="let item of getKeys(menu)">
            <li  [routerLink]="menu[item].link">
               <img src="{{menu[item].image}}" alt="">
               <span class="content text-uppercase">{{'dashboardPage.menu.' + menu[item].name | translate }}</span>
            </li>
         </ng-container>
      </ul>
  </div>

   <div class="with-vertical-menu-field">
      <section class="field-content">
         <div class="container-fluid h-100">
            <router-outlet></router-outlet>
         </div>
      </section>
   </div>

</section>

<alc-busy-indicator></alc-busy-indicator>
```

«dashboard.component.ts»

```
@Component({
  selector: 'tt-dashboard',
  templateUrl: './dashboard.component.html',
```

```
  styleUrls: ['./dashboard.component.scss']
})
export class DashboardComponent implements OnInit {

  menu = dashboardMenu;

  constructor() { }

  ngOnInit() {
  }

  getKeys( obj: Object): string[] {
    if (obj) {
      return Object.keys(obj);
    }
  }

}
```

«storage.service.ts»

```
@Injectable()
export class StorageService {
    constructor() { }

    set(id: string, val: string, inLocalScope: boolean = false) {
      if (inLocalScope) {
        localStorage.setItem(id, JSON.stringify(val));
      } else {
        sessionStorage.setItem(id, JSON.stringify(val));
      }
    }

    get(id: string) {
      let val;
      val = localStorage.getItem(id);
      if (!val) {
```

```
      val = sessionStorage.getItem(id);
    }
    try {
      return val ? JSON.parse(val) : null;
    } catch (e) {
      return null;
    }
  }

  remove(id: any) {
    localStorage.removeItem(id);
    sessionStorage.removeItem(id);
  }

  clear() {
    localStorage.clear();
    sessionStorage.clear();
  }
}
```

«error-handler.interceptor.ts»

```
@Injectable()
export class ErrorHandlerInterceptor implements ErrorHandler {

  public ErrorSubject: Subject<string> = new Subject<string>();

  constructor (private inject: Injector) { }

  handleError(exception: any): void {

    const authService = this.inject.get(AuthService);
    const router = this.inject.get(Router);
    const zona = this.inject.get(NgZone);

    const satatus = exception.status;
```

```
    switch (status) {
      case 401: {
        this.ErrorSubject.next(exception.error);
        authService.logOut();
        zona.run(() => {
          router.navigate(['/account']);
        });
      }break;
      case 404: {
        this.ErrorSubject.next(exception.error);
      }break;
      case 403: {
        this.ErrorSubject.next(exception.error);
      }break;
      case 422: {
        this.ErrorSubject.next(exception.error);
      }break;
      case 504: {
        this.ErrorSubject.next(exception.error);
      }break;
      default: {
        console.error(exception);
      }
    }
  }
}
```

«core.module.ts»

```
import { NgModule, Optional, SkipSelf } from '@angular/core';
import { InterceptorsModule } from './interceptors/interceptors.module';
import { TokenService } from './authentication/token.service';
import { StorageService } from './services/storage.service';
import { ApiSrevice } from './http/api.service';
import {HttpClientModule, HttpClient} from '@angular/common/http';
import { RouterModule } from '@angular/router';
```

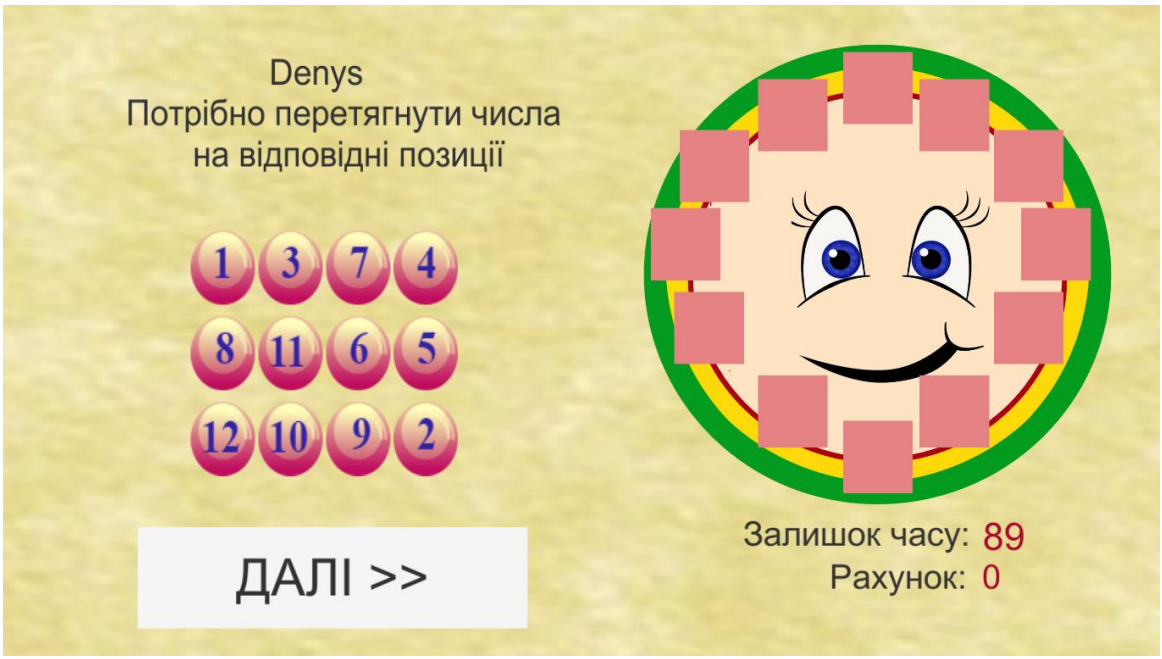
```
import {TranslateModule, TranslateLoader} from '@ngx-translate/core';
import {TranslateHttpLoader} from '@ngx-translate/http-loader';
import {TranslateService} from '@ngx-translate/core';
import { AuthService } from './authentication/auth.service';
import { LanguageService } from './services/language.service';
import { SignInGuard } from './guards/signIn.guard';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
import { BusyIndicatorService } from './services/busy-indicator.service';

export function HttpLoaderFactory(http: HttpClient) {
  return new TranslateHttpLoader(http, './assets/i18n/', '.json');
}

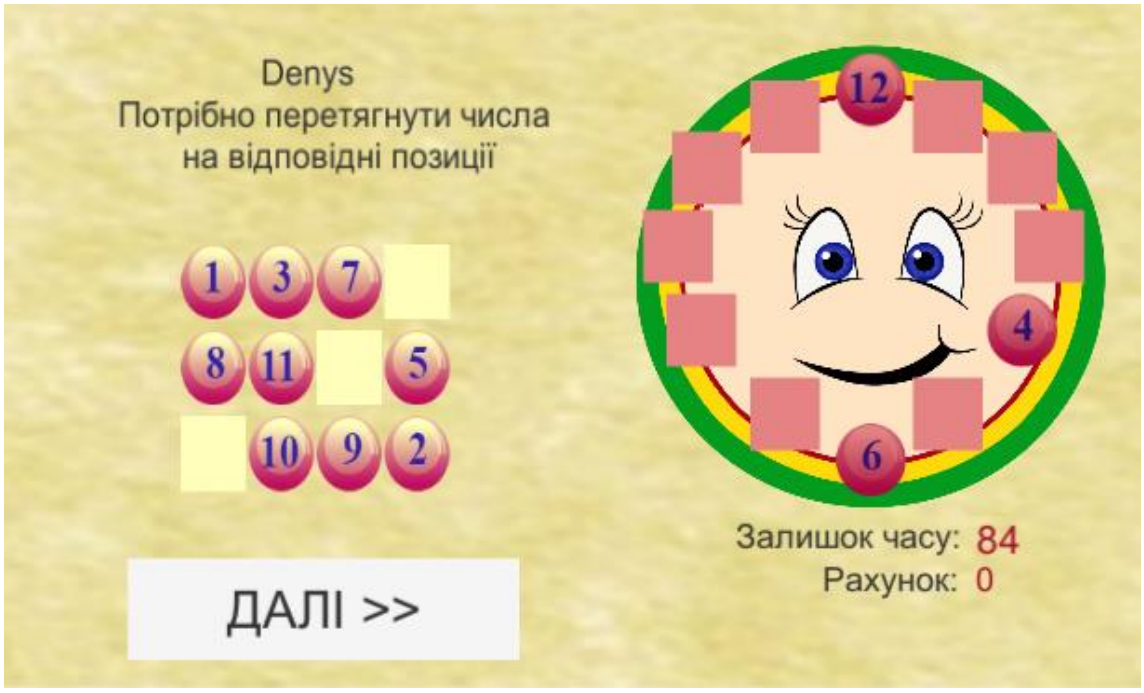
@NgModule({
  declarations: [
  ],
  imports: [
    BrowserAnimationsModule,
    InterceptorsModule,
    HttpClientModule,
    RouterModule,
    TranslateModule.forRoot({
      loader: {
          provide: TranslateLoader,
          useFactory: HttpLoaderFactory,
          deps: [HttpClient]
      }
    })
  ],
  exports: [
  ],
  providers: [
    TokenService,
    StorageService,
```

```
    ApiSrevice,
    TranslateService,
    AuthService,
    LanguageService,
    SignInGuard,
    BusyIndicatorService
   ],
  bootstrap: [ ]
})
export class CoreModule {
    constructor( @Optional() @SkipSelf() parentModule: CoreModule) {
      if (parentModule) {
          throw new Error('CoreModule is already loaded. Import it in the AppModule only');
      }
   }
  }
```

Додаток В. Зображення інтерфейсу ігрового процесу програмного комплексу



а



б

Рисунок В.1 – Інтерфейс модуля «DragNumbers»

а – початок ігри; б – процес проходження гри



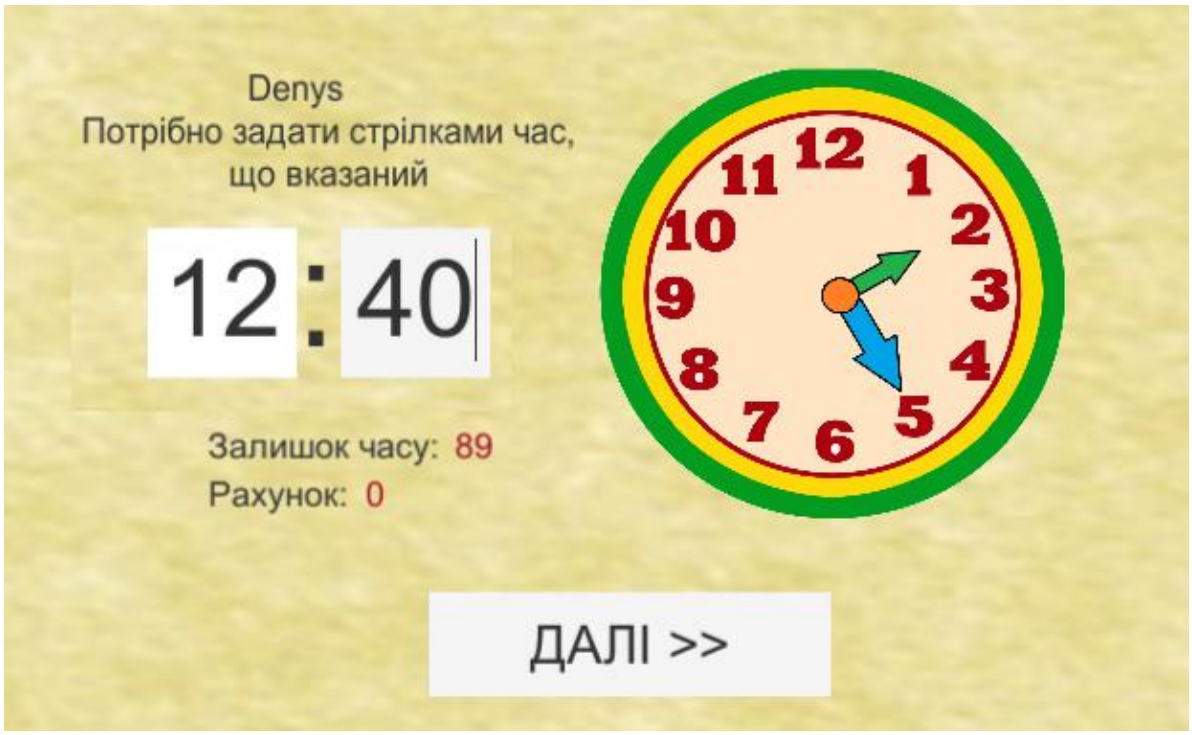
а



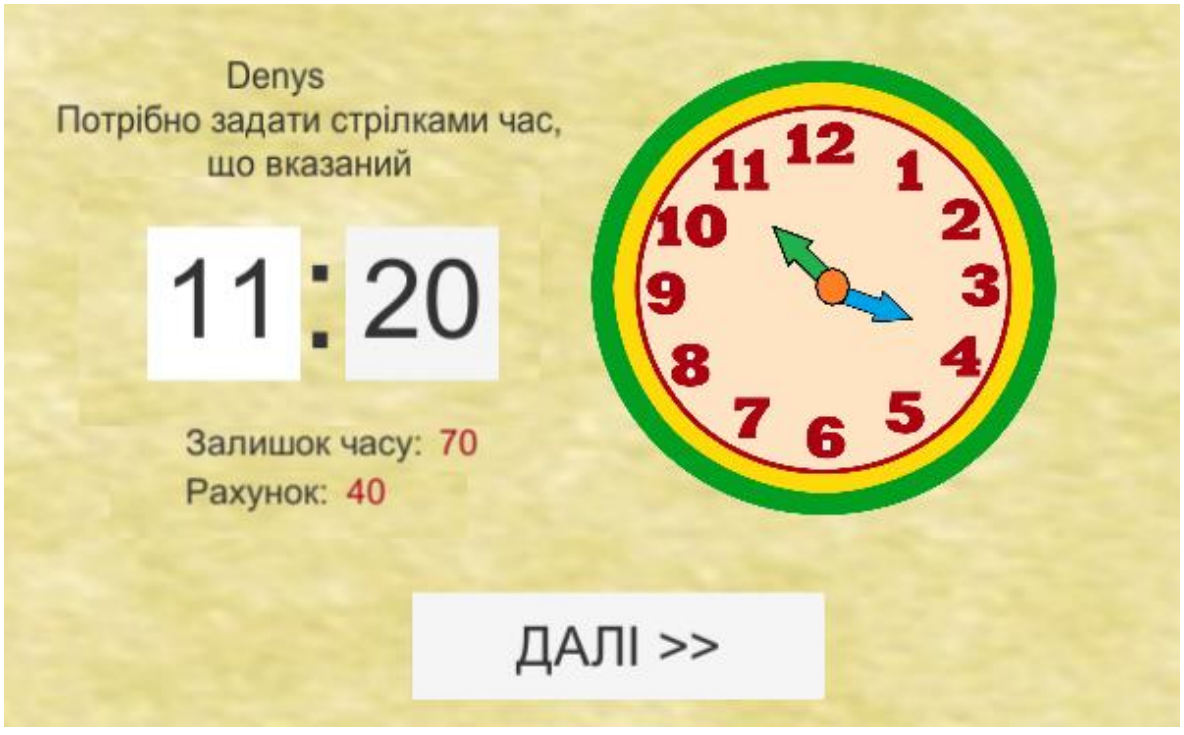
б

Рисунок В.2 – Інтерфейс модуля «TimeArrow»

а – початок ігри; б – процес проходження гри



а



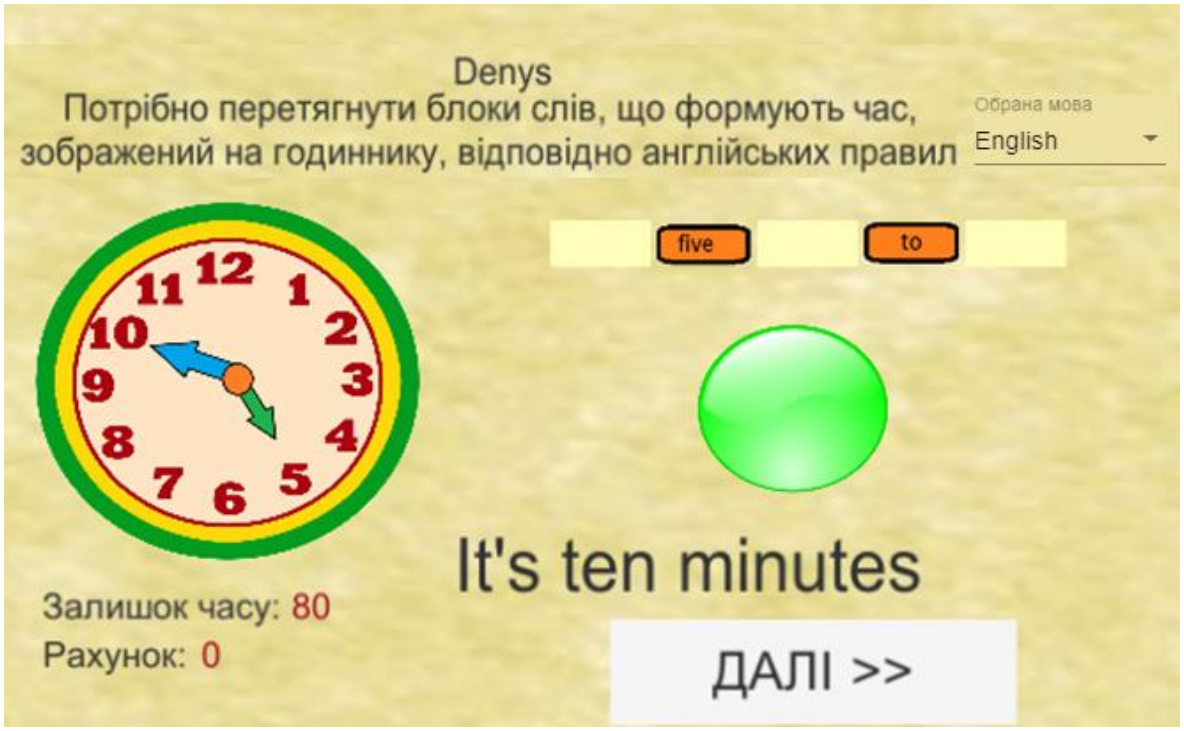
б

Рисунок В.3 – Інтерфейс модуля «ArrowManipulation»

а – початок ігри; б – процес проходження гри



а



б

Рисунок В.4 – Інтерфейс модуля «LanguageTime»

а – початок ігри; б – процес проходження гри

Додаток Г. Ілюстративний матеріал до захисту магістерської кваліфікаційної роботи

Завідувач кафедри ПЗ:

д. т. н., професор _____ А. М. Пєтух

Науковий керівник:

к. т. н., старший викладач кафедри ПЗ _____ Н. П. Бабюк

Рецензент:

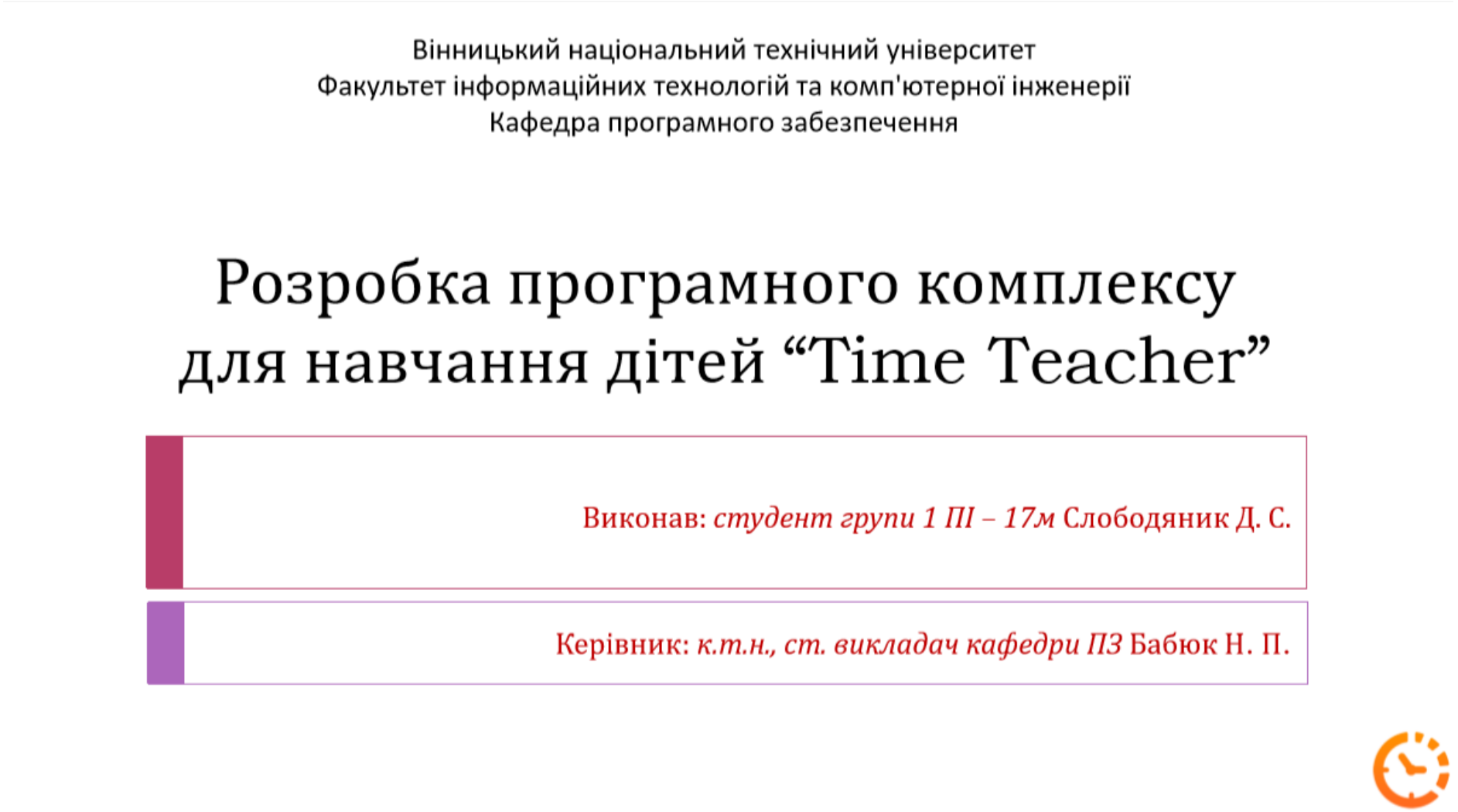
к.т.н., доцент кафедри КН _____ О. К. Колесницький

Нормоконтроль:

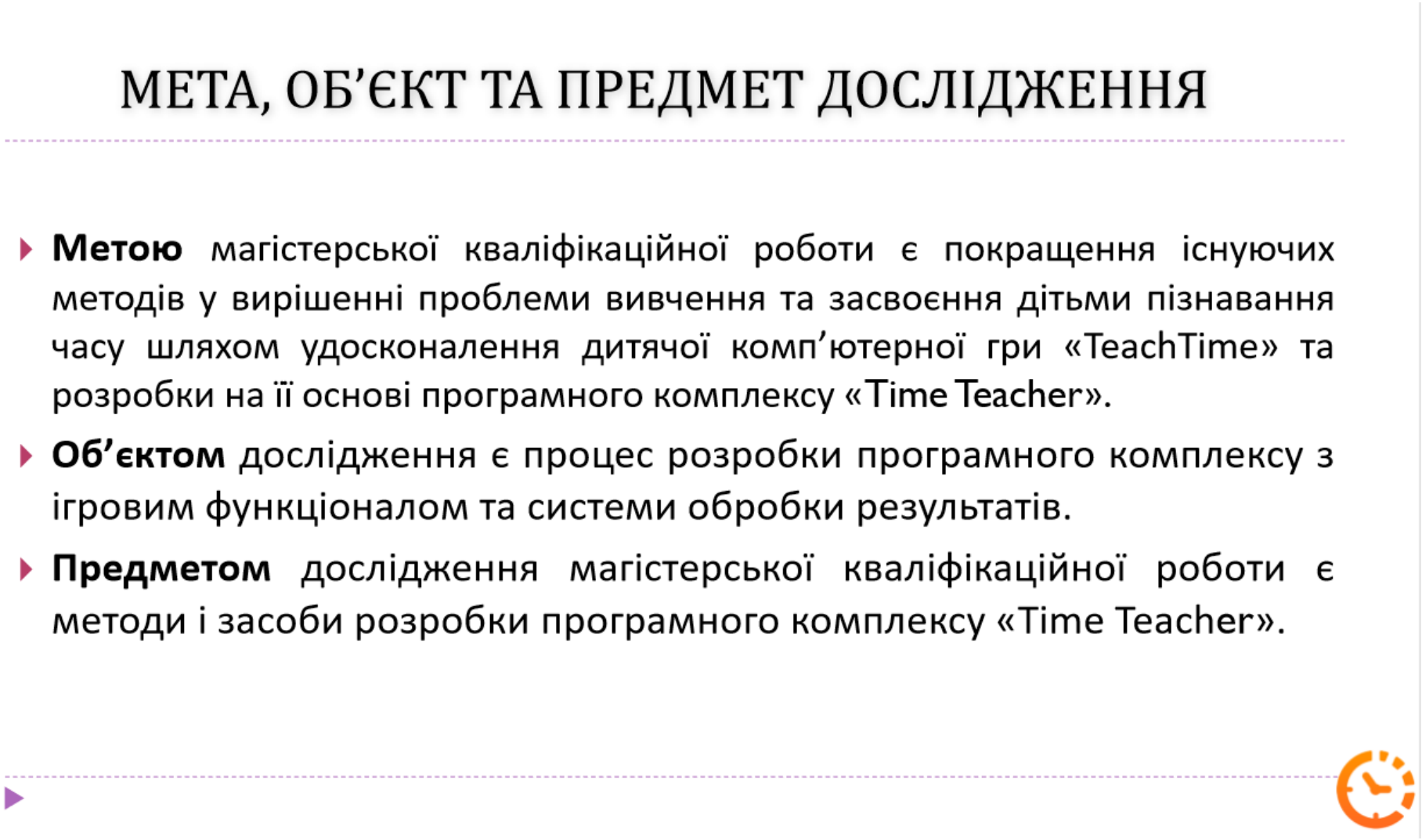
к. т. н., старший викладач кафедри ПЗ _____ Н. П. Бабюк

Виконавець:

студент групи 1ПІ-17м _____ Д. С. Слободяник



Плакат 1 – Тема, автор, науковий керівник магістерської кваліфікаційної роботи



Плакат 2 – Мета, об'єкт та предмет дослідження

НАУКОВА НОВИЗНА ДОСЛІДЖЕННЯ

- **Наукова новизна** полягає у вдосконаленні методів для швидкого пізнання часу дітьми, застосуванні методу вивчення часу на різних мовах світу, зокрема, англійської. Також використано групу методів навчання, зокрема програмоване навчання, що спрямовані на: стимулювання й мотивацію до навчально-пізнавальної діяльності; контроль та корекцію ефективності пізнавання часу дітьми.



Плакат 3 – Наукова новизна дослідження

АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

- Все життя людини тісно пов’язане з часом, з умінням розподіляти, вимірювати і цінувати час.
- Часова послідовність та поняття про тривалість подій засвоюється дітьми з великими труднощами.
- Важливо орієнтуватися у часі відповідно мов світу, зокрема правил англійської мови. Багато хто відкладає вивчення часу на потім, а насправді це дуже важливо, тому що в розмовній мові постійно запитують час один у одного.



Плакат 4 – Аналіз предметної області

ПРОБЛЕМА

- Виявлено проблему пізнання часу дітьми, оскільки навчальний матеріал важко сприймається.
- Все більшої популярності набуває вивчення іноземних мов. Тому вивчення часу відповідно правил мов світу надзвичайно важливе.



Плакат 5 – Проблема

АНАЛІЗ АНАЛОГІВ



“ ГОДИННИК ІЗ ЗОЗУЛЕЮ ”

- Основна мета - вказати відповідний час, представлений на годиннику, у числовому вигляді.
- Ціль – вказати часу максимум правильних відповідей. При кожній правильній відповіді з’являється зозуля у відповідній області.



“ВИВЧАЄМО ЧАС”

- Основна мета - вказати розташування позицій чисел на годиннику.
- Ціль – вказати всі правильні позиції розташування чисел



“READING THE CLOCK”

- Основна мета - вказати розташування позицій стрілок на годиннику відповідно до заданого числового значення.
- Ціль – вказати за певний проміжок часу максимум правильних відповідей.

Плакат 6 – Аналіз аналогів



Плакат 7 – Діаграма варіантів використання



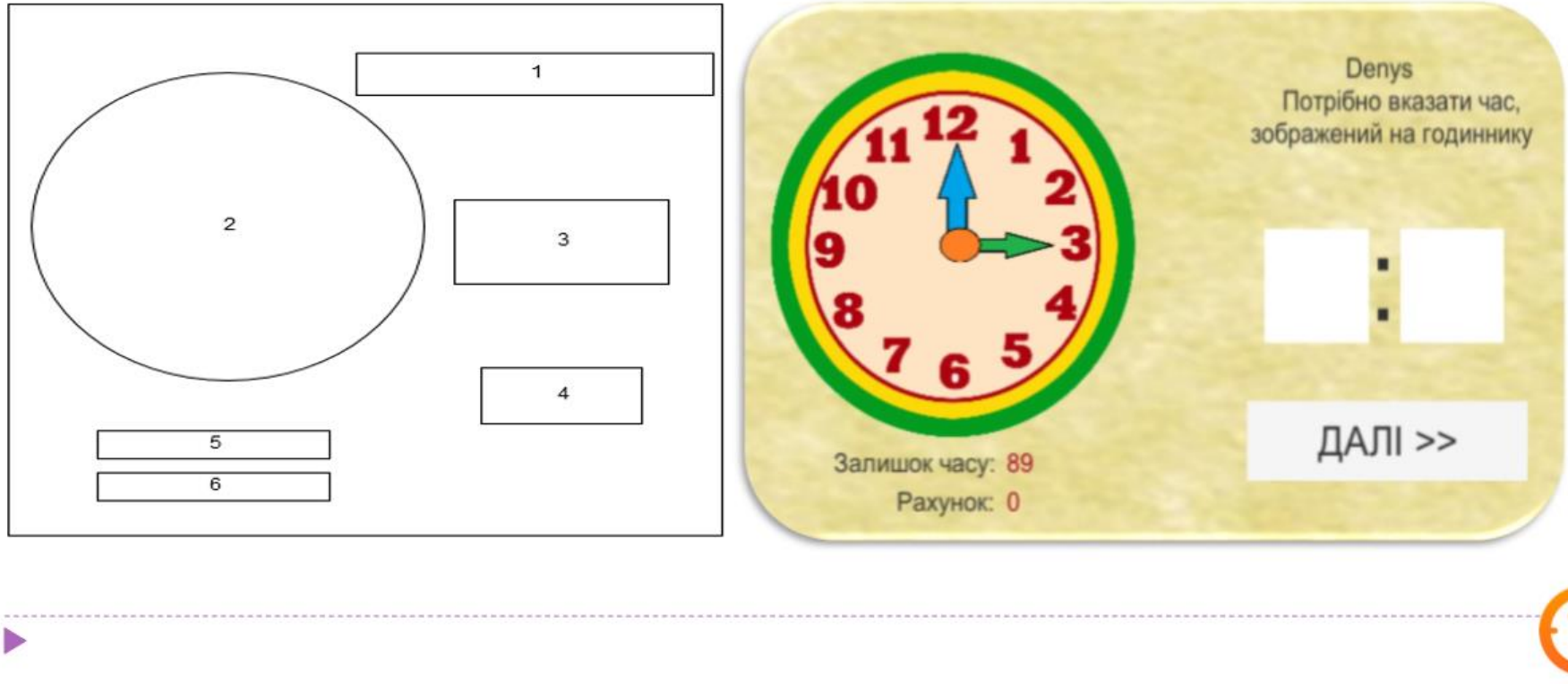
Плакат 8 – Структура бази даних



Плакат 9 – Можливості ігрового функціоналу



Плакат 10 – Перетягування чисел

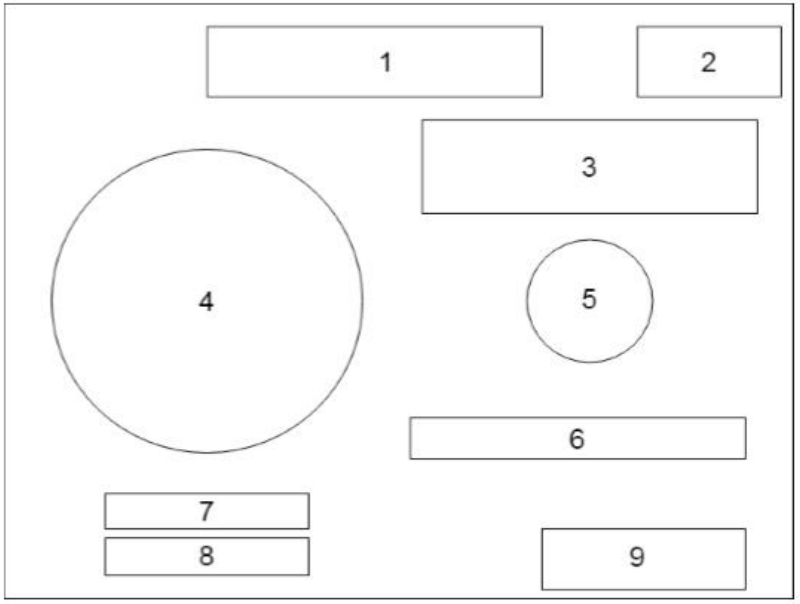


Плакат 11 – Час по стрілках



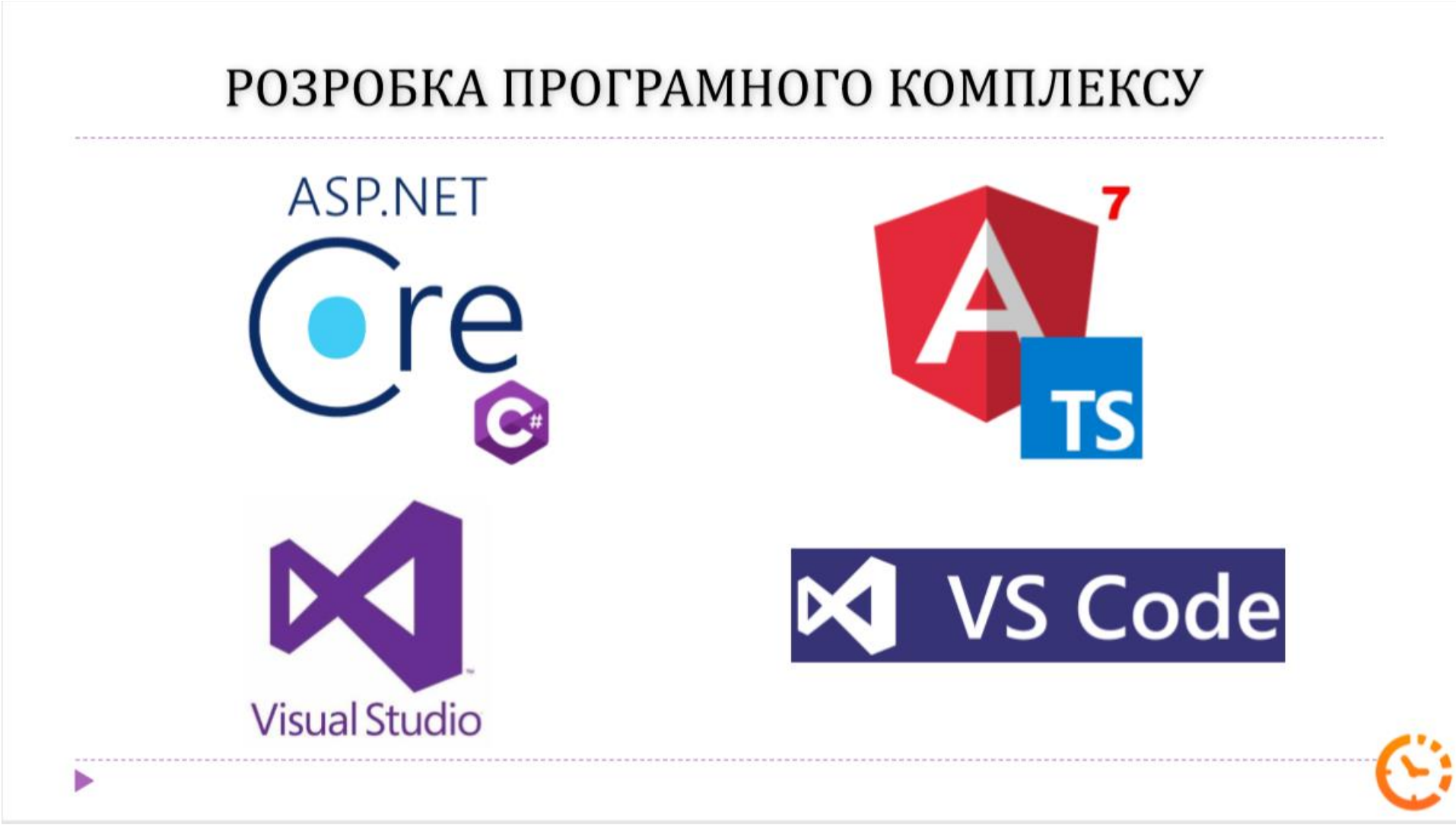
Плакат 12 – Маніпуляції зі стрілками

ЧАС ВІДПОВІДНО ПРАВИЛ ОБРАНОЇ МОВИ





Плакат 13 – Час відповідно правил обраної мови



Плакат 14 – Розробка програмного комплексу



Плакат 15 – Робота ігрового функціоналу (перетягування чисел та час по стрілках)



Плакат 16 – Робота ігрового функціоналу (маніпуляції зі стрілками та час відповідно правил обраної мови)

ЕКОНОМІЧНЕ ОБГРУНТУВАННЯ

- Загальні витрати на розробку - 35 384,17 грн.
- Вартість чистого прибутку ПП – 368 944,03 грн (> 0).
- Абсолютна ефективність вкладених інвестицій Еабс – 333 559,86 грн (> 0).
- Відносна (щорічна) ефективність вкладених інвестицій Ев – 118% (>30%).
- Період окупності Ток - 0.84 року (< 3 років).

- Враховуючи всі фактори, фінансування наукової розробки є доцільним.

Плакат 17 – Економічне обгрунтування

АПРОБАЦІЯ РЕЗУЛЬТАТІВ

- XLVI Науково-технічна конференція підрозділів Вінницького національного технічного університету (2017). Розглянуто тему: «Розробка дитячої комп’ютерної гри «TeachTime».
- Міжнародна науково-практична Інтернет-конференція «Молодь в технічних науках: дослідження, проблеми, перспективи» (2017). Розглянуто тему: «Розробка методу вивчення часу на англійській мові для дитячої комп'ютерної гри «TeachTime».
- Отримано свідоцтво про авторське право на твір «Дитяча комп'ютерна гра «TeachTime» №73041.

Плакат 18 – Апробація результатів

ВИСНОВКИ

- Виявлено, що проблема навчання дітей пізнання часу є актуальною.
- Оптимізовано підхід до вивчення орієнтації в часі – “Час відповідно правил обраної мови” та використано при розробці методу програмованого навчання.
- Розроблено алгоритми роботи модулів додатку “Time Teacher”.
- Спроектовано базу даних програмного комплексу.
- Розроблено та проведено тестування роботи ігрового функціоналу “Time Teacher”.
- Обґрунтовано економічну ефективність розробки програмного комплексу.



Плакат 19 – Висновки



Плакат 20 – Завершення